肥猫商城系统详细设计文档

文档信息

项目名称	肥猫商城系统
文档版本	v2.2
编写日期	2026年11月1日
文档类型	详细设计文档
URL	https://github.com/kangxuebing/Hot_Mall
适用范围	系统开发、测试、部署

目录

1. 系统概述
2. 系统架构设计
3. 功能模块设计
4. 数据库设计
5. 业务流程设计
6. 时序图设计
7. 状态图设计
8. POS收银模块设计
9. 人脸识别模块设计
10. 系统安全设计
11. 性能优化设计
12. 部署运维设计
13. 附录
14. 2026年11月系统升级变更

功能模块设计子目录

3. 功能模块设计

- 3.1 用户管理模块
- 3.2 商品管理模块
- 3.3 订单管理模块
- 3.4 支付管理模块
- 3.5 购物车模块
- 3.6 抽奖营销模块
- 3.7 POS收银模块
- 3.8 供应商管理模块
- 3.9 库存管理模块
- 3.10 人脸识别模块
- 3.11 会员管理模块
- 3.12 促销商品管理模块

- 3.13 业务授权模块

---

1. 系统概述

1.1 系统简介

肥猫商城系统是一个基于Django框架开发的综合性电商平台，集成了商品管理、订单处理、支付结算、会员管理、抽奖营销、POS收银、供应商管理等核心功能。系统采用前后端分离架构，支持多终端访问，提供完整的电商解决方案。

1.2 系统目标

- 提供稳定、高效的电商交易平台
- 支持多种支付方式和订单管理
- 实现用户会员体系和积分管理
- 提供营销活动功能（抽奖、优惠券等）
- 支持线下POS收银系统
- 实现供应商和库存管理
- 支持人脸识别登录等创新功能

1.3 技术栈

后端技术

- **框架**: Django 3.2.25
- **数据库**: MySQL 8.0
- **缓存**: Redis 6.0
- **搜索**: Whoosh (全文搜索)
- **异步任务**: Celery
- **人脸识别**: face_recognition
- **支付**: 支付宝、微信支付

前端技术

- **框架**: Vue.js 2.x
- **UI组件**: Bootstrap 4
- **图表**: ECharts
- **模板引擎**: Jinja2
- **构建工具**: Webpack

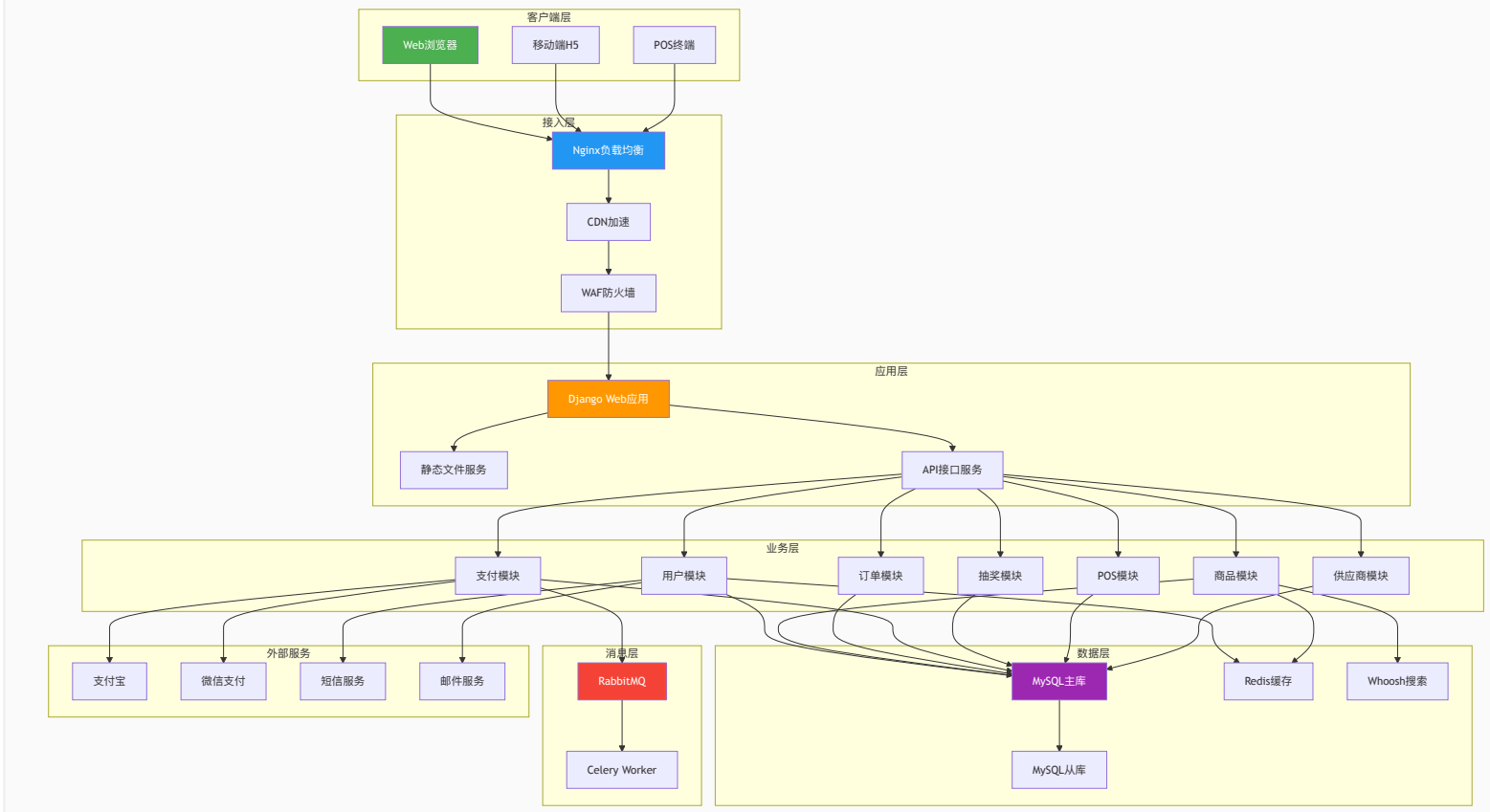
部署技术

- **Web服务器**: Nginx 1.18
- **应用服务器**: uWSGI 2.0
- **容器化**: Docker & Docker Compose
- **监控**: Prometheus + Grafana

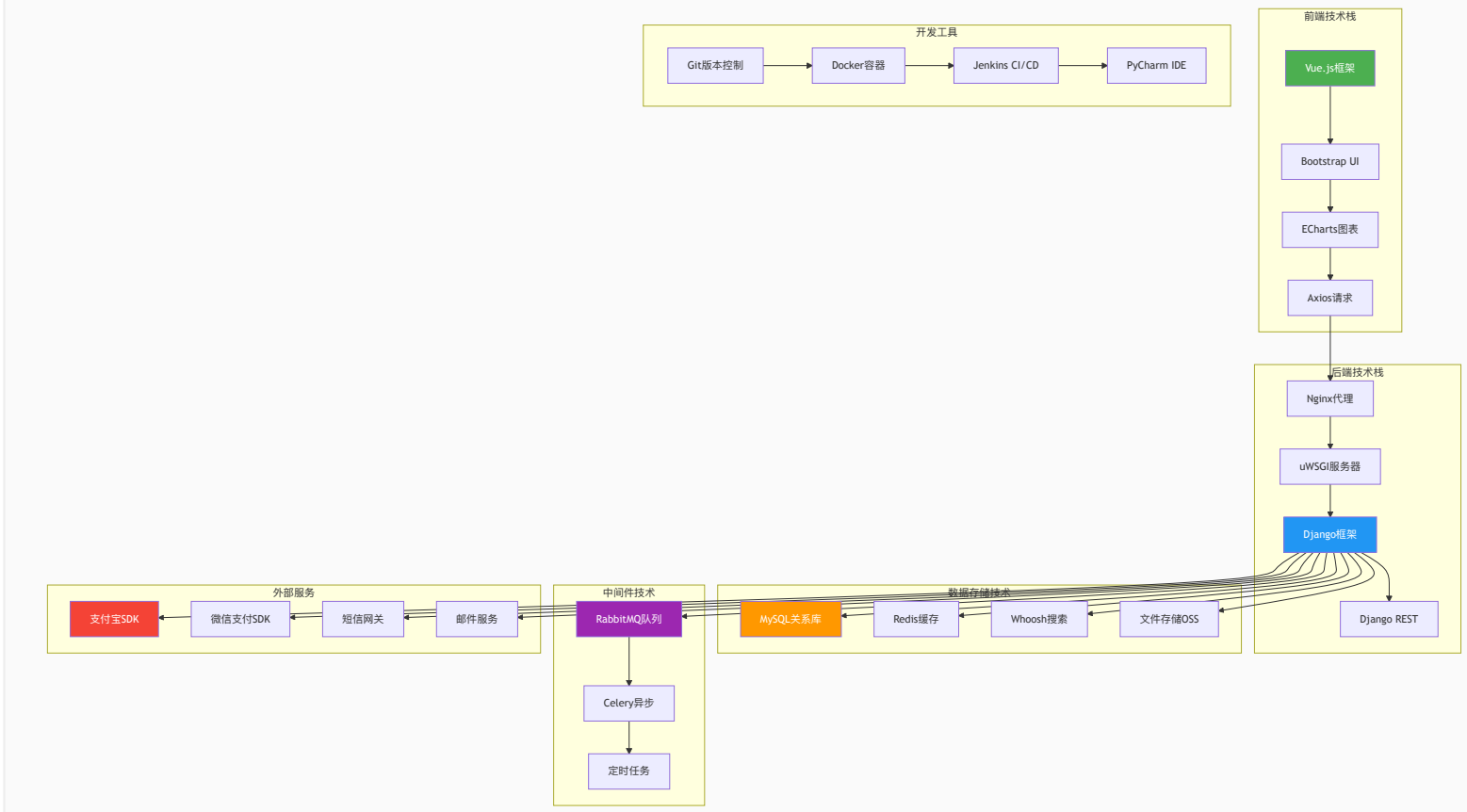
---

2. 系统架构设计

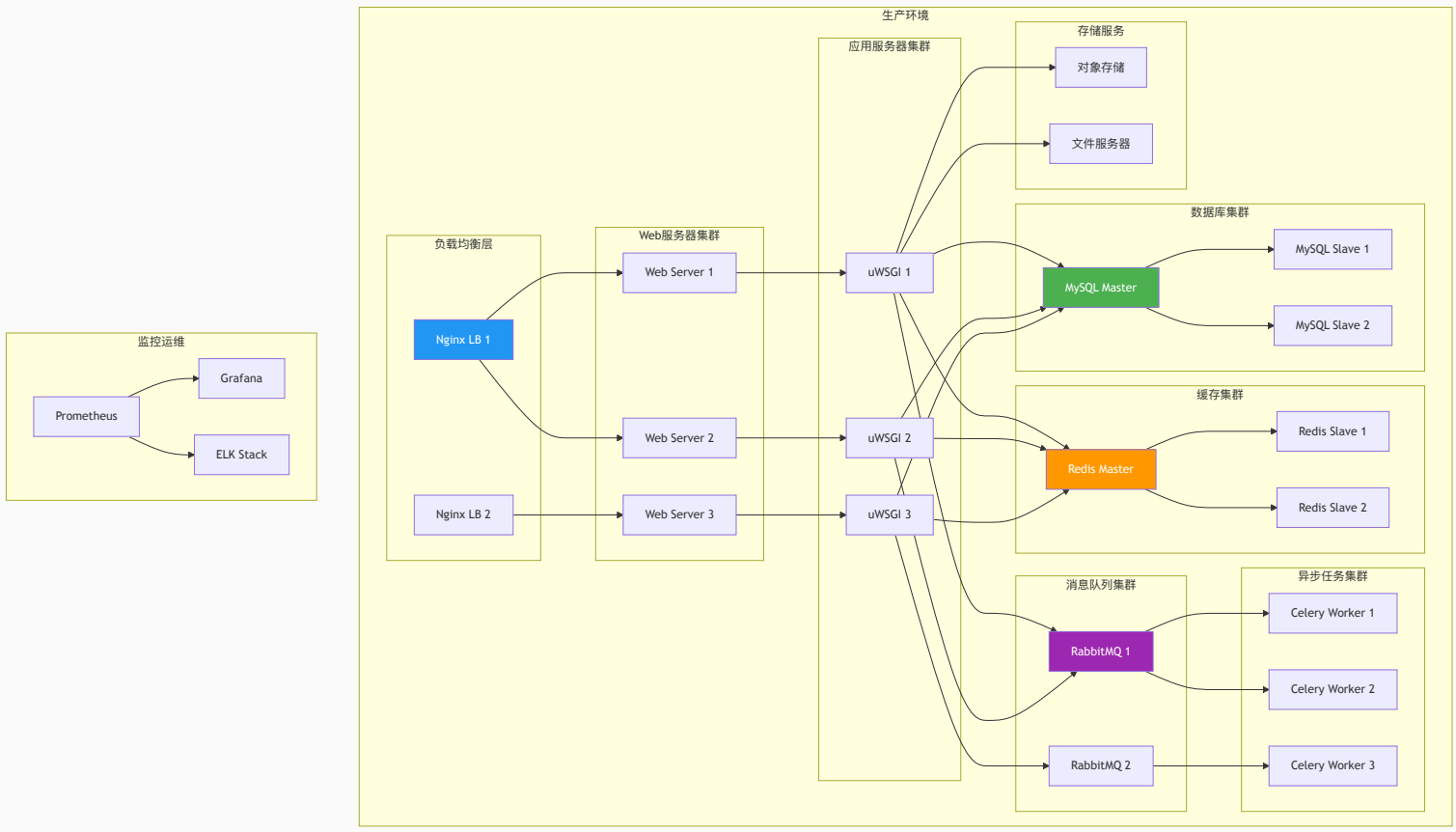
2.1 总体架构图



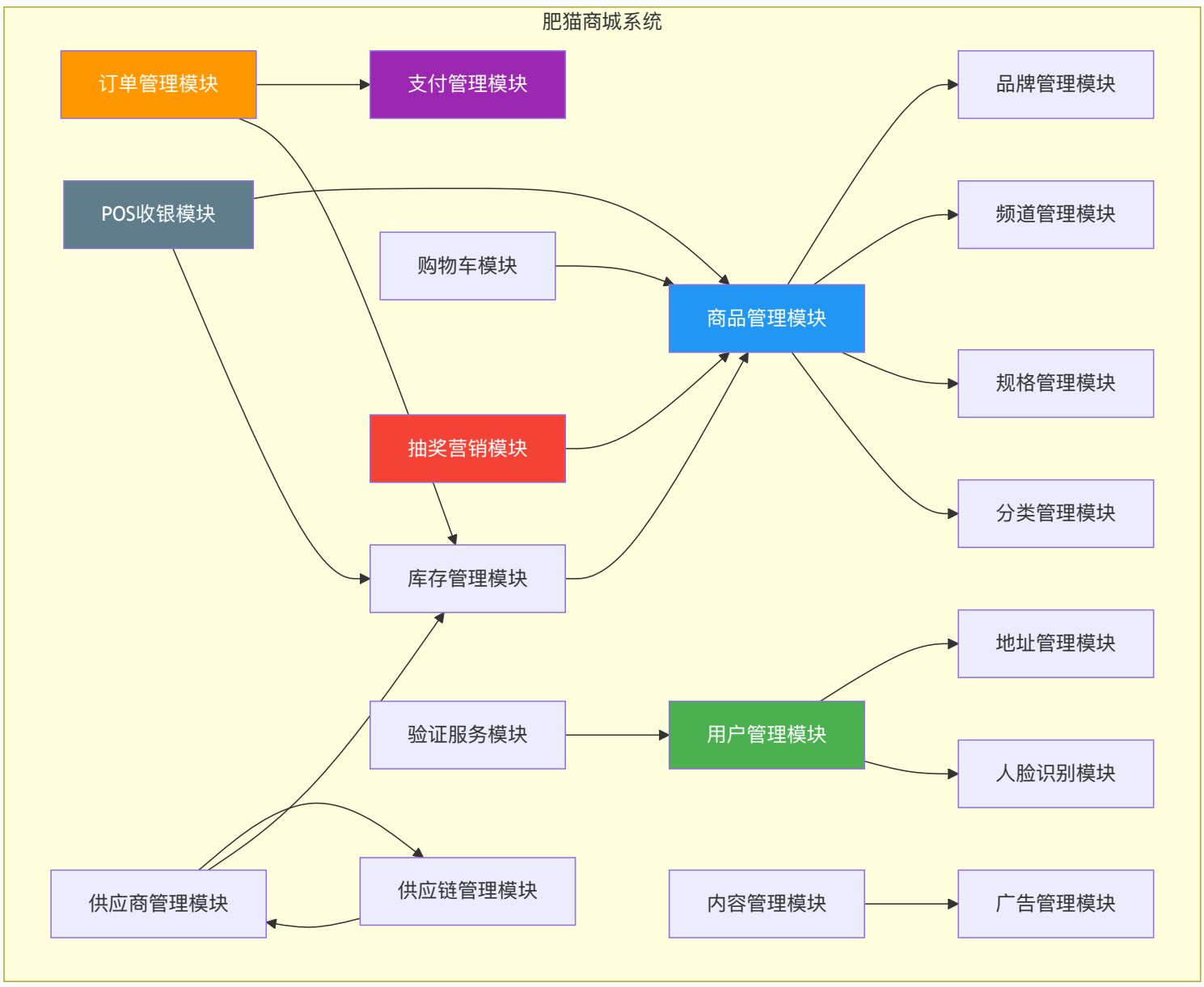
2.2 技术架构图



2.3 部署架构图



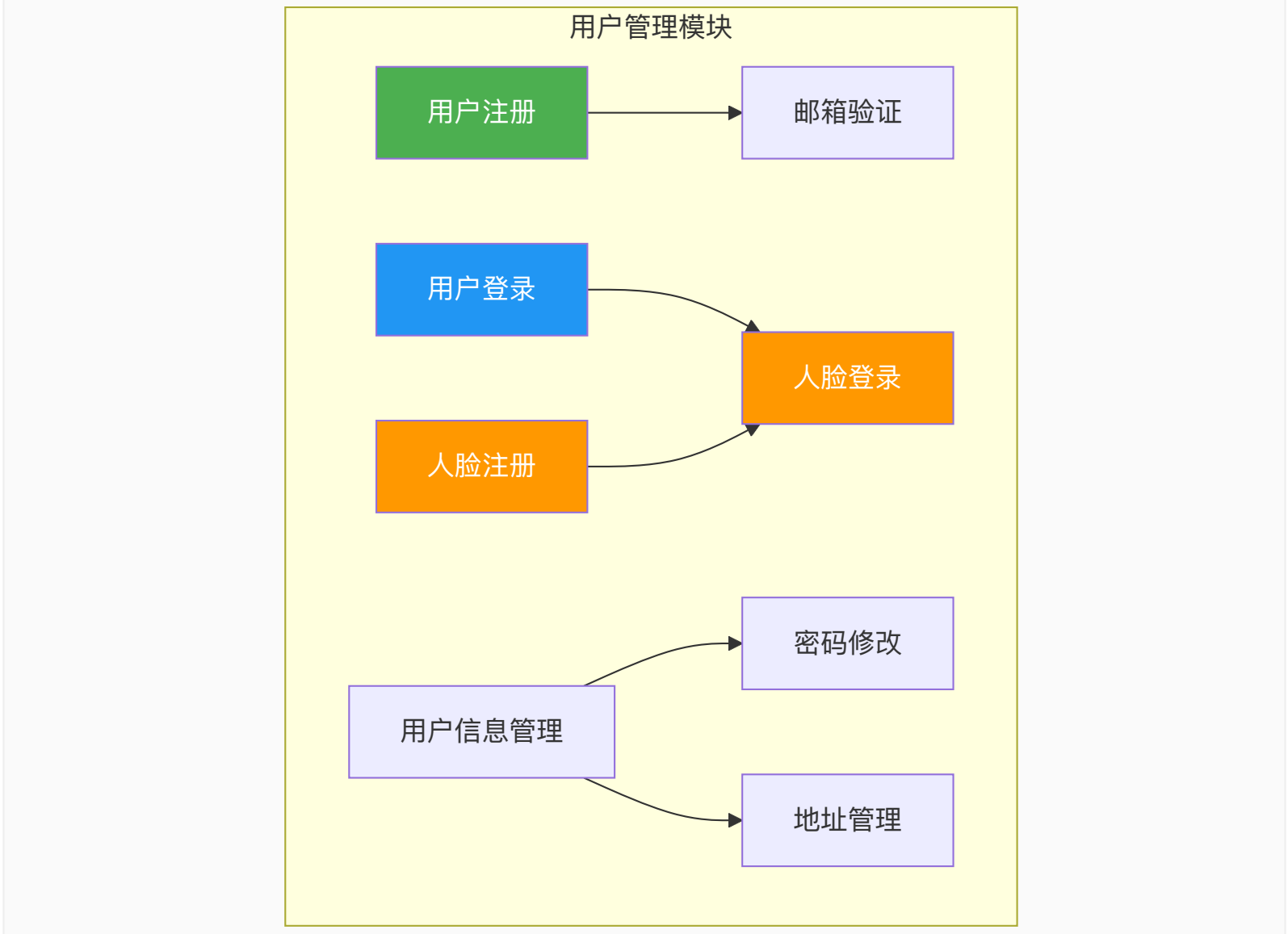
2.4 功能模块图



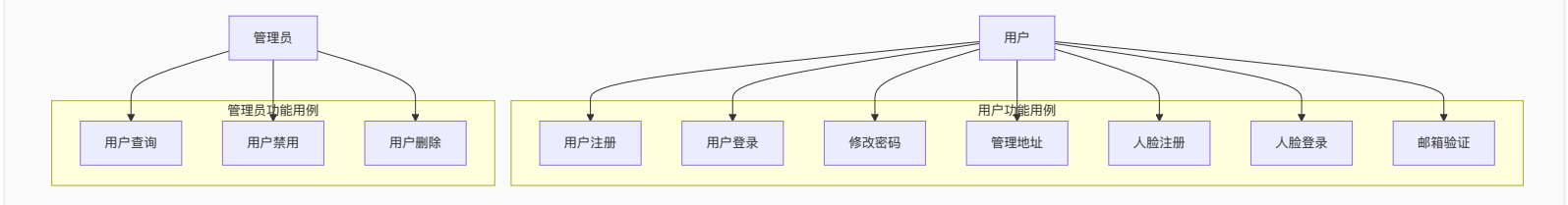
3. 功能模块设计

3.1 用户管理模块

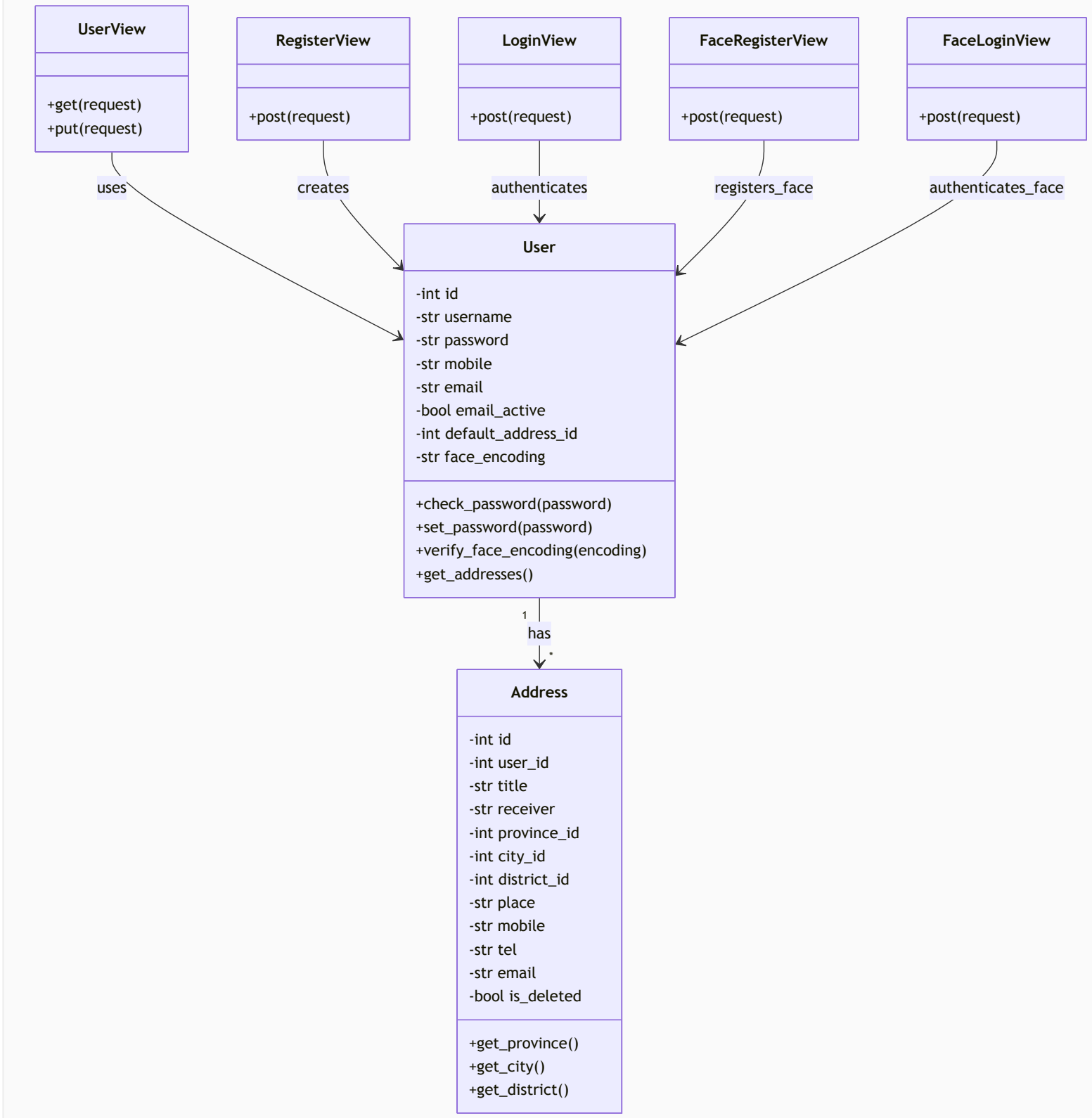
3.1.1 用户模块功能图



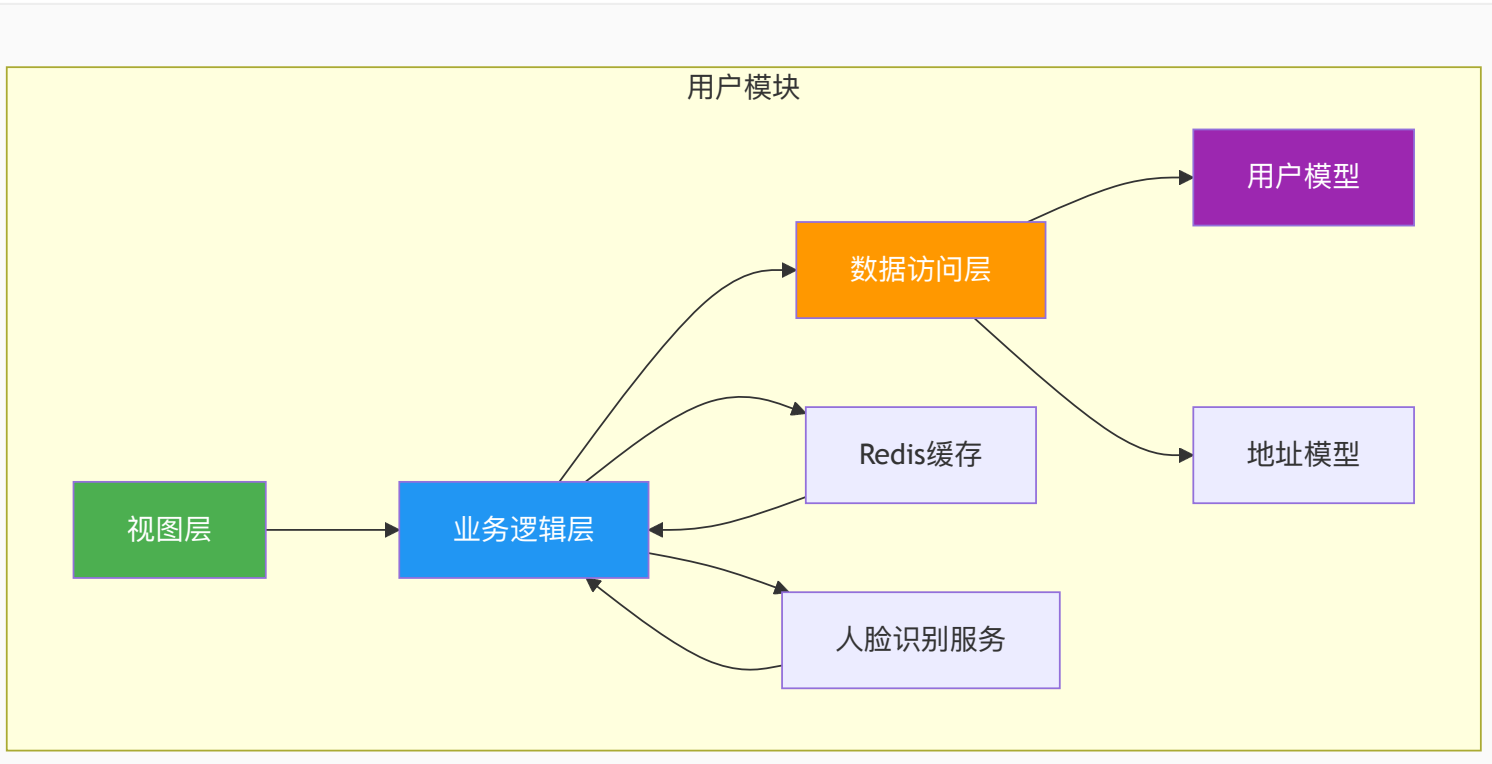
3.1.2 用户模块用例图



3.1.3 用户模块类图

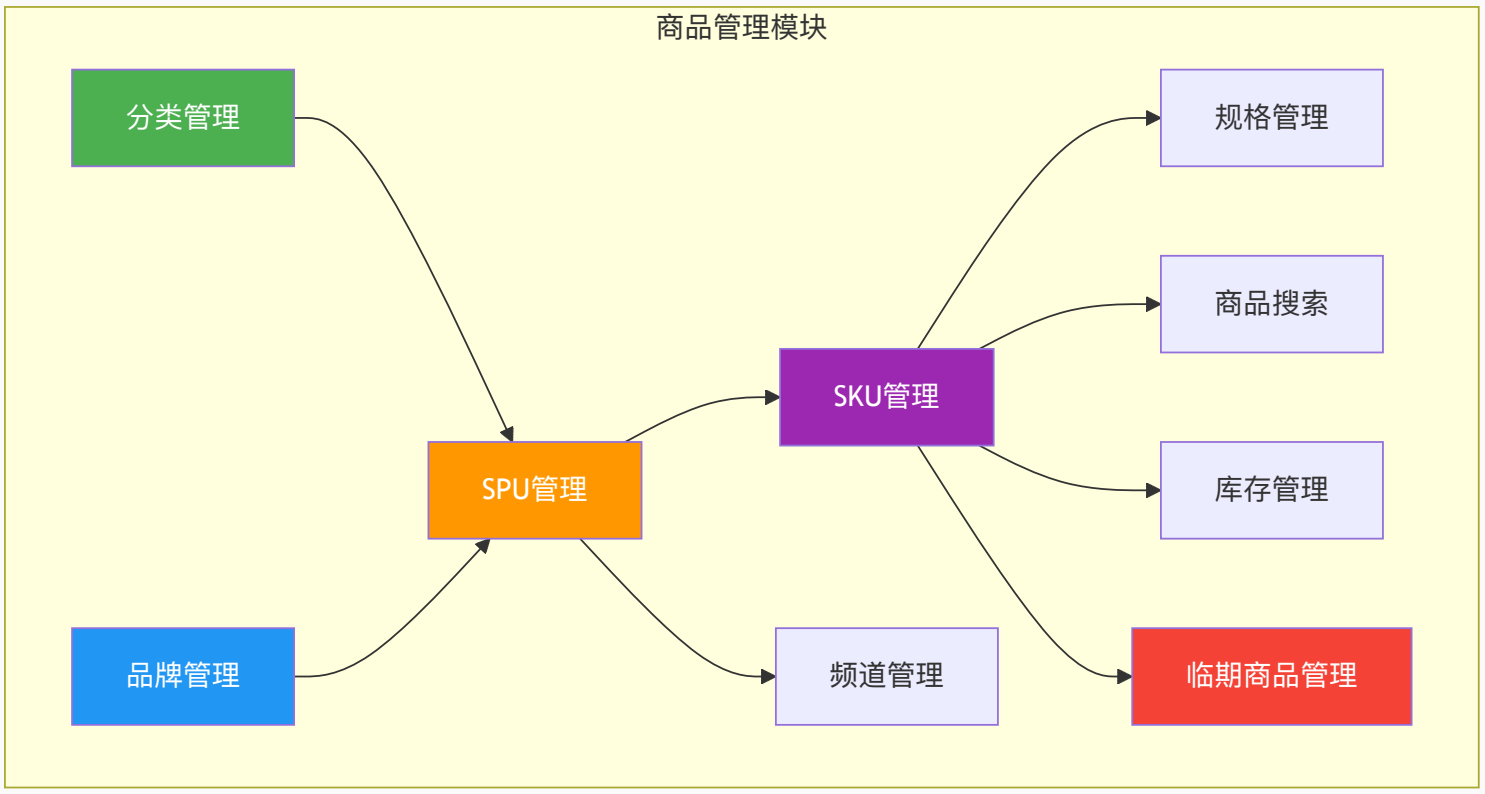


3.1.4 用户模块内部交互拓扑图

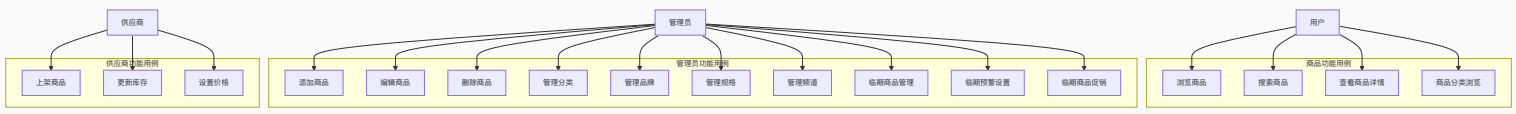


3.2 商品管理模块

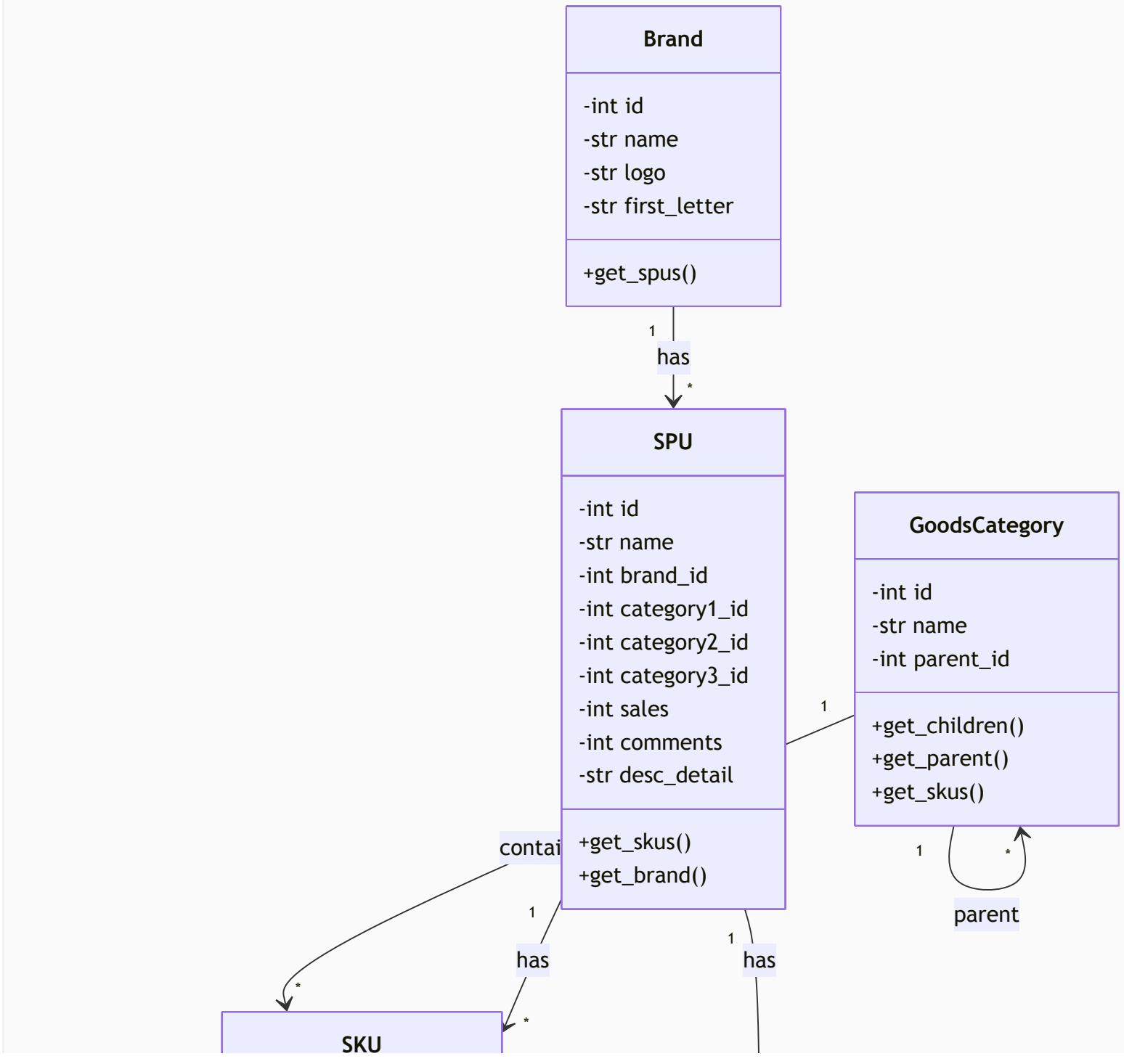
3.2.1 商品模块功能图

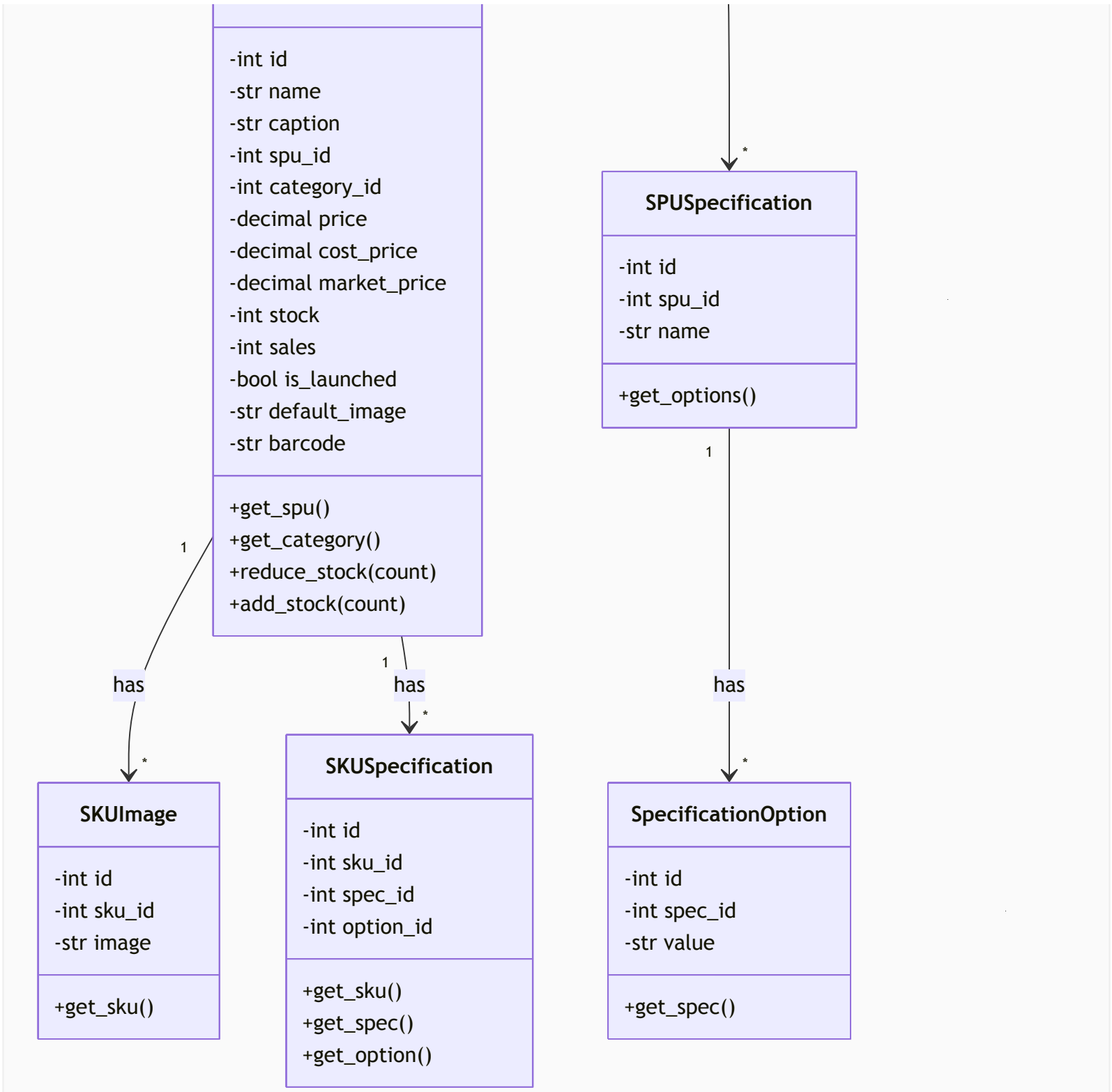


3.2.2 商品模块用例图

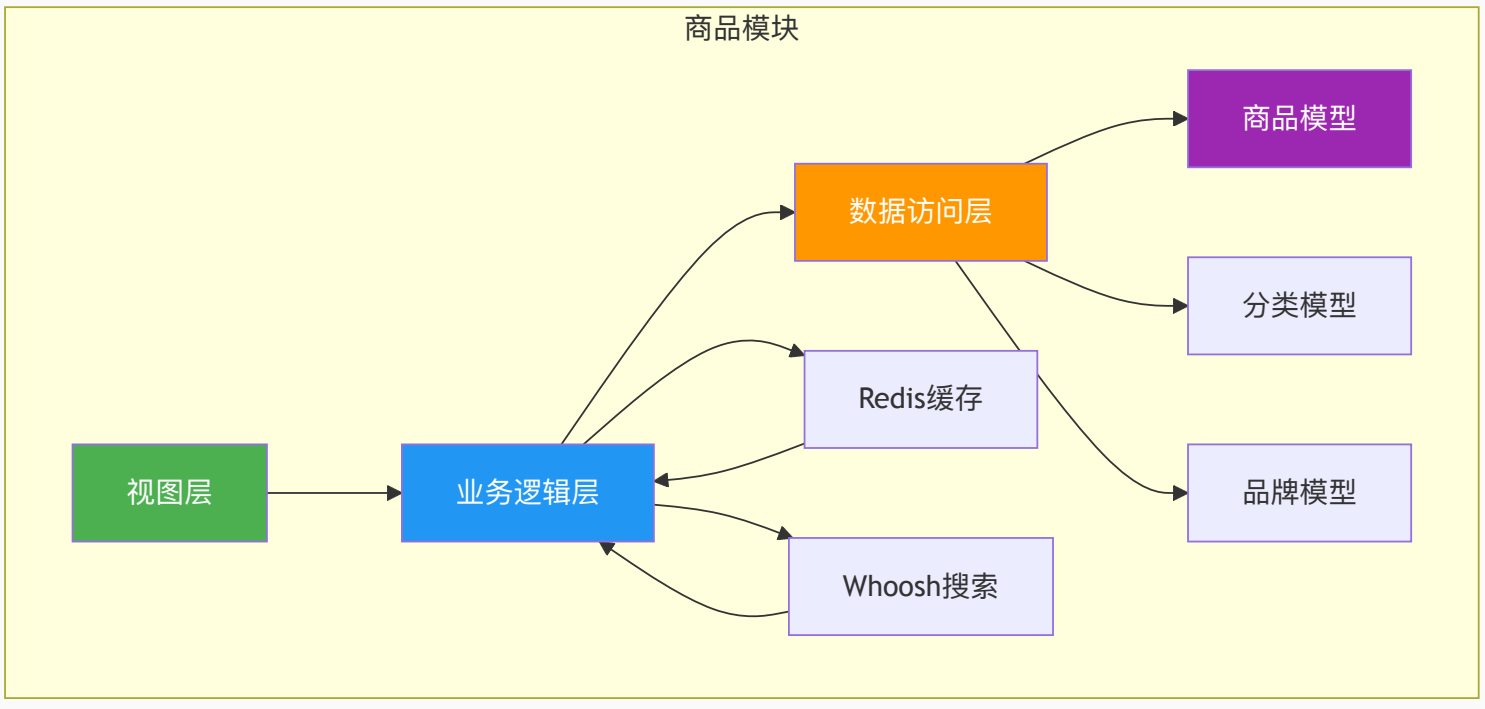


3.2.3 商品模块类图





3.2.4 商品模块内部交互拓扑图



3.2.5 临期商品管理功能设计

3.2.5.1 临期商品业务说明

临期商品管理是指对即将过期的商品进行监控、预警、促销和处理的完整管理流程。系统根据商品的生产日期和保质期自

动计算过期日期，对临期商品进行分类管理，帮助商家减少库存损失。

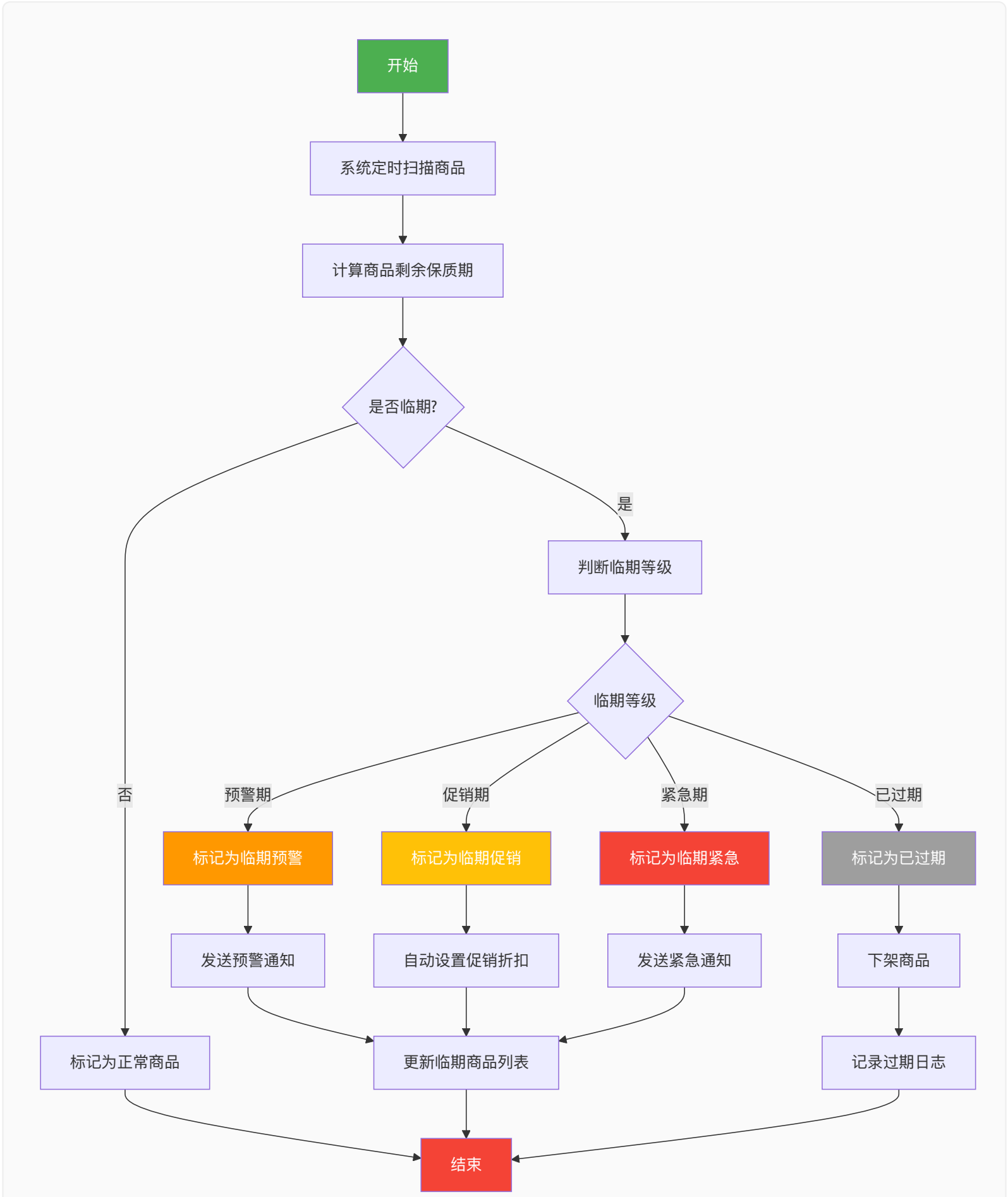
业务场景

- 商品即将过期需要降价促销
- 顾客需要查看临期商品优惠信息
- 管理员需要设置临期预警规则
- 系统自动推送临期商品提醒

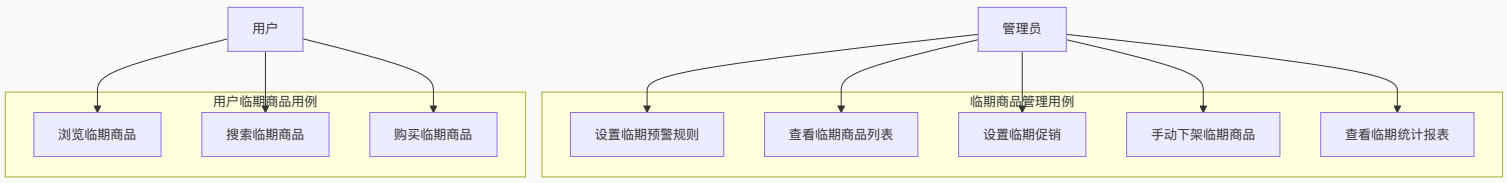
临期分类

- **临期预警期**: 距离过期30天内的商品
- **临期促销期**: 距离过期15天内的商品
- **临期紧急期**: 距离过期7天内的商品
- **已过期**: 已超过过期日期的商品

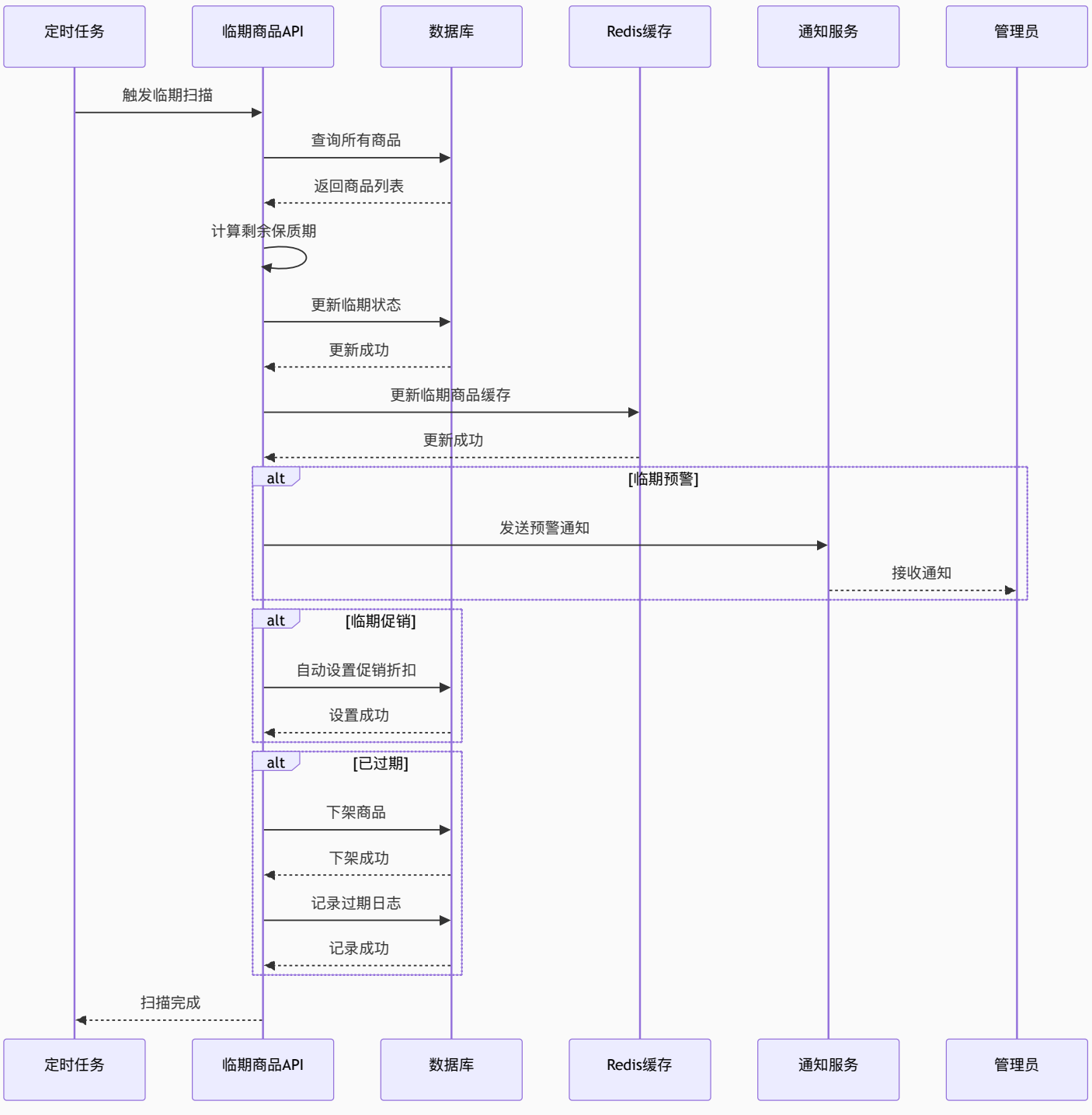
3.2.5.2 临期商品管理流程图



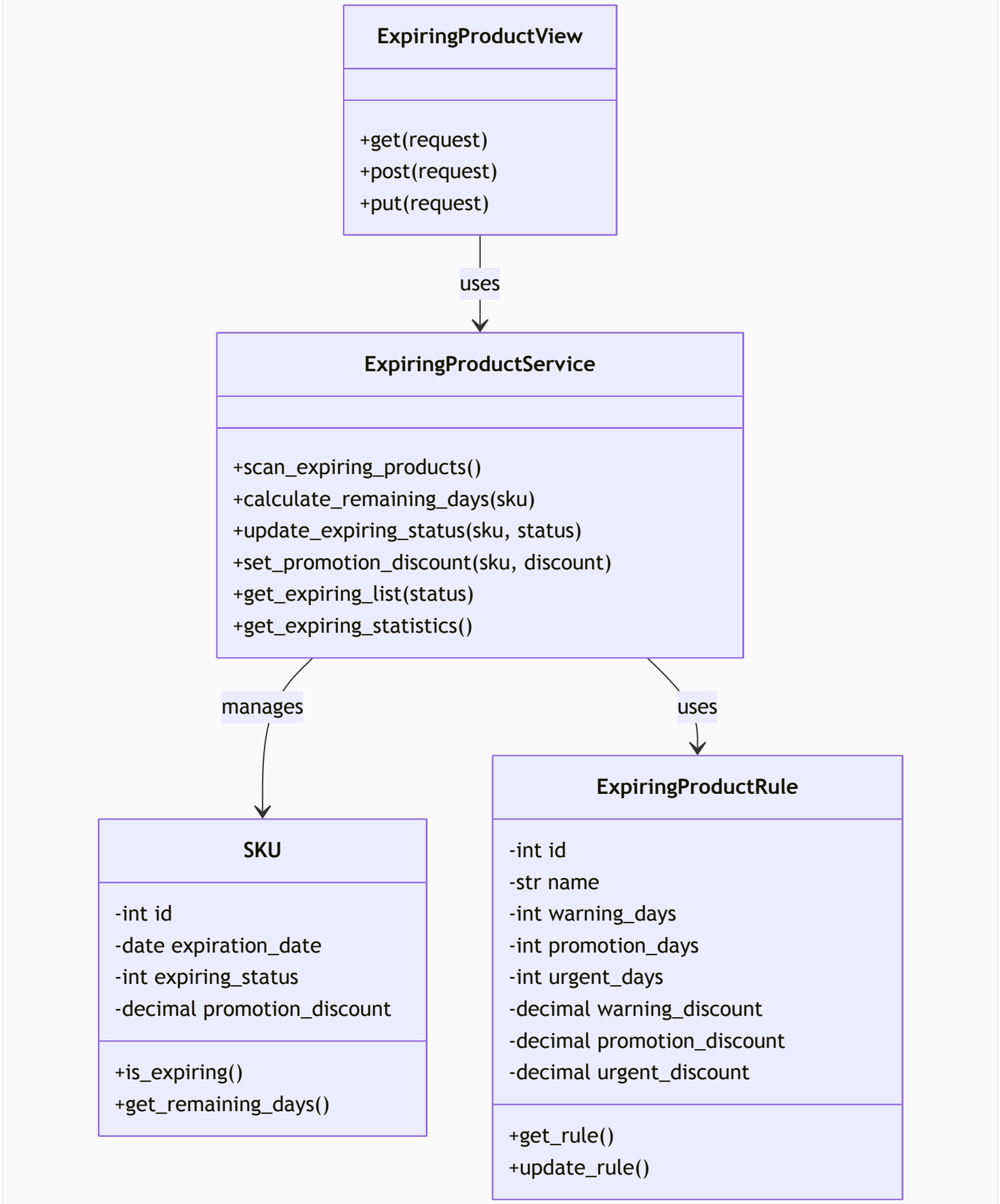
3.2.5.3 临期商品管理用例图



3.2.5.4 临期商品管理时序图



3.2.5.5 临期商品管理类图



3.2.5.6 临期商品数据表设计

tb_expiring_product_rule (临期商品规则表)

字段名	类型	长度	允许空	默认值	说明
id	INT	11	NO	AUTO_INCREMENT	主键ID
name	VARCHAR	50	NO	-	规则名称
warning_days	INT	11	NO	30	预警天数
promotion_days	INT	11	NO	15	促销天数
urgent_days	INT	11	NO	7	紧急天数
warning_discount	DECIMAL	5,2	NO	0.90	预警折扣(90%)
promotion_discount	DECIMAL	5,2	NO	0.70	促销折扣(70%)
urgent_discount	DECIMAL	5,2	NO	0.50	紧急折扣(50%)
is_active	BOOLEAN	-	NO	True	是否启用
create_time	DATETIME	-	NO	CURRENT_TIMESTAMP	创建时间
update_time	DATETIME	-	NO	CURRENT_TIMESTAMP	更新时间

索引设计

- PRIMARY KEY: `id`
- UNIQUE KEY: `name`
- INDEX: `is_active`

tb_expiring_product_log (临期商品日志表)

字段名	类型	长度	允许空	默认值	说明
id	INT	11	NO	AUTO_INCREMENT	主键ID
sku_id	INT	11	NO	-	商品SKU ID
old_status	SMALLINT	6	NO	-	原状态
new_status	SMALLINT	6	NO	-	新状态
old_discount	DECIMAL	5,2	YES	NULL	原折扣
new_discount	DECIMAL	5,2	YES	NULL	新折扣
action_type	VARCHAR	20	NO	-	操作类型
remark	VARCHAR	200	YES	NULL	备注
create_time	DATETIME	-	NO	CURRENT_TIMESTAMP	创建时间

索引设计

- PRIMARY KEY: `id`
- INDEX: `sku_id`
- INDEX: `create_time`
- INDEX: `action_type`

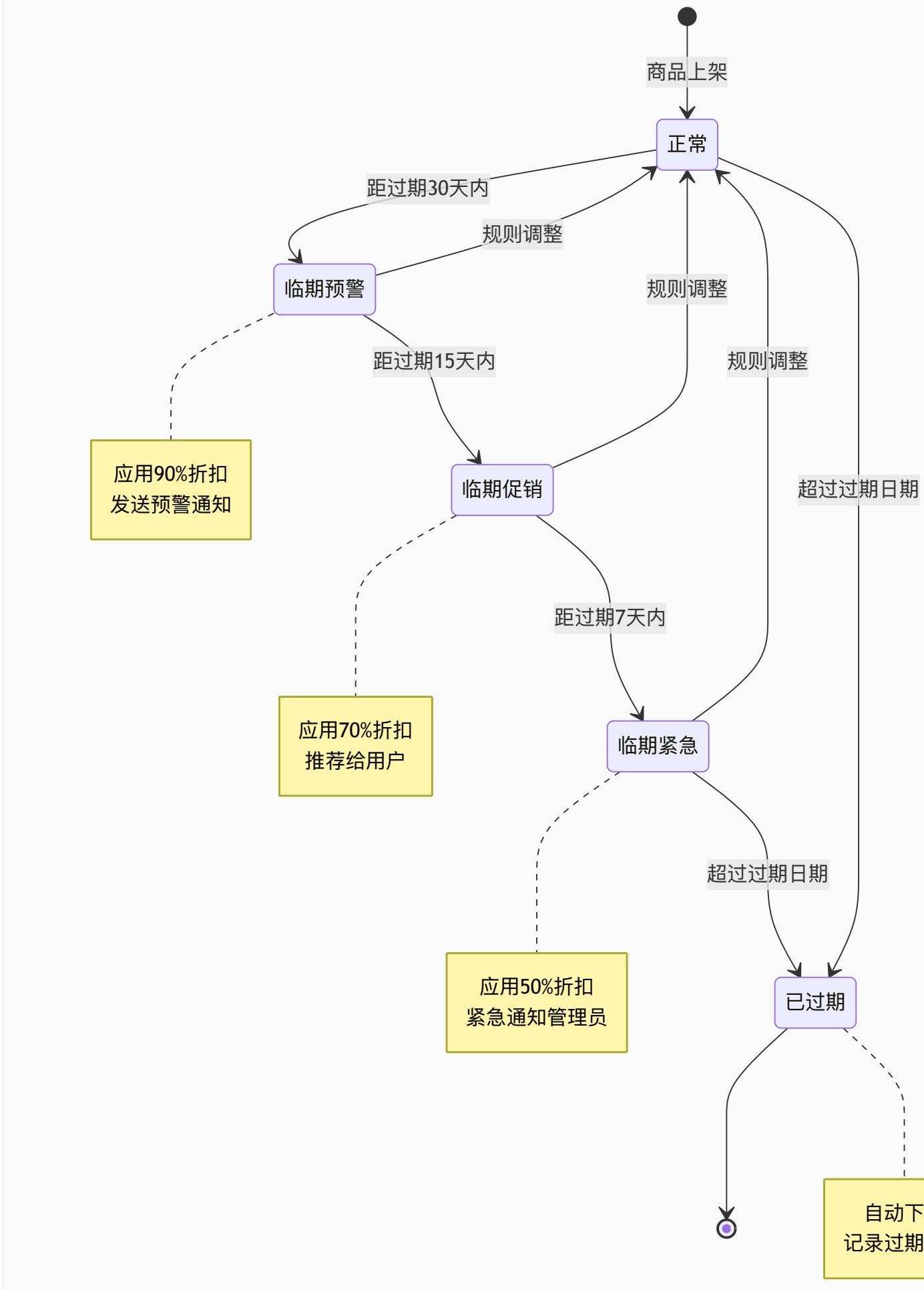
SKU表新增字段

字段名	类型	长度	允许空	默认值	说明
expiring_status	SMALLINT	6	NO	0	临期状态(0:正常,1:预警,2:促销,3:紧急,4:已过期)
promotion_discount	DECIMAL	5,2	YES	NULL	促销折扣

索引设计

- INDEX: `expiring_status`

3.2.5.7 临期商品状态流转图



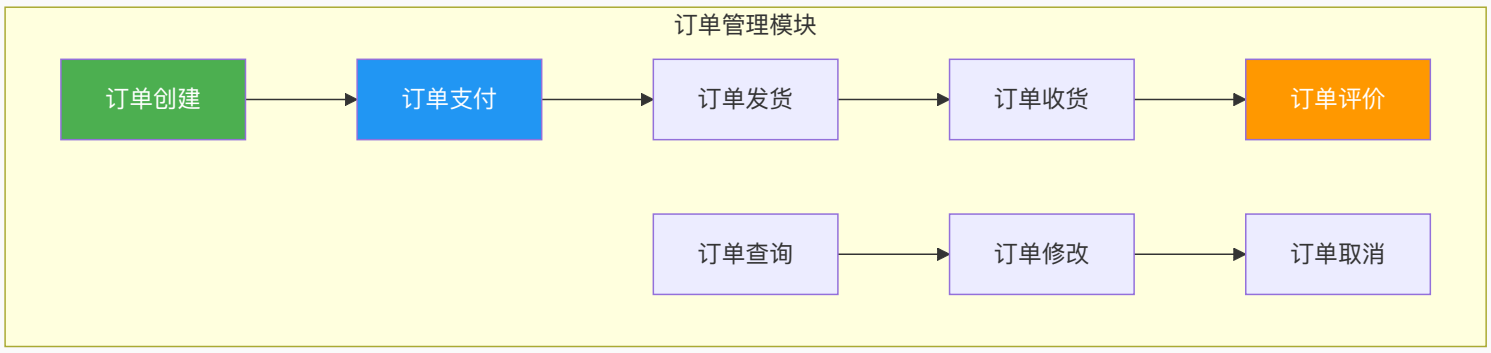
3.2.5.8 临期商品功能特点

1. **自动监控**: 系统定时扫描商品保质期，自动识别临期商品

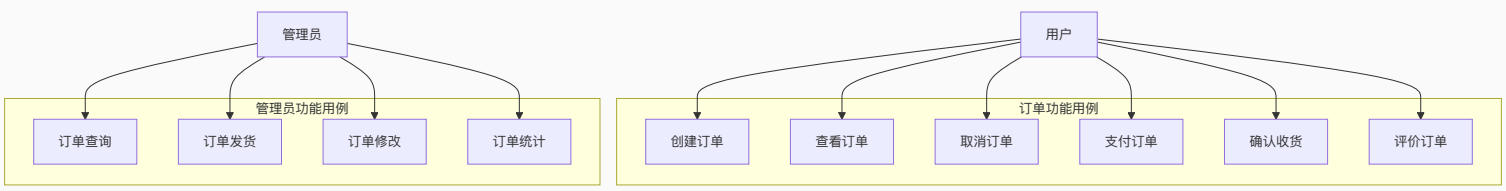
2. **分级管理**: 根据剩余保质期分为预警、促销、紧急三个等级
3. **自动促销**: 根据临期等级自动应用不同的促销折扣
4. **智能预警**: 临期商品自动发送通知给管理员
5. **用户推荐**: 临期商品在首页和分类页优先展示
6. **统计报表**: 提供临期商品统计和损失分析
7. **灵活配置**: 管理员可自定义临期天数和折扣比例

3.3 订单管理模块

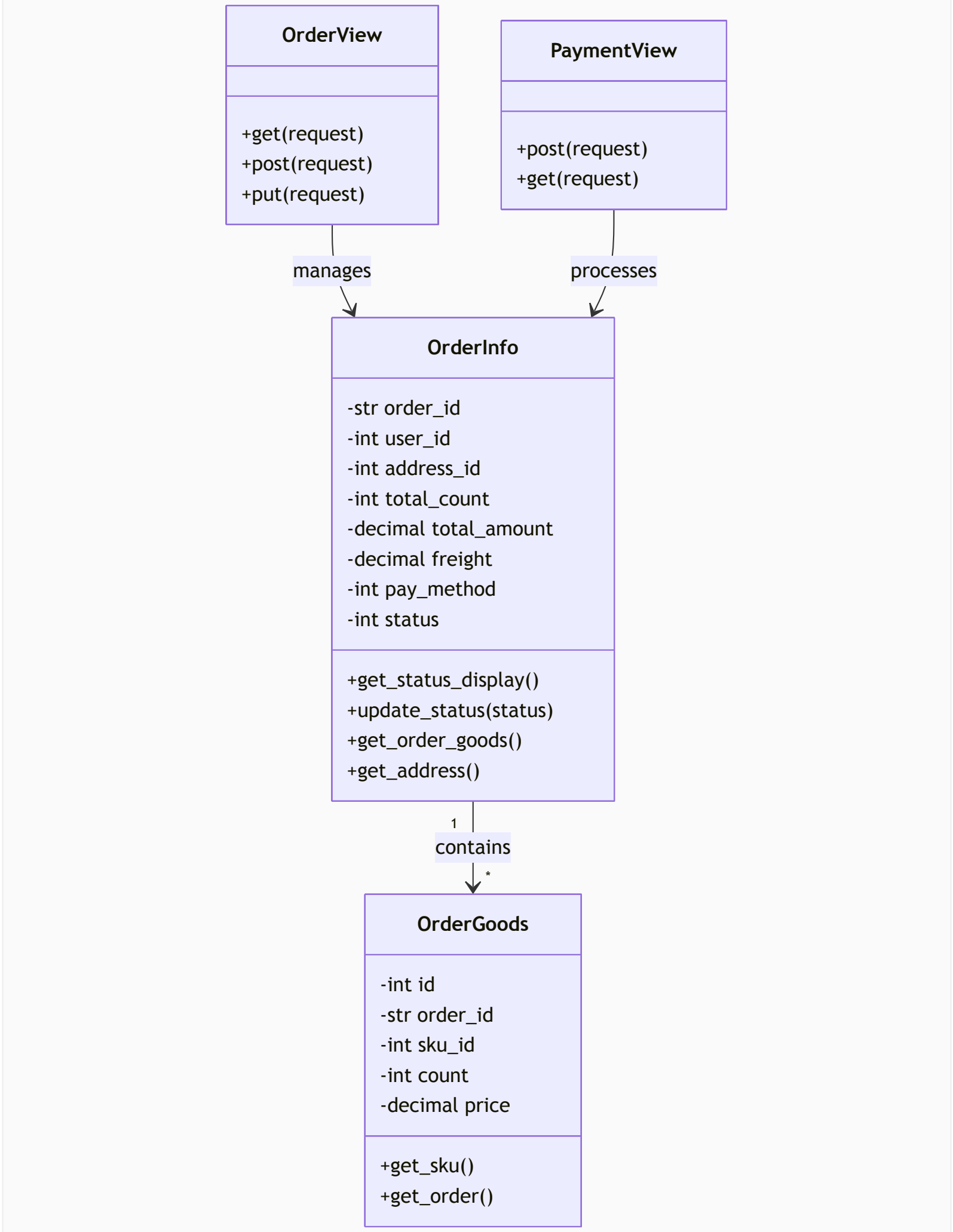
3.3.1 订单模块功能图



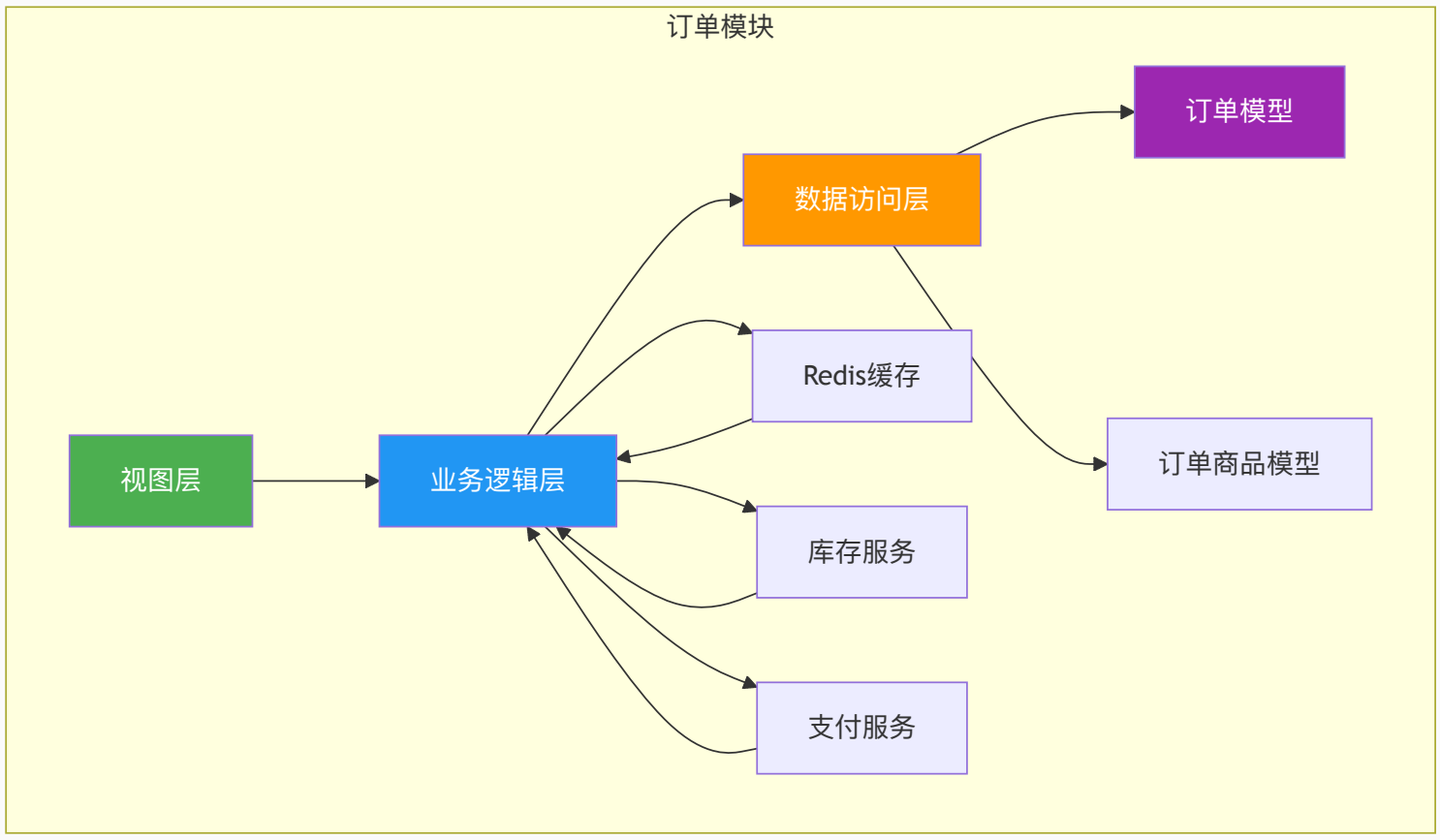
3.3.2 订单模块用例图



3.3.3 订单模块类图

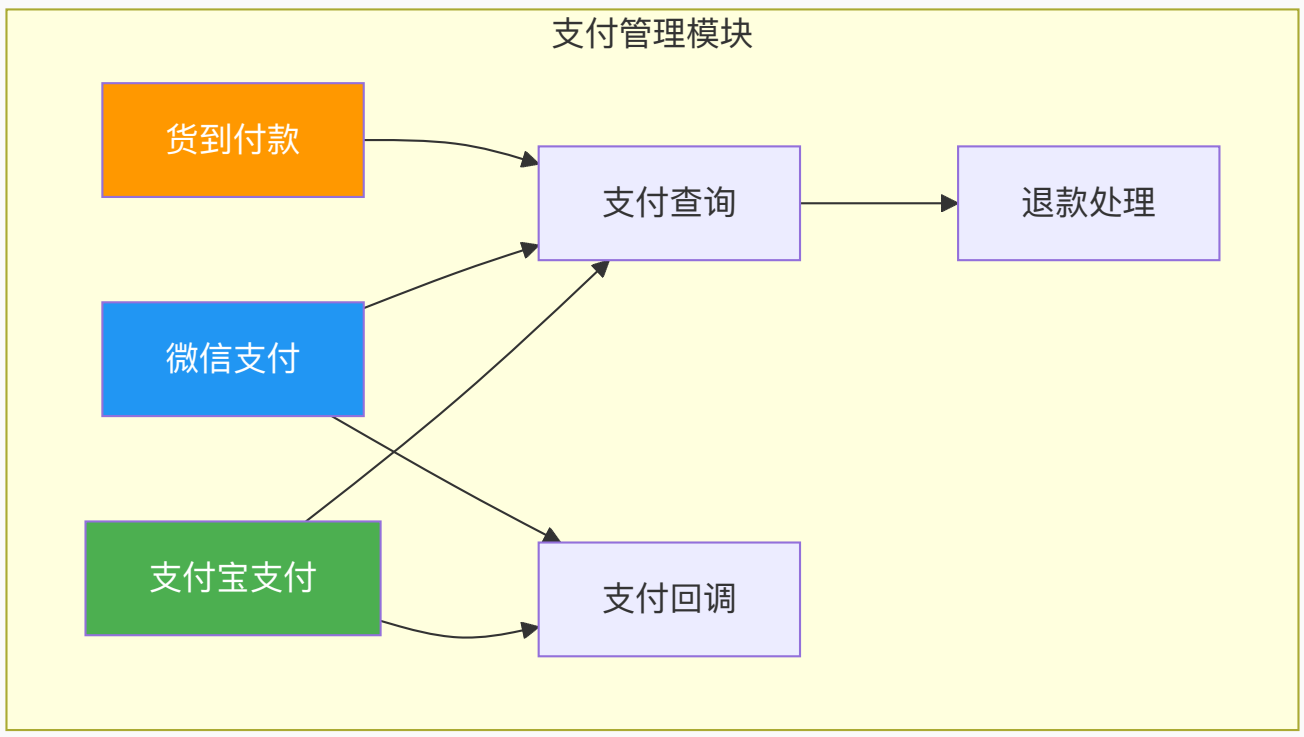


3.3.4 订单模块内部交互拓扑图

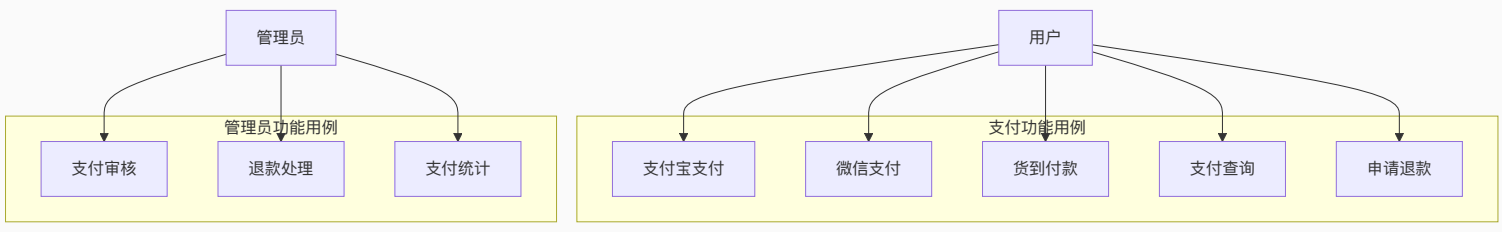


3.4 支付管理模块

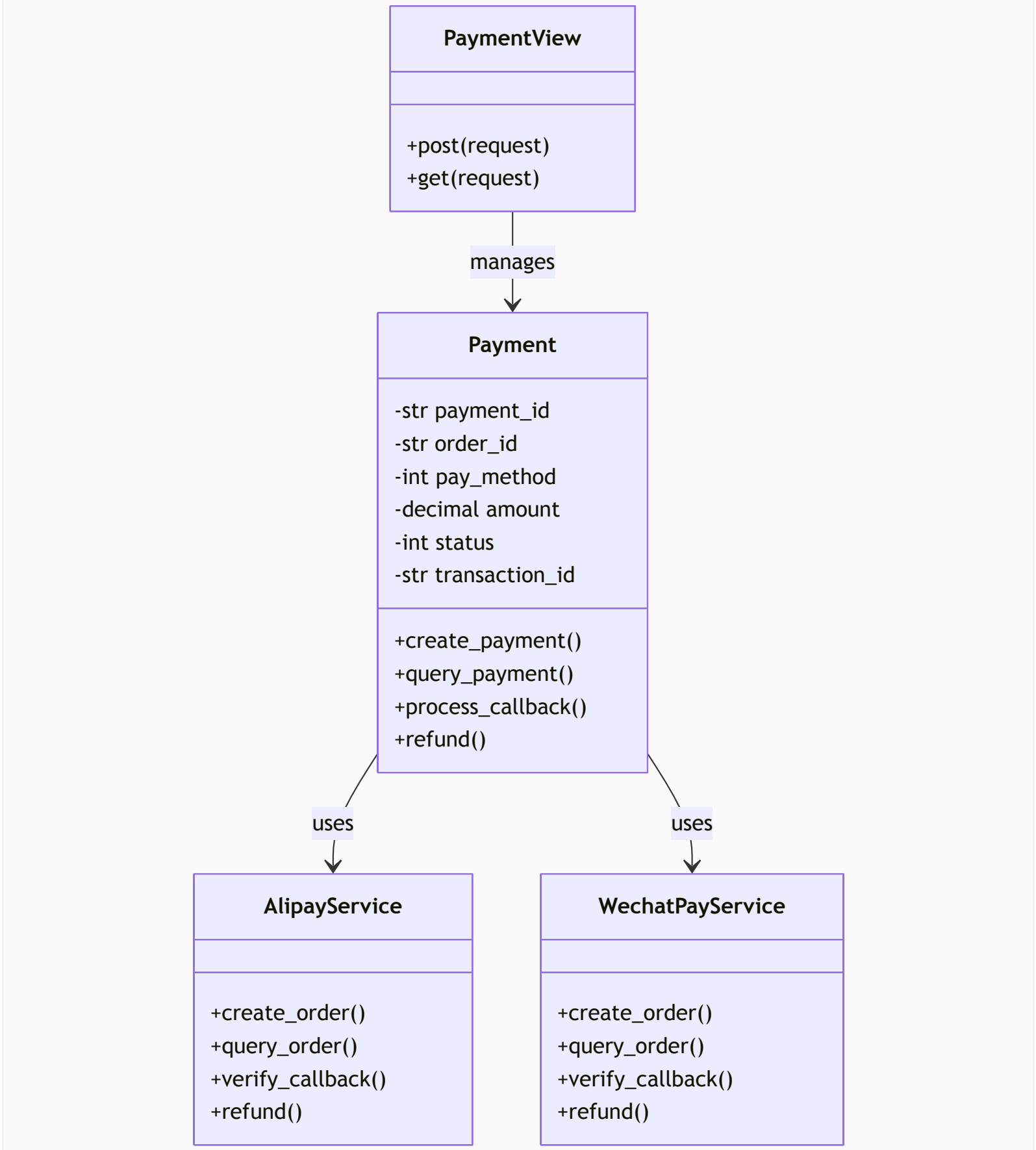
3.4.1 支付模块功能图



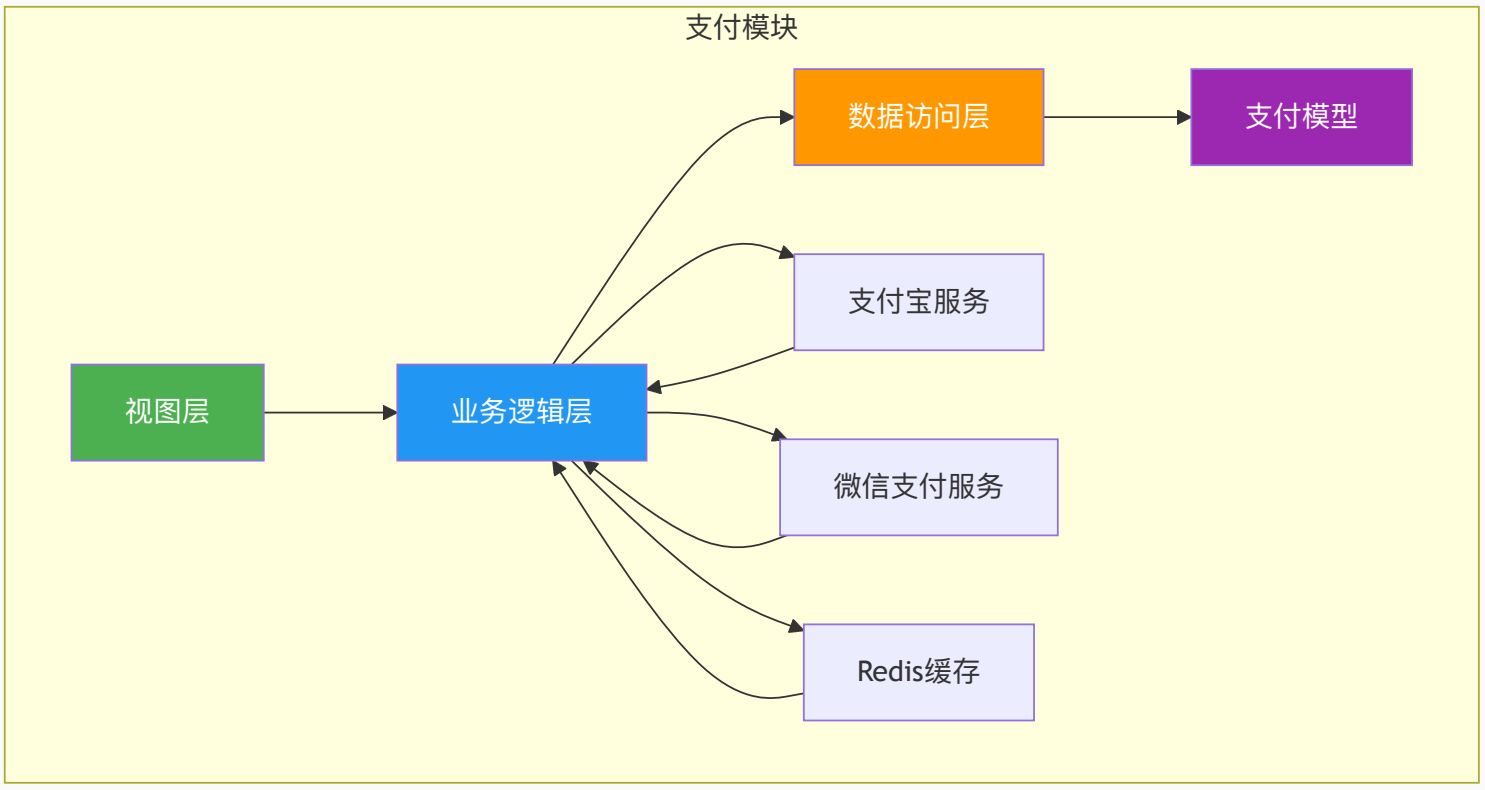
3.4.2 支付模块用例图



3.4.3 支付模块类图

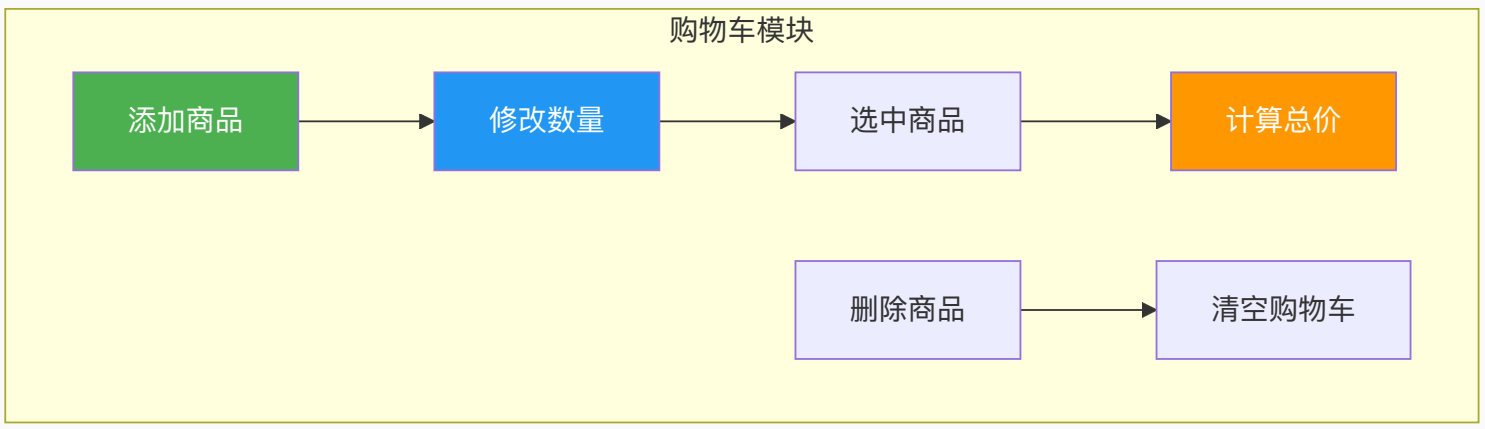


3.4.4 支付模块内部交互拓扑图

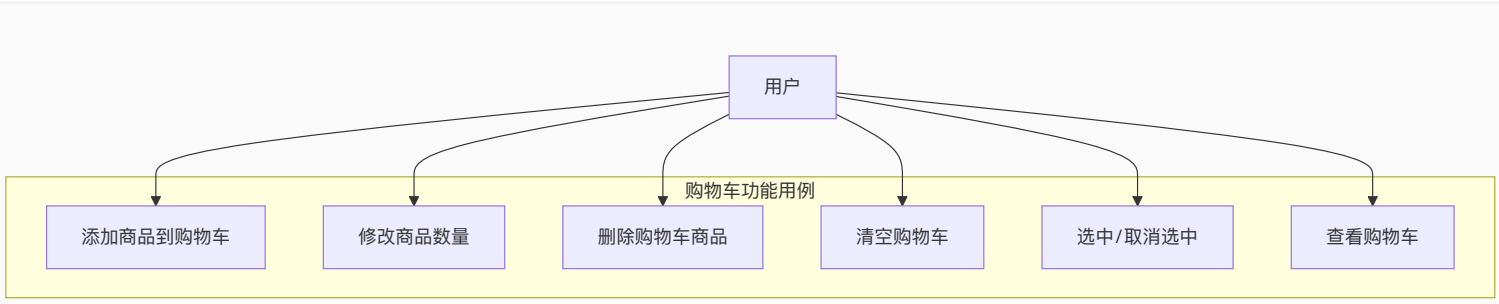


3.5 购物车模块

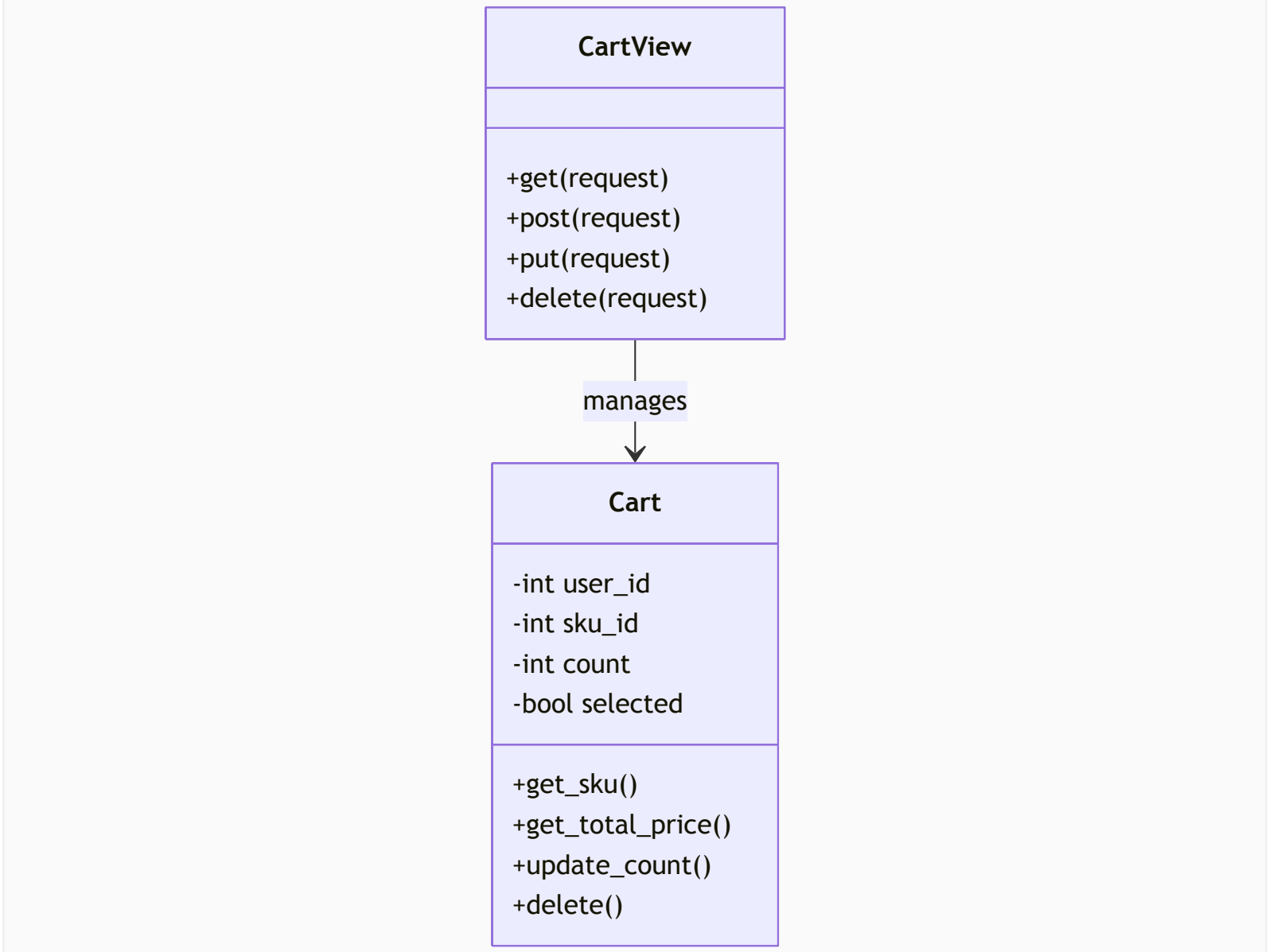
3.5.1 购物车模块功能图



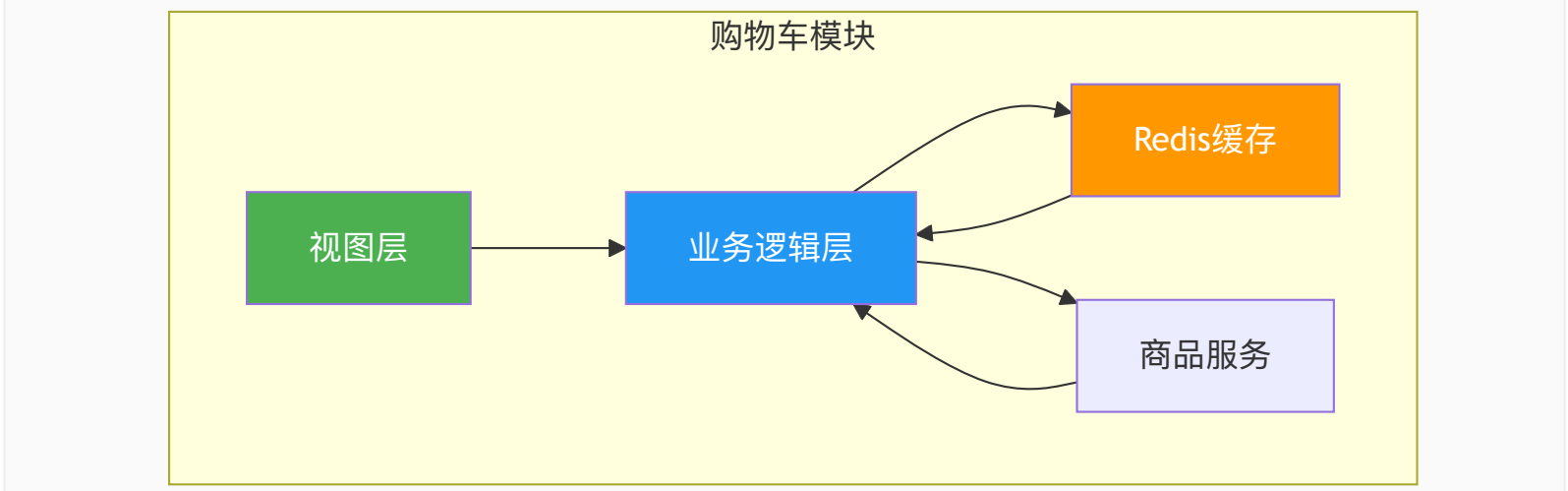
3.5.2 购物车模块用例图



3.5.3 购物车模块类图

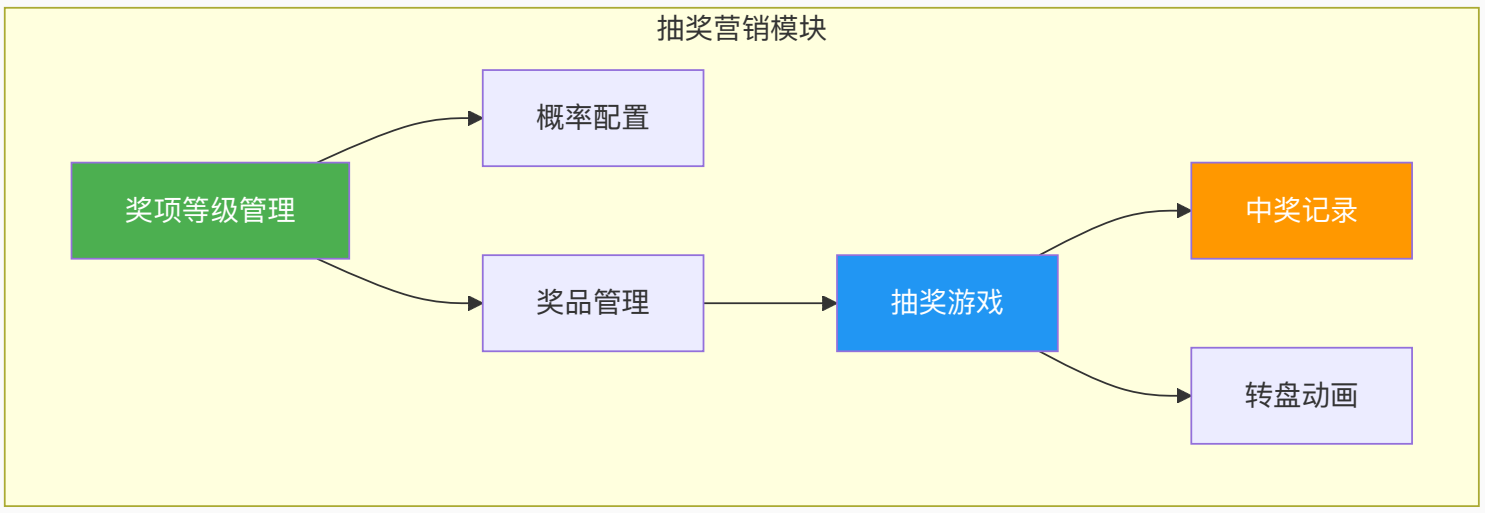


3.5.4 购物车模块内部交互拓扑图

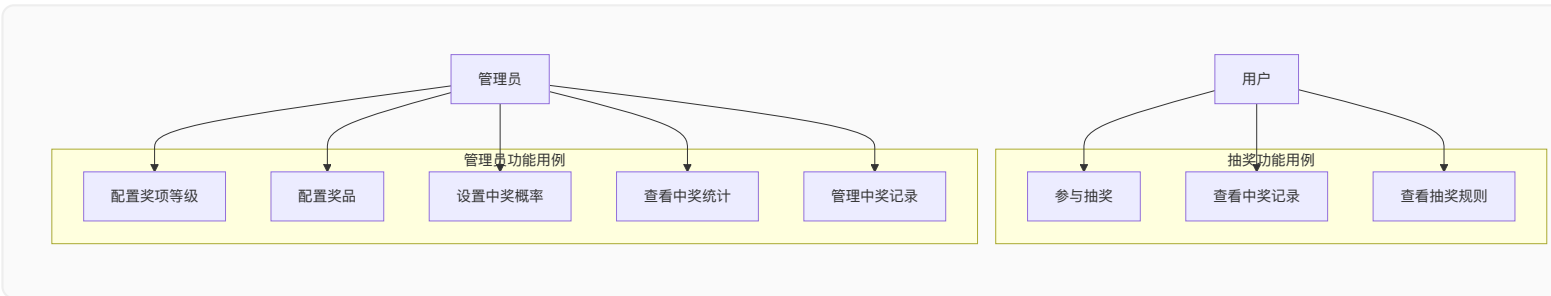


3.6 抽奖营销模块

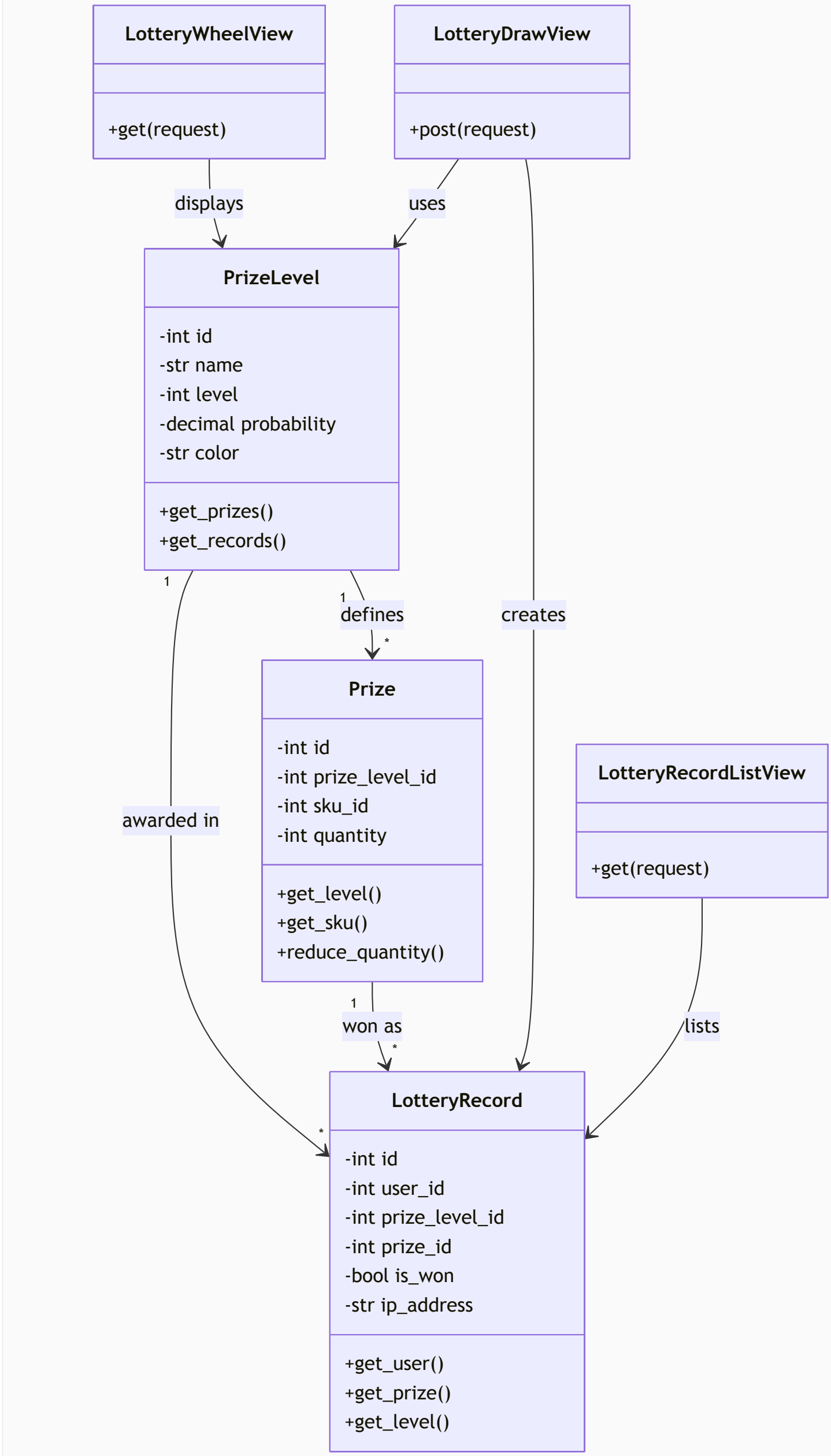
3.6.1 抽奖模块功能图



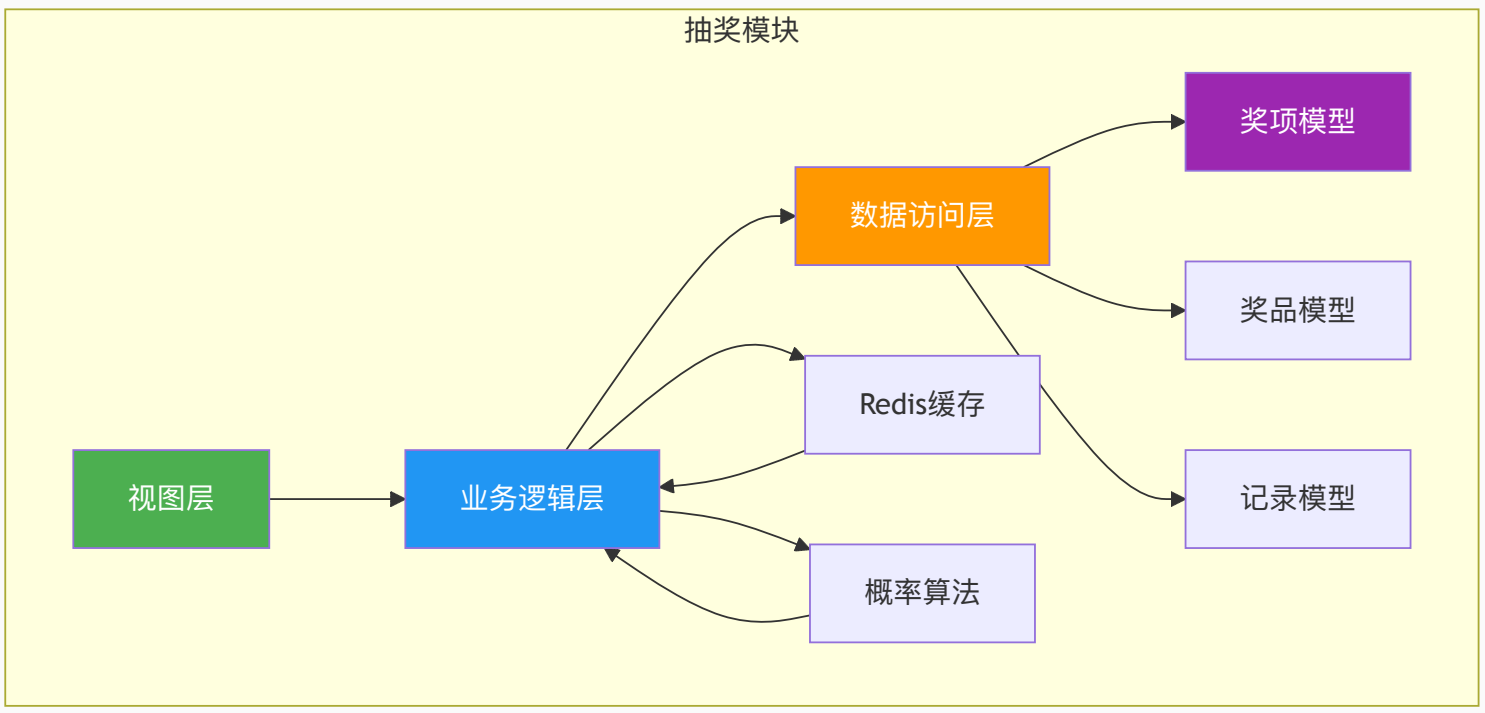
3.6.2 抽奖模块用例图



3.6.3 抽奖模块类图

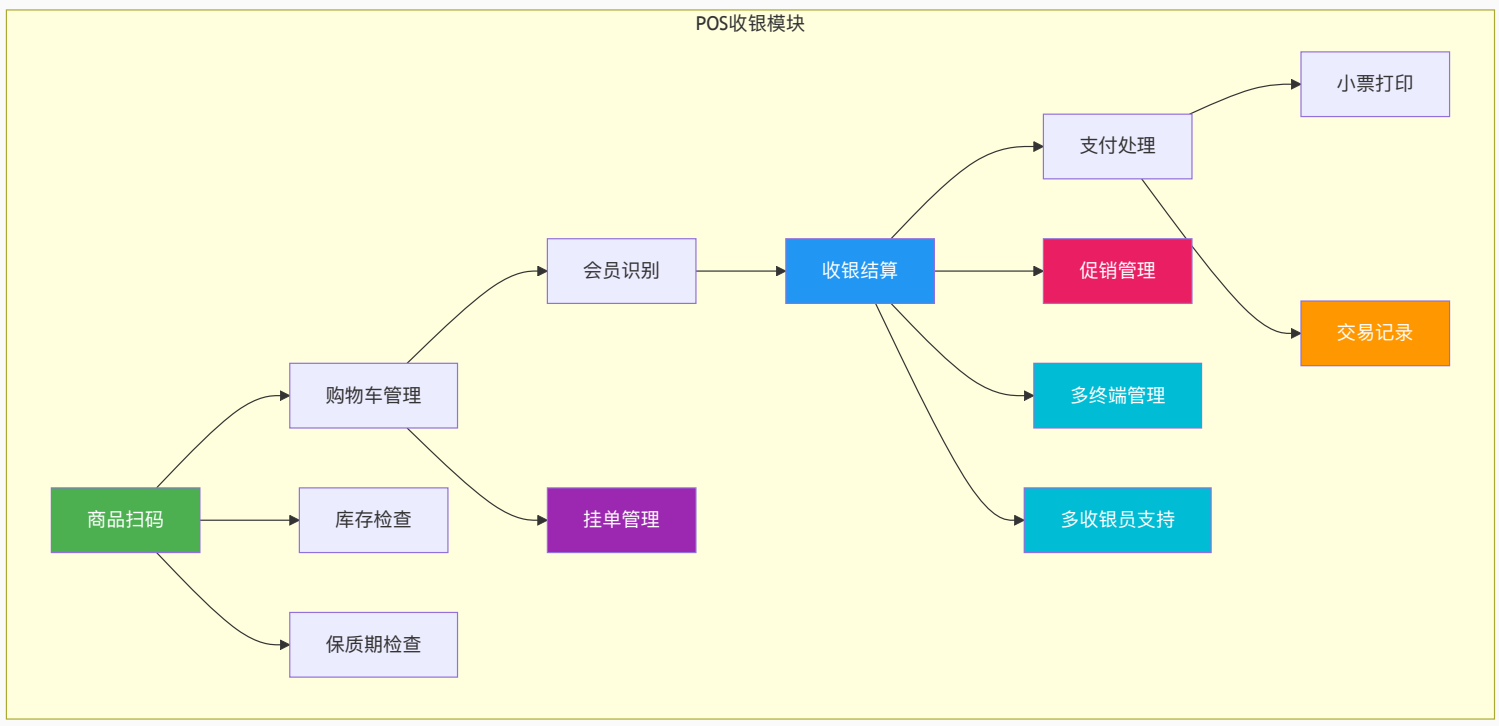


3.6.4 抽奖模块内部交互拓扑图

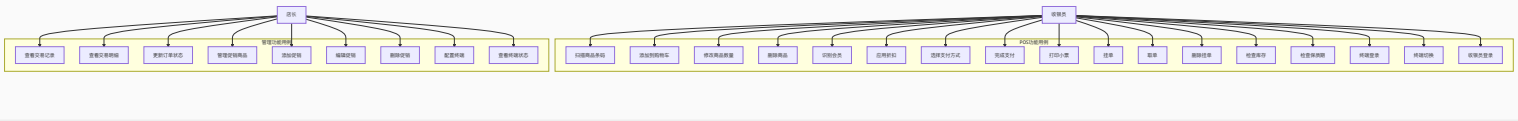


3.7 POS收银模块

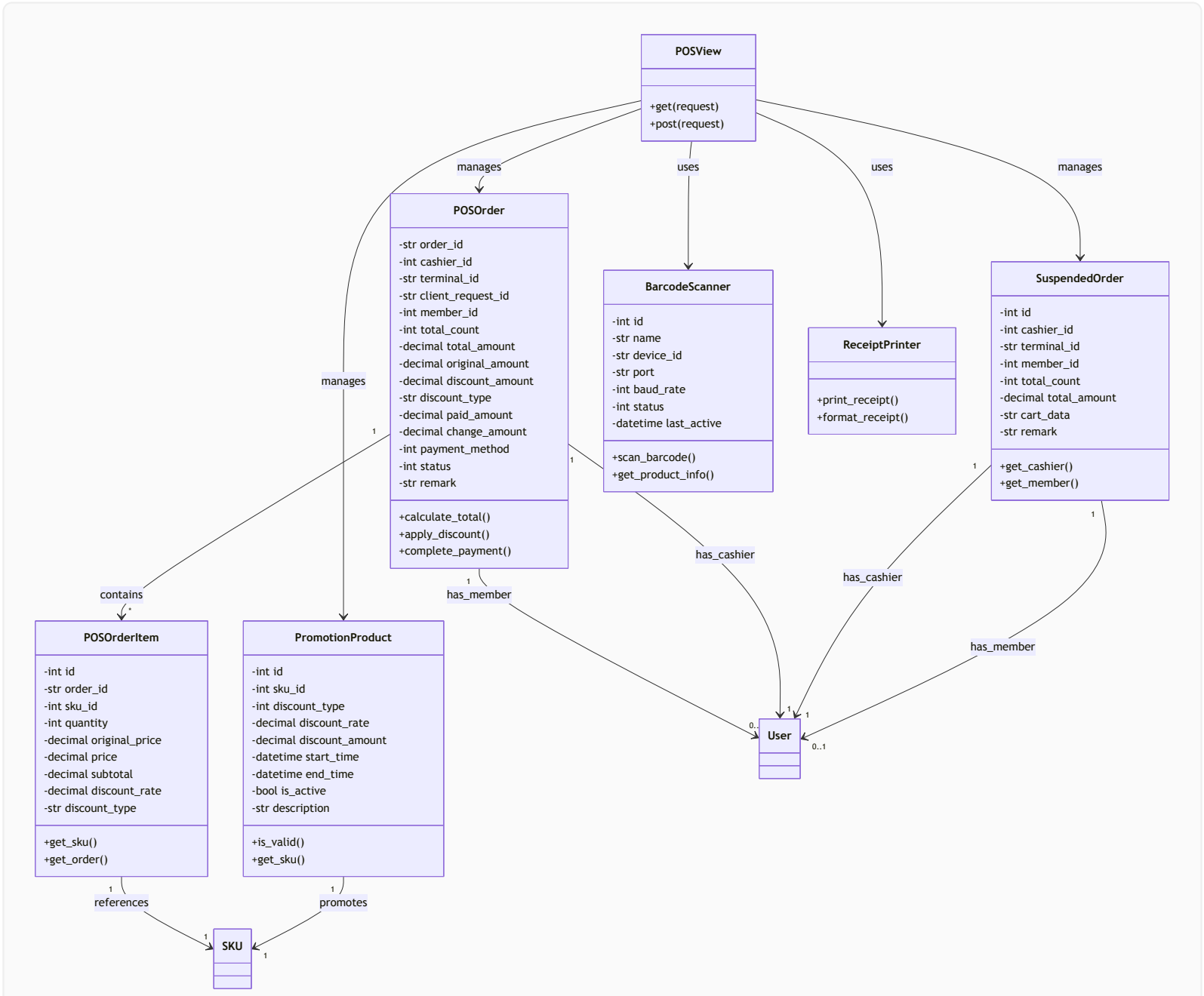
3.7.1 POS模块功能图



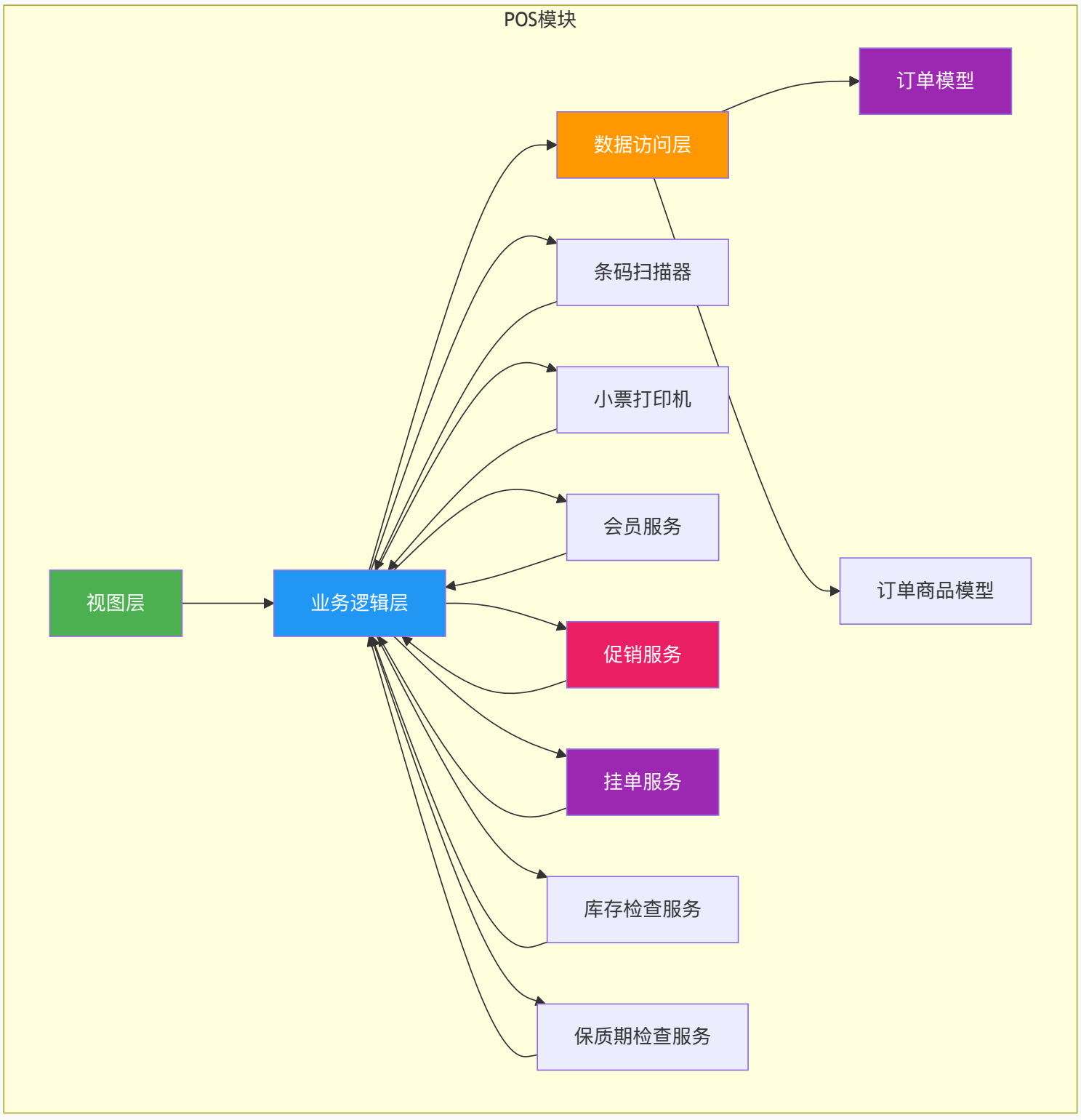
3.7.2 POS模块用例图



3.7.3 POS模块类图



3.7.4 POS模块内部交互拓扑图



3.7.5 POS模块数据表设计

tb_pos_order (收银台订单表)

字段名	类型	长度	允许空	默认值	说明
order_id	VARCHAR	64	NO	-	订单号(主键)
store_id	BIGINT	-	YES	NULL	所属门店ID；历史订单可为空
cashier_id	INT	11	NO	-	收银员ID
terminal_id	VARCHAR	100	NO	''	收银终端
client_request_id	VARCHAR	64	YES	NULL	客户端请求号
member_id	INT	11	YES	NULL	会员ID
total_count	INT	11	NO	0	商品总数
total_amount	DECIMAL	10,2	NO	-	总金额
original_amount	DECIMAL	10,2	NO	0	原始总金额
discount_amount	DECIMAL	10,2	NO	0	优惠金额
discount_type	VARCHAR	20	NO	'member'	折扣类型
paid_amount	DECIMAL	10,2	NO	0	实付金额
change_amount	DECIMAL	10,2	NO	0	找零
payment_method	SMALLINT	6	NO	1	支付方式
status	SMALLINT	6	NO	1	订单状态
remark	TEXT	-	NO	''	备注
create_time	DATETIME	-	NO	CURRENT_TIMESTAMP	创建时间
update_time	DATETIME	-	NO	CURRENT_TIMESTAMP	更新时间

索引设计

- PRIMARY KEY: `order_id`
- INDEX: `store_id`
- INDEX: `cashier_id`
- INDEX: `terminal_id`
- INDEX: `client_request_id` (UNIQUE)
- INDEX: `member_id`
- INDEX: `status`
- INDEX: `create_time`

tb_pos_order_item (收银台订单商品表)

字段名	类型	长度	允许空	默认值	说明
id	INT	11	NO	AUTO_INCREMENT	主键ID
order_id	VARCHAR	64	NO	-	订单号
sku_id	INT	11	NO	-	商品SKU ID
quantity	INT	11	NO	1	数量
original_price	DECIMAL	10,2	NO	0	原始单价
price	DECIMAL	10,2	NO	-	单价
subtotal	DECIMAL	10,2	NO	-	小计
discount_rate	DECIMAL	3,2	NO	1.00	折扣率
discount_type	VARCHAR	20	NO	'none'	折扣类型
create_time	DATETIME	-	NO	CURRENT_TIMESTAMP	创建时间
update_time	DATETIME	-	NO	CURRENT_TIMESTAMP	更新时间

索引设计

- PRIMARY KEY: `id`
- INDEX: `order_id`
- INDEX: `sku_id`

tb_promotion_product (促销商品表)

字段名	类型	长度	允许空	默认值	说明
id	INT	11	NO	AUTO_INCREMENT	主键ID
sku_id	INT	11	NO	-	商品SKU ID
store_id	BIGINT	-	YES	NULL	适用门店ID；NULL表示全局促销
discount_type	SMALLINT	6	NO	1	折扣类型(1:固定折扣率,2:固定金额)
discount_rate	DECIMAL	3,2	NO	1.00	折扣率
discount_amount	DECIMAL	10,2	NO	0	优惠金额
start_time	DATETIME	-	NO	-	促销开始时间
end_time	DATETIME	-	NO	-	促销结束时间
is_active	BOOLEAN	-	NO	True	是否启用
description	TEXT	-	NO	''	促销说明
create_time	DATETIME	-	NO	CURRENT_TIMESTAMP	创建时间
update_time	DATETIME	-	NO	CURRENT_TIMESTAMP	更新时间

索引设计

- PRIMARY KEY: `id`
- INDEX: `sku_id`

- INDEX: `is_active`
- INDEX: `start_time`
- INDEX: `end_time`

tb_suspended_order (挂单表)

字段名	类型	长度	允许空	默认值	说明
id	INT	11	NO	AUTO_INCREMENT	主键ID
store_id	BIGINT	-	YES	NULL	所属门店ID；历史挂单可为空
cashier_id	INT	11	NO	-	收银员ID
terminal_id	VARCHAR	100	NO	''	收银终端
member_id	INT	11	YES	NULL	会员ID
total_count	INT	11	NO	0	商品总数
total_amount	DECIMAL	10,2	NO	-	总金额
cart_data	TEXT	-	NO	-	购物车数据JSON
remark	TEXT	-	NO	''	备注
create_time	DATETIME	-	NO	CURRENT_TIMESTAMP	创建时间
update_time	DATETIME	-	NO	CURRENT_TIMESTAMP	更新时间

索引设计

- PRIMARY KEY: `id`
- INDEX: `cashier_id`
- INDEX: `terminal_id`
- INDEX: `member_id`
- INDEX: `create_time`

tb_barcode_scanner (条码扫描器设备表)

字段名	类型	长度	允许空	默认值	说明
id	INT	11	NO	AUTO_INCREMENT	主键ID
name	VARCHAR	50	NO	-	设备名称
device_id	VARCHAR	100	NO	-	设备ID
store_id	BIGINT	-	YES	NULL	归属门店ID
port	VARCHAR	50	NO	-	串口端口
baud_rate	INT	11	NO	9600	波特率
status	SMALLINT	6	NO	2	设备状态(1:在线,2:离线,3:故障)
last_active	DATETIME	-	YES	NULL	最后活跃时间
create_time	DATETIME	-	NO	CURRENT_TIMESTAMP	创建时间
update_time	DATETIME	-	NO	CURRENT_TIMESTAMP	更新时间

索引设计

- PRIMARY KEY: `id`

- UNIQUE KEY: `device_id`
- INDEX: `status`

门店与收银终端相关表

`tb_store`（门店表）字段如下：

字段名	类型	长度	允许空	默认值	说明
id	BIGINT	-	NO	AUTO_INCREMENT	主键ID
store_code	VARCHAR	20	NO	-	唯一门店编号
store_name	VARCHAR	50	NO	-	门店名称
address	VARCHAR	200	NO	-	门店地址
phone	VARCHAR	20	NO	''	联系电话
manager	VARCHAR	50	NO	''	店长姓名
status	SMALLINT	6	NO	1	状态（1:启用，0:停用）
business_hours	VARCHAR	100	NO	''	营业时间
create_time / update_time	DATETIME	-	NO	自动维护	创建与更新时间

`tb_user_store`（用户门店关系表）通过`user_id`和`store_id`关联用户与门店；每个用户在同一门店只有一条关系，角色为店长（1）、收银员（2）或库管（3）。关系字段包含主键、用户外键、门店外键、角色及创建/更新时间，并对`(user_id, store_id)`建立唯一约束。

`tb_store_stock`（门店库存表）字段如下：

字段名	类型	长度	允许空	默认值	说明
id	BIGINT	-	NO	AUTO_INCREMENT	主键ID
store_id	BIGINT	-	NO	-	门店外键
sku_id	BIGINT	-	NO	-	SKU外键
stock	INT	11	NO	0	门店库存总量
near_expiry_num	INT	11	NO	0	临期商品数量
expired_num	INT	11	NO	0	过期商品数量
create_time / update_time	DATETIME	-	NO	自动维护	创建与更新时间

唯一约束为`(store_id, sku_id)`，另有同字段复合索引。表中库存记录是POS门店销售的库存来源。

`tb_store_terminal`（门店收银终端表）按“每台收银机一条记录”管理终端：

字段名	类型	长度	允许空	默认值	说明
id	BIGINT	-	NO	AUTO_INCREMENT	主键ID
store_id	BIGINT	-	NO	-	所属门店ID，删除门店时级联删除
terminal_code	VARCHAR	100	NO	-	全局唯一的收银终端编号
name	VARCHAR	100	NO	-	终端显示名称
ip_address	VARCHAR	45	NO	''	数据库管理的终端IPv4或IPv6地址
certificate_fingerprint	VARCHAR	64	NO	''	客户端X.509证书DER编码的SHA-256指纹，启用设备必填
status	SMALLINT	6	NO	1	状态（1:启用，0:停用）
create_time	DATETIME	-	NO	CURRENT_TIMESTAMP	创建时间
update_time	DATETIME	-	NO	CURRENT_TIMESTAMP	更新时间

约束：`terminal_code`全局唯一，`store_id`为门店外键；按门店及终端编号排序。终端管理视图校验`certificate_fingerprint`格式并拒绝绑定到其他终端的重复指纹，但当前数据库未对该字段建立唯一索引。`ip_address`保存单个IPv4/IPv6地址，`certificate_fingerprint`保存64位十六进制指纹。对应迁移为`pos.0021_storeterminal`、`pos.0022_storeterminal_ip_address`和`pos.0023_storeterminal_certificate_fingerprint`。

`tb_store_stock`（门店库存表）以`store_id + sku_id`唯一约束实现门店库存隔离，保存库存、临期数量和过期数量。`tb_promotion_product.store_id`可为空，为空时表示全局促销，否则表示门店促销。

3.7.6 POS模块核心功能说明

3.7.6.1 商品扫码与保质期检查

系统支持通过条码扫描器扫描商品条码，自动识别商品信息。在扫码过程中，系统会检查商品的保质期状态：

- **正常商品**: 距离过期日期超过3天的商品
- **临保质期商品**: 距离过期日期3天内的商品（系统会显示警告信息但不阻止销售）
- **已过期商品**: 已超过过期日期的商品（系统会显示过期信息但不阻止销售）

保质期检查仅用于信息提示，不阻止商品销售，确保收银流程的连续性。

3.7.6.2 会员识别与折扣策略

系统支持通过手机号或用户名识别会员，并根据会员等级和促销活动应用不同的折扣策略：

- **非会员**: 所有商品按原价销售，不享受任何折扣
- **会员**:
  - 促销商品：比较促销价和会员折扣价，取较小值
  - 非促销商品：按原价销售，不使用会员折扣

折扣类型包括：

- `member`：会员折扣
- `promotion`：商品促销
- `none`：无折扣

3.7.6.3 挂单与取单功能

系统支持挂单功能，允许收银员将当前购物车暂时保存，以便处理其他紧急事务：

- **挂单**: 保存当前购物车数据到挂单表，包括商品信息、会员信息、备注等
- **取单**: 从挂单表恢复购物车数据，继续收银流程
- **删除挂单**: 删除不需要的挂单记录

挂单记录按收银员和收银终端隔离，确保数据安全。

3.7.6.4 促销商品管理

系统提供完整的促销商品管理功能，管理员可以：

- **添加促销**: 为指定商品设置促销规则（固定折扣率或固定金额优惠）
- **编辑促销**: 修改现有促销规则
- **删除促销**: 删除不再需要的促销
- **促销查询**: 查看所有促销商品列表，支持按商品名称搜索

促销规则包含时间范围，系统会自动判断促销是否在有效期内。

3.7.6.5 库存检查与扣减

在创建订单时，系统会检查商品库存是否充足：

- 考虑过期商品和临保质期商品的数量，计算可用库存
- 如果库存不足，返回详细的库存信息（总库存、过期商品数量、临保质期商品数量）
- 使用乐观锁机制扣减库存，防止超卖
- 根据商品保质期状态更新临保质期或过期商品数量
- POS使用 `StoreStock` 作为门店库存来源；未初始化到当前门店的SKU按该店库存为0处理，不自动把全局SKU库存复制到门店
- 管理员通过门店库存初始化页面将SKU当前库存、临期数量和过期数量复制到目标门店；初始化仅创建缺失记录，重复执行不会覆盖已有门店库存

3.7.6.6 交易记录查询

系统提供交易记录查询功能，支持：

- **交易记录查询**: 查看订单列表，支持按日期范围和订单号筛选
- **交易明细查询**: 查看订单商品明细，支持按日期范围和订单号筛选
- **订单状态更新**: 支持更新订单状态（取消、删除）
- **分页显示**: 所有查询结果支持分页显示

3.7.6.7 小票打印

系统支持小票打印功能，小票包含以下信息：

- 订单基本信息（订单号、终端编号、时间、收银员）
- 订单关联的门店编号和门店名称；终端编号与门店编号分开展示，历史上未关联门店的订单显示“未关联门店”
- 会员信息（会员名、会员等级）
- 支付方式
- 商品明细（商品名、原价、折扣率、应付金额）
- 金额汇总（商品数量、折扣前总金额、优惠金额、折扣后金额）
- 折扣信息（会员折扣率、订单折扣方式）

3.7.6.8 多收银员多终端并发收银

系统支持多收银员在多个收银终端同时进行收银操作，确保数据隔离和并发安全：

终端管理机制

- 管理员在门店管理中按每台收银机一条记录维护终端名称、全局唯一终端编号和启停状态
- 收银台根据当前门店展示其启用终端列表，收银员选择本机对应终端后，页面把该编号随请求提交
- 如果当前门店已登记任何终端，服务端仅接受该门店状态为启用的终端编号；全部停用时拒绝收银请求，不回退全局

配置

- 终端编号和IP地址均以当前门店的StoreTerminal数据库记录为准；未配置终端或终端IP时拒绝POS请求
- 终端标识通过HTTP头（X_POS_TERMINAL_CODE）或请求参数传递
- 订单和挂单记录都包含terminal_id字段，用于标识收银终端

收银员隔离

- 每个收银员通过登录系统获得身份认证
- 订单记录包含cashier_id字段，标识操作收银员
- 挂单记录按收银员和终端双重隔离，确保收银员只能访问自己的挂单
- 交易记录查询支持按收银员筛选

并发控制

- 使用client_request_id实现请求幂等性，防止重复提交
- 幂等性检查同时验证cashier_id和terminal_id，确保同一请求只能由同一收银员在同一终端提交
- 如果请求标识已被其他收银员或终端使用，系统拒绝处理并返回错误
- 库存扣减使用乐观锁机制，防止多终端同时扣减同一商品导致的超卖

数据隔离保证

- 挂单查询：按收银员、终端编号和当前门店过滤
- 挂单取单：必须同时匹配挂单ID、收银员、终端编号和当前门店
- 幂等性检查：验证 `existing_order.cashier_id != request.user.id or existing_order.terminal_id != terminal_id`
- 当前门店从登录Session读取，并在每次业务请求中重新验证门店启用状态及用户门店关系；关系撤销或门店停用后，Session中的门店选择会被清除

配置说明

- 新部署优先在门店终端配置页面登记每台收银机； `terminal_code` 必须唯一，停用的终端不能发起POS操作
- `POS_TERMINAL_CODE` 、 `POS_TERMINAL_IP_MAP` 和 `POS_ENABLE_IP_WHITELIST` 不再用于终端校验；门店终端编号和IP配置完全由数据库记录管理
- 每次POS请求同时校验当前门店、启用终端编号和直连客户端IP； `X-Forwarded-For` 不直接作为可信IP来源
- 门店切换后，收银台终端列表随当前门店变化；浏览器保存的旧终端编号只有在新门店列表中仍有效时才会被选中

3.7.7 门店管理、用户关联与库存初始化

门店管理：超级管理员通过门店列表新增、编辑和删除门店。每个门店具有唯一门店编号、名称、地址、联系方式、店长、营业时间和启停状态。删除门店时，终端、门店库存和用户门店关系按外键级联策略清理；被POS订单引用的门店使用保护删除，避免订单历史失联。

用户门店关联：管理员可以为用户新增门店关系、修改角色或删除关系。用户可被分配到多个门店，但在同一门店最多一条关系。普通用户只能选择被分配且启用的门店；超级管理员可选择任意启用门店。

门店选择：登录后进入收银台，如果Session中没有有效门店则跳转到门店选择页。选店POST请求重新验证用户权限和门店状态后保存 `current_store_id` 、 `current_store_name` 。后续POS请求不能只依赖旧Session值，必须再次验证权限。

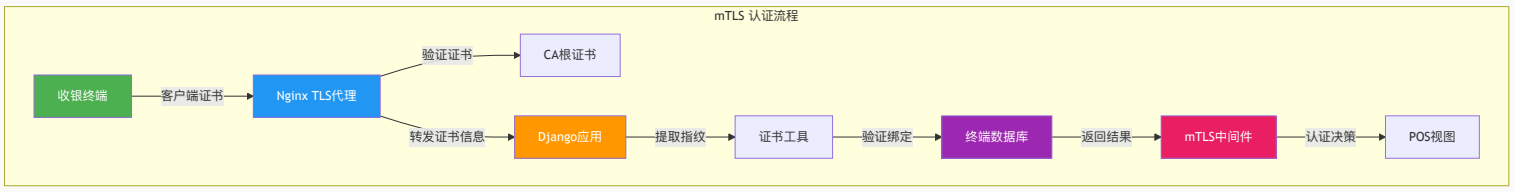
门店库存初始化：管理员选择目标门店后遍历SKU，为缺失的门店库存记录创建初始数据，初值取SKU全局库存、临期数量和过期数量。已有记录保留不变，流程在数据库事务中执行。未初始化SKU在门店扫码时最多建立零库存记录，不得隐式继承全局库存；库存检查和订单结算均以门店库存为准。

3.7.8 mTLS 设备认证

3.7.8.1 mTLS 认证概述

系统采用双向 TLS（mTLS）认证机制，确保收银终端设备的身份可信性和通信安全性。每台收银终端配置唯一的客户端证书，通过证书指纹与数据库中的终端记录绑定，实现设备级别的访问控制。

认证架构



核心组件

1. **客户端证书**: 每台收银终端持有唯一的 X.509 客户端证书
2. **TLS 代理**: Nginx 或其他 TLS 终止代理，负责验证客户端证书并转发证书信息
3. **证书指纹提取**: 使用 SHA-256 算法计算证书指纹
4. **证书绑定验证**: 将证书指纹与数据库中的终端记录进行匹配验证
5. **mTLS 中间件**: Django 中间件，拦截请求并执行 mTLS 认证

3.7.8.2 证书指纹提取与验证

证书指纹计算

系统使用 SHA-256 算法计算客户端证书的指纹，确保证书唯一性：

```
def get_certificate_fingerprint_from_pem(cert_pem: str) -> str:
    """
    从 PEM 格式证书计算 SHA-256 指纹

    参数:
        cert_pem: PEM 编码的证书字符串

    返回:
        SHA-256 指纹十六进制字符串（小写，无冒号）
    """
    from cryptography import x509
    from cryptography.hazmat.backends import default_backend
    import hashlib

    cert = x509.load_pem_x509_certificate(cert_pem.encode(), default_backend())
    fingerprint = hashlib.sha256(cert.public_bytes()).hexdigest()
    return fingerprint
```

证书指纹格式

- 算法: SHA-256
- 输出格式: 64 位十六进制字符串（小写，无冒号分隔）
- 示例: `6EA6AEFEF61607D68CAE8E43669E918012E3BD35FA332C2C34A0F9E9968806E9`

证书提取来源

系统支持从多种来源提取客户端证书信息：

1. **HTTP 头转发**: TLS 代理通过 HTTP 头转发证书（默认 `X-Client-Cert`）
2. **WSGI 环境变量**: Django 直接运行在 TLS 终止服务器时（如 mod_wsgi）
3. **环境变量配置**: 通过 `MTLS_CLIENT_CERT_HEADER` 配置自定义 HTTP 头名称

3.7.8.3 证书绑定管理

证书绑定数据模型

`StoreTerminal` 模型包含证书指纹字段：

```
class StoreTerminal(BaseModel):
    """门店收银终端配置"""
    store = models.ForeignKey(Store, on_delete=models.CASCADE, related_name='terminals')
    terminal_code = models.CharField(max_length=100, unique=True, verbose_name="终端编号")
    name = models.CharField(max_length=100, verbose_name="终端名称")
    ip_address = models.CharField(max_length=45, default='', verbose_name="终端IP地址")
    certificate_fingerprint = models.CharField(
        max_length=64, default='', blank=True, verbose_name="客户端证书SHA-256指纹"
    )
    status = models.SmallIntegerField(choices=((1, "启用"), (0, "停用")), default=1)
```

证书绑定 API

系统提供 RESTful API 用于管理证书绑定：

1. **绑定证书**：`POST /pos/api/certificate/bind/`
   - 请求参数: `terminal_code`, `certificate_fingerprint`
   - 功能: 将证书指纹绑定到指定终端
   - 验证: 检查指纹是否已绑定到其他终端
2. **解绑证书**：`POST /pos/api/certificate/unbind/`
   - 请求参数: `terminal_code`
   - 功能: 解除终端的证书绑定
3. **验证证书**：`POST /pos/api/certificate/verify/`
   - 请求参数: 无（从当前请求提取证书）
   - 功能: 验证当前请求的证书是否有效并已绑定
   - 返回: 绑定状态、终端信息、错误原因

证书绑定验证逻辑

```
def verify_certificate_binding(request: HttpRequest, terminal_code: str) -> dict:
    """
    验证客户端证书与终端绑定

    参数:
        request: Django HTTP 请求对象
        terminal_code: 终端编号

    返回:
        验证结果字典，包含:
        - valid: bool, 是否验证通过
        - reason: str, 验证失败原因
        - terminal_id: str, 绑定的终端编号
    """
    # 提取证书指纹
    fingerprint = extract_client_certificate_fingerprint(request)

    if not fingerprint:
        return {
            'valid': False,
            'reason': 'NO_CERTIFICATE',
            'terminal_id': None
        }

    # 查询终端
    terminal = StoreTerminal.objects.filter(
        terminal_code=terminal_code,
        status=1
    ).first()

    if not terminal:
        return {
            'valid': False,
            'reason': 'TERMINAL_NOT_FOUND',
            'terminal_id': None
        }

    # 验证证书指纹
    if terminal.certificate_fingerprint != fingerprint:
        return {
            'valid': False,
            'reason': 'CERTIFICATE_MISMATCH',
            'terminal_id': terminal.terminal_code
        }

    return {
        'valid': True,
        'reason': None,
        'terminal_id': terminal.terminal_code
    }
```

3.7.8.4 mTLS 中间件设计

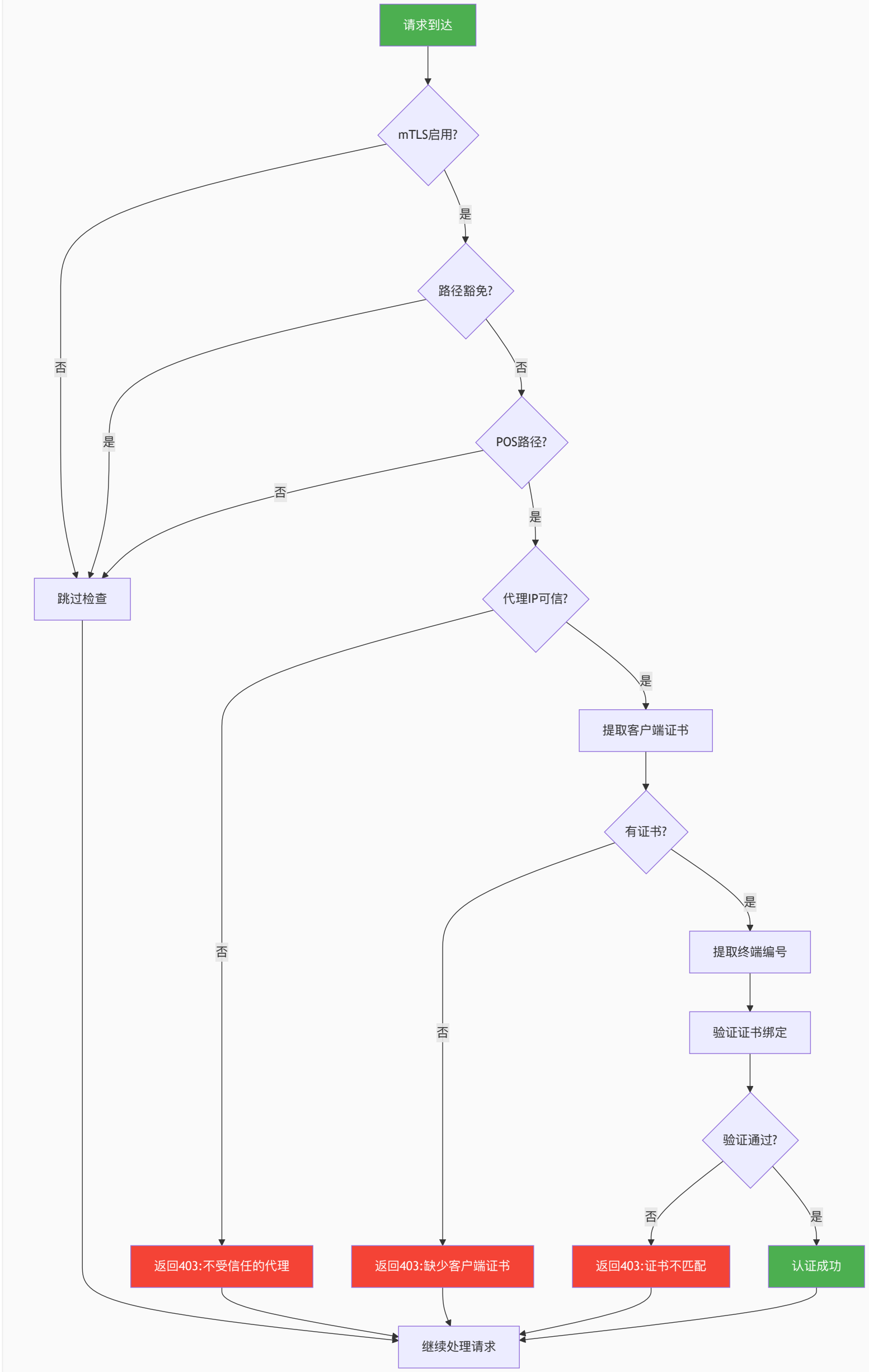
中间件配置

mTLS 认证通过 Django 中间件实现，在 `settings.py` 中配置：

```
MIDDLEWARE = [
    'django.middleware.security.SecurityMiddleware',
    'hot_mall.license_middleware.LicenseMiddleware',
    'hot_mall.mtls_middleware.MTLSAuthenticationMiddleware',  # mTLS 认证中间件
    'django.contrib.sessions.middleware.SessionMiddleware',
    # ... 其他中间件
]

# mTLS 配置
MTLS_ENABLED = True  # 是否启用 mTLS 认证
MTLS_CLIENT_CERT_HEADER = 'HTTP_X_CLIENT_CERT'  # 客户端证书 HTTP 头
MTLS_EXEMPT_PATHS = ['/pos/api/certificate/verify/']  # 免除 mTLS 检查的路径
POS_TRUSTED_TLS_PROXY_IPS = ['127.0.0.1', '::1']  # 受信任的 TLS 代理 IP
```

中间件处理流程


请求到达
mTLS启用?
是
否
路径豁免?
否
是
POS路径?
否
是
跳过检查
代理IP可信?
是
否
提取客户端证书
有证书?
是
否
提取终端编号
验证证书绑定
验证通过?
否
是
返回403:不受信任的代理
返回403:缺少客户端证书
返回403:证书不匹配
认证成功
继续处理请求

中间件错误码

错误码	说明	HTTP 状态码
UNTRUSTED_PROXY	请求来源不受信任	403
NO_CERTIFICATE	缺少客户端证书	403
TERMINAL_NOT_FOUND	终端不存在或已停用	403
CERTIFICATE_NOT_BOUND	终端未绑定证书	403
CERTIFICATE_MISMATCH	证书与终端不匹配	403
TERMINAL_DISABLED	终端已停用	403

3.7.8.5 Nginx TLS 代理配置

Nginx 配置示例

```
server {
    listen 8443 ssl;
    server_name localhost;

    # 服务器证书和私钥
    ssl_certificate     /path/to/server.crt;
    ssl_certificate_key /path/to/server.key;

    # 客户端证书验证
    ssl_client_certificate /path/to/ca.crt;
    ssl_verify_client optional;  # 允许但不强制要求客户端证书

    # SSL 协议和加密套件
    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_ciphers HIGH:!aNULL:!MD5;

    location / {
        proxy_pass http://127.0.0.1:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;

        # 转发客户端证书信息
        proxy_set_header X-Client-Cert $ssl_client_escaped_cert;
        proxy_set_header X-Verified-Client-Cert $ssl_client_escaped_cert;
        proxy_set_header X-Client-Cert-Verify $ssl_client_verify;
        proxy_set_header X-Verified-Client-IP $remote_addr;
    }
}
```

配置说明

- `ssl_client_certificate`：CA 根证书路径，用于验证客户端证书
- `ssl_verify_client`：客户端证书验证模式（`optional` 允许但不强制，`on` 强制要求）
- `X-Client-Cert`：转发客户端证书 PEM 编码（URL 转义）
- `X-Client-Cert-Verify`：转发证书验证结果（`SUCCESS` 或 `FAILED`）

3.7.8.6 证书生成与管理

证书生成流程

```
# 1. 生成 CA 根证书
openssl genrsa -out ca.key 4096
openssl req -new -x509 -days 3650 -key ca.key -out ca.crt

# 2. 生成服务器证书
openssl genrsa -out server.key 2048
openssl req -new -key server.key -out server.csr
openssl x509 -req -days 3650 -in server.csr -CA ca.crt -CAkey ca.key -CAcreateserial -out server.crt

# 3. 生成客户端证书（每台终端一个）
openssl genrsa -out terminal001.key 2048
openssl req -new -key terminal001.key -out terminal001.csr
openssl x509 -req -days 3650 -in terminal001.csr -CA ca.crt -CAkey ca.key -CAcreateserial -out terminal

# 4. 计算证书指纹
openssl x509 -in terminal001.crt -noout -fingerprint -sha256 | sed 's/://g' | cut -d'=' -f2
```

证书管理最佳实践

1. **证书有效期**: 建议设置为 1-3 年，到期前及时更新
2. **证书保护**: 私钥文件权限设置为 600，仅允许特定用户访问
3. **证书备份**: 保留 CA 根证书和私钥的备份，用于重新签发客户端证书
4. **证书撤销**: 建立证书撤销列表（CRL）或使用 OCSP 协议
5. **证书监控**: 监控证书过期时间，提前通知管理员更新

3.7.8.7 开发环境配置

开发环境 mTLS 配置

在开发环境中，可以通过环境变量控制 mTLS 行为：

```
# settings/dev.py

# 开发环境默认关闭 mTLS，方便测试
MTLS_ENABLED = os.getenv('MTLS_ENABLED', 'False').lower() == 'true'

# 开发环境信任本地回环地址
POS_TRUSTED_TLS_PROXY_IPS = '127.0.0.1,::1'

# 开发环境豁免更多路径
MTLS_EXEMPT_PATHS = [
    '/pos/api/certificate/verify/',
    '/admin/',
]
```

开发环境测试

开发环境可以通过以下方式测试 mTLS：

1. **使用 curl 测试**:

```
curl -k --cert /path/to/client.crt \
  --key /path/to/client.key \
  -H "X-POS-Terminal-Code: TERMINAL001" \
  https://localhost:8443/pos/settlement/
```

2. **使用浏览器测试**: 浏览器会提示选择客户端证书

3. **禁用 mTLS**: 设置 `MTLS_ENABLED=False` 跳过 mTLS 检查

3.7.8.8 安全考虑

安全特性

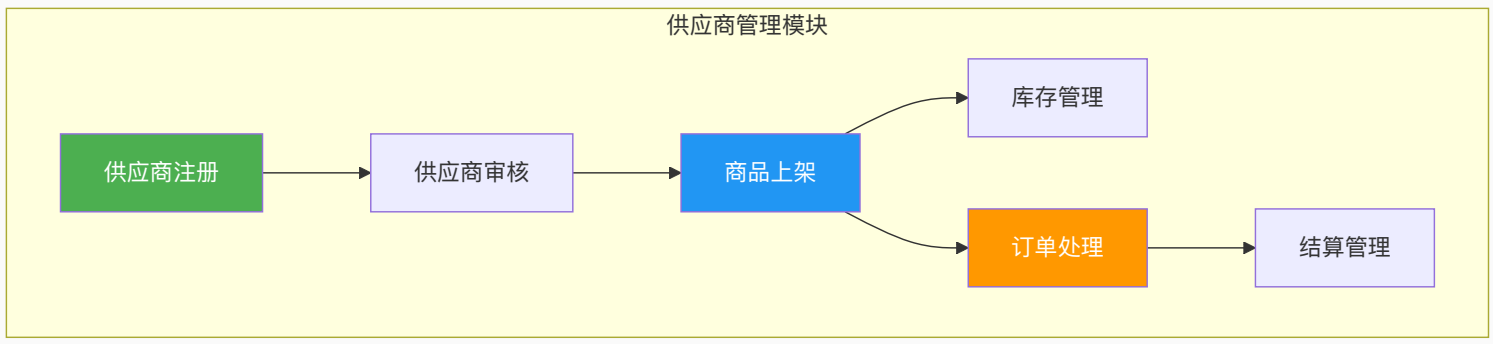
1. **双向认证**: 服务器和客户端互相验证身份
2. **证书指纹**: 使用 SHA-256 指纹确保证书唯一性
3. **IP 白名单**: 仅信任指定的 TLS 代理 IP
4. **路径豁免**: 某些路径（如证书验证接口）豁免 mTLS 检查
5. **终端隔离**: 每台终端独立配置证书，互不影响

安全建议

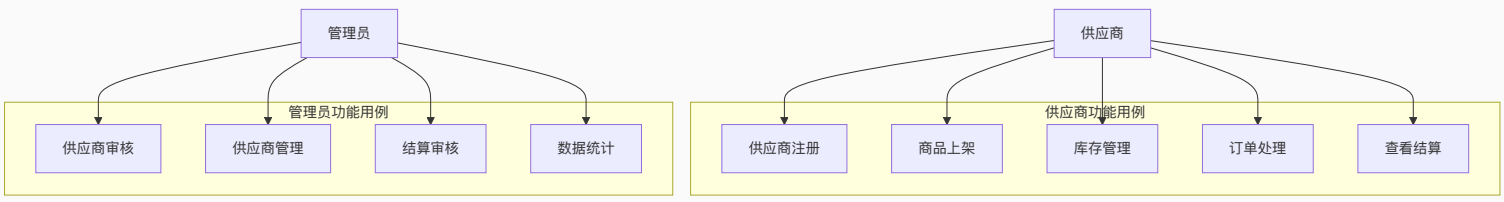
1. **证书保护**: 客户端证书私钥必须妥善保管，不得泄露
2. **证书更新**: 定期更新客户端证书，避免长期使用同一证书
3. **监控告警**: 监控证书验证失败的情况，及时发现异常
4. **日志审计**: 记录所有 mTLS 认证请求，便于事后审计
5. **最小权限**: 仅为必要的终端启用 mTLS，减少攻击面

3.8 供应商管理模块

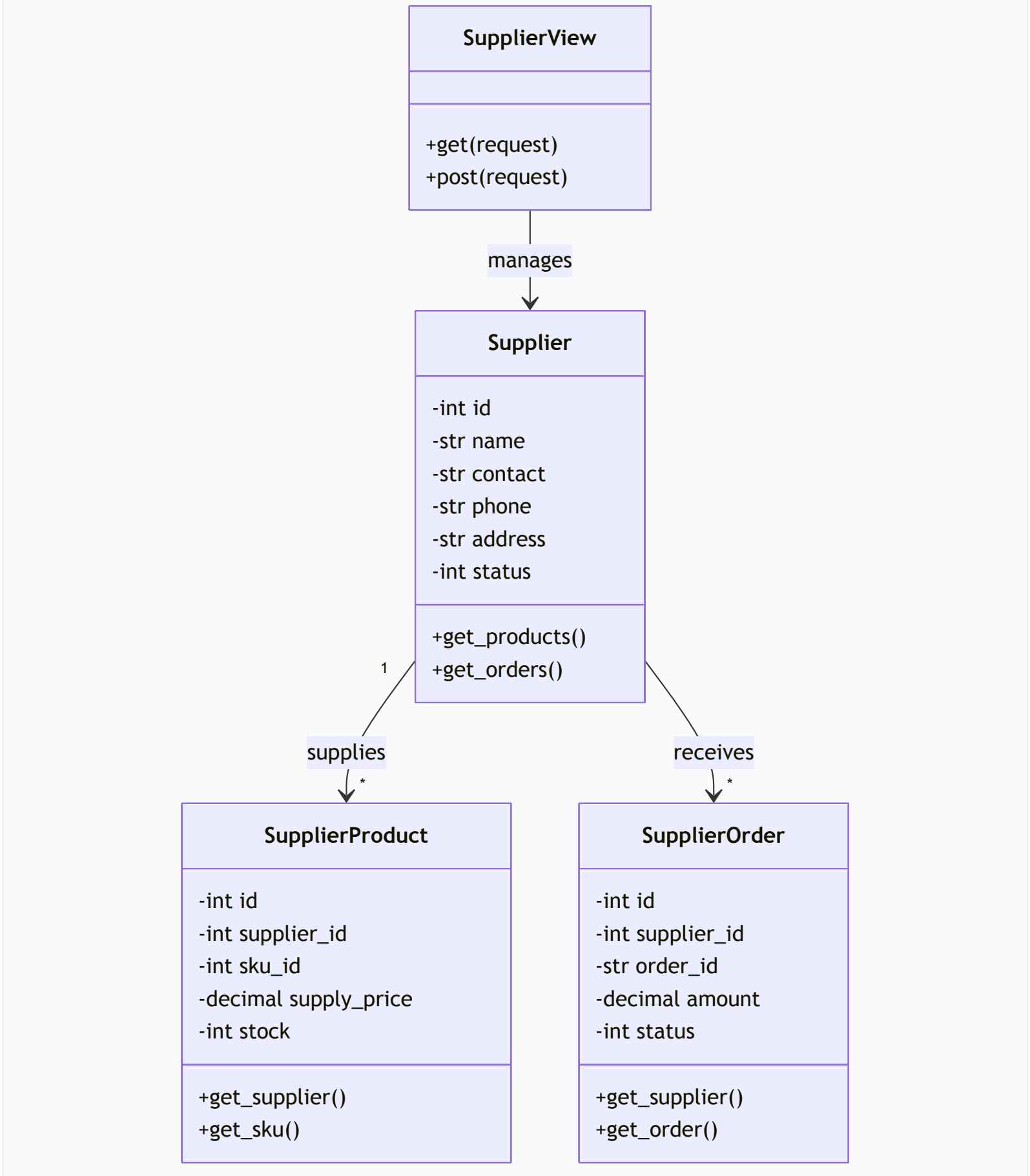
3.8.1 供应商模块功能图



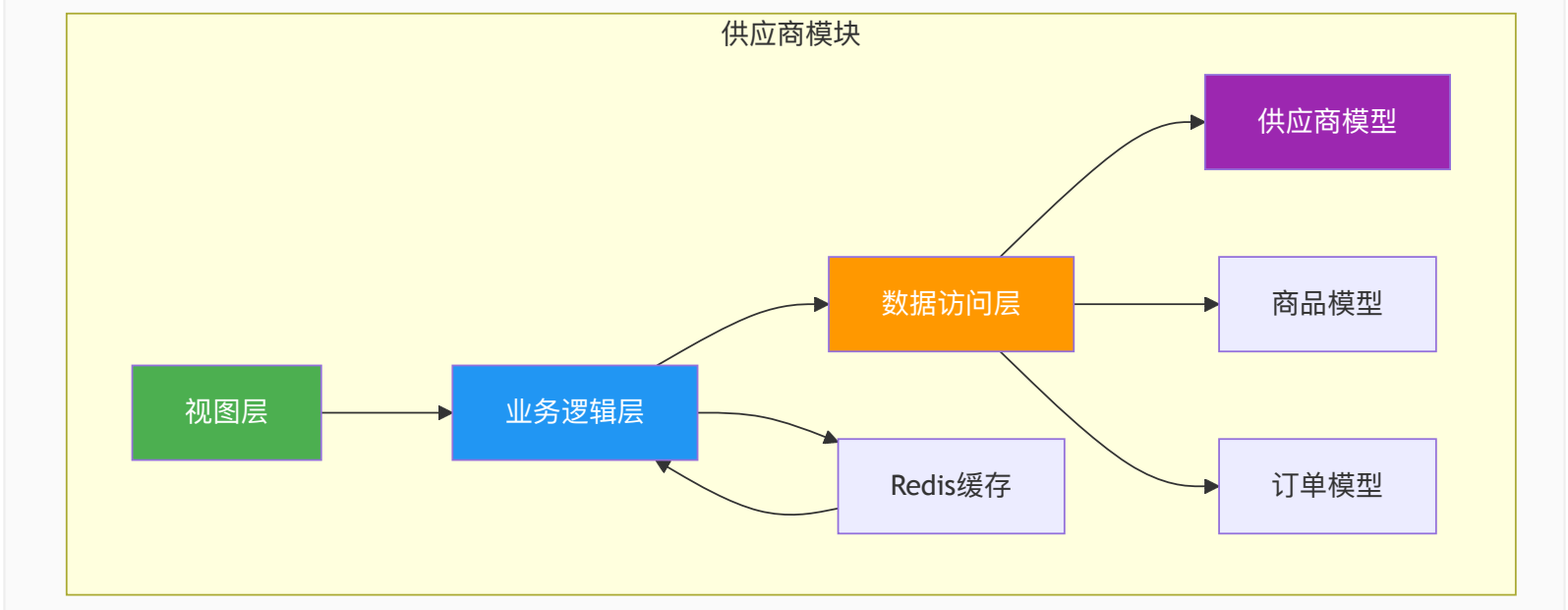
3.8.2 供应商模块用例图



3.8.3 供应商模块类图

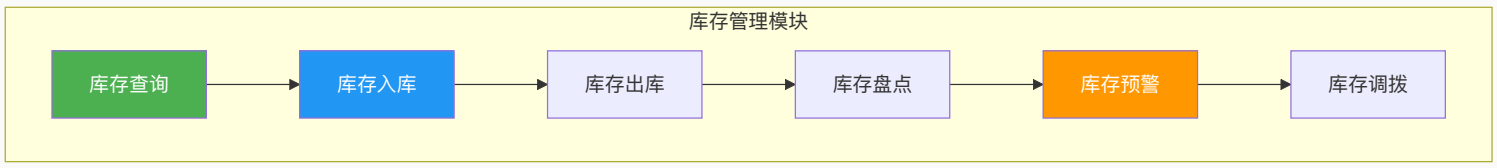


3.8.4 供应商模块内部交互拓扑图

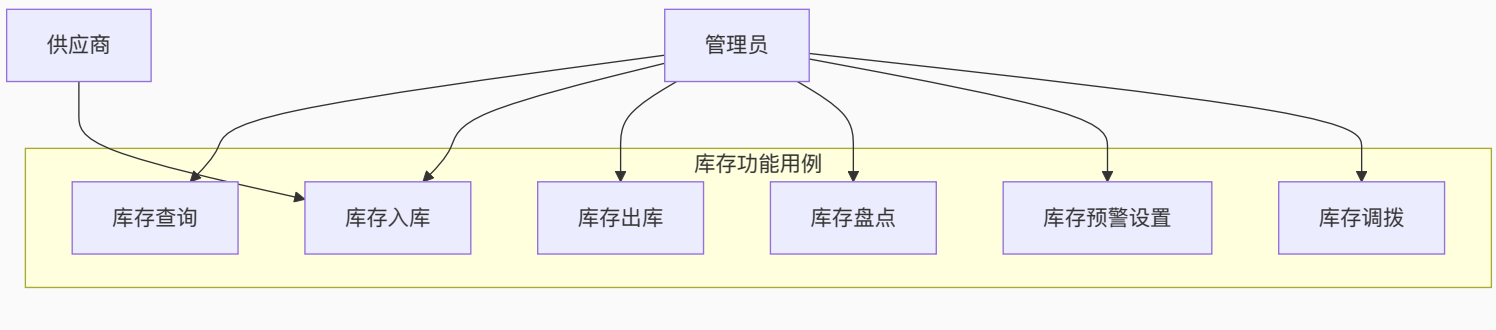


3.9 库存管理模块

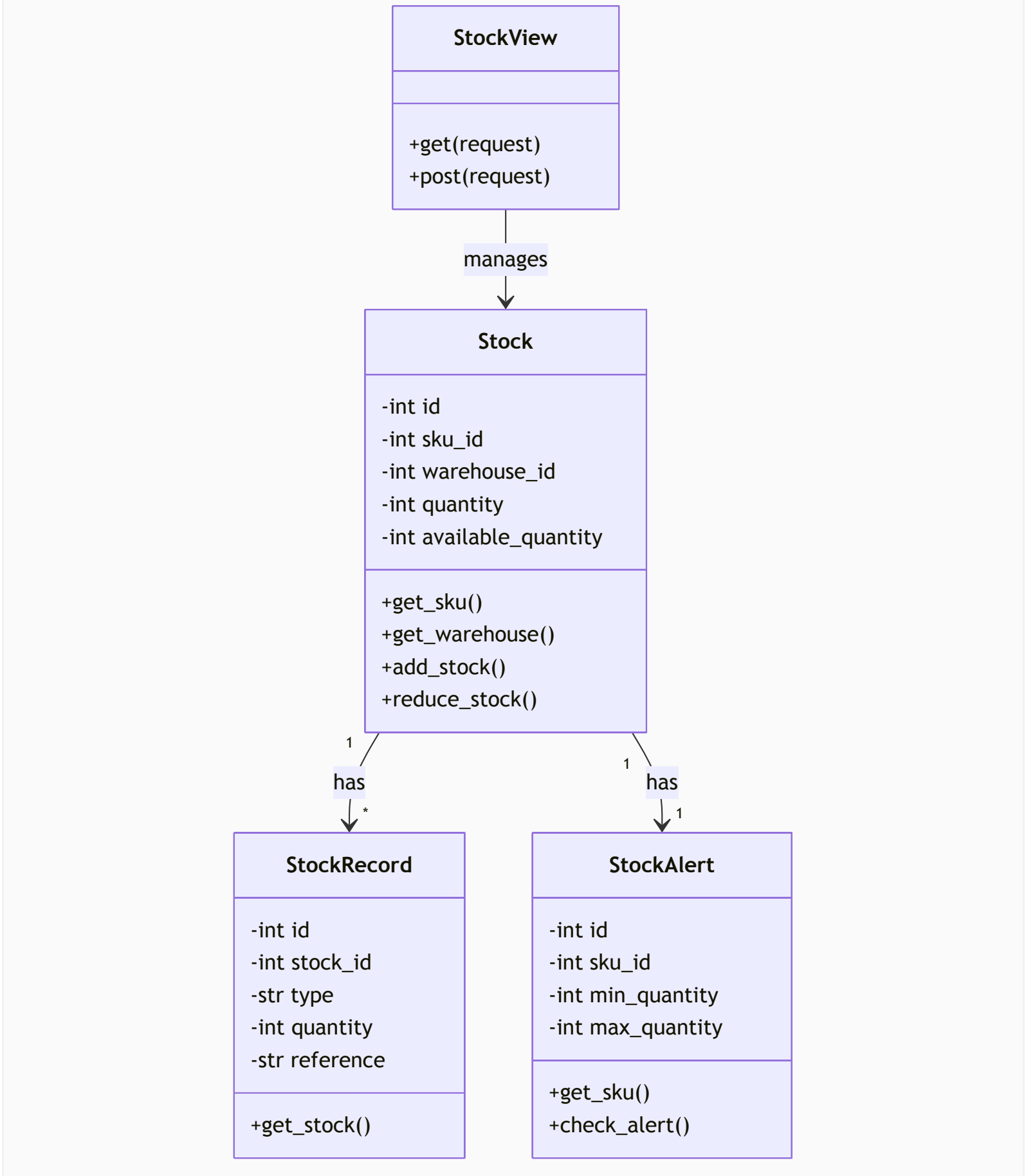
3.9.1 库存模块功能图



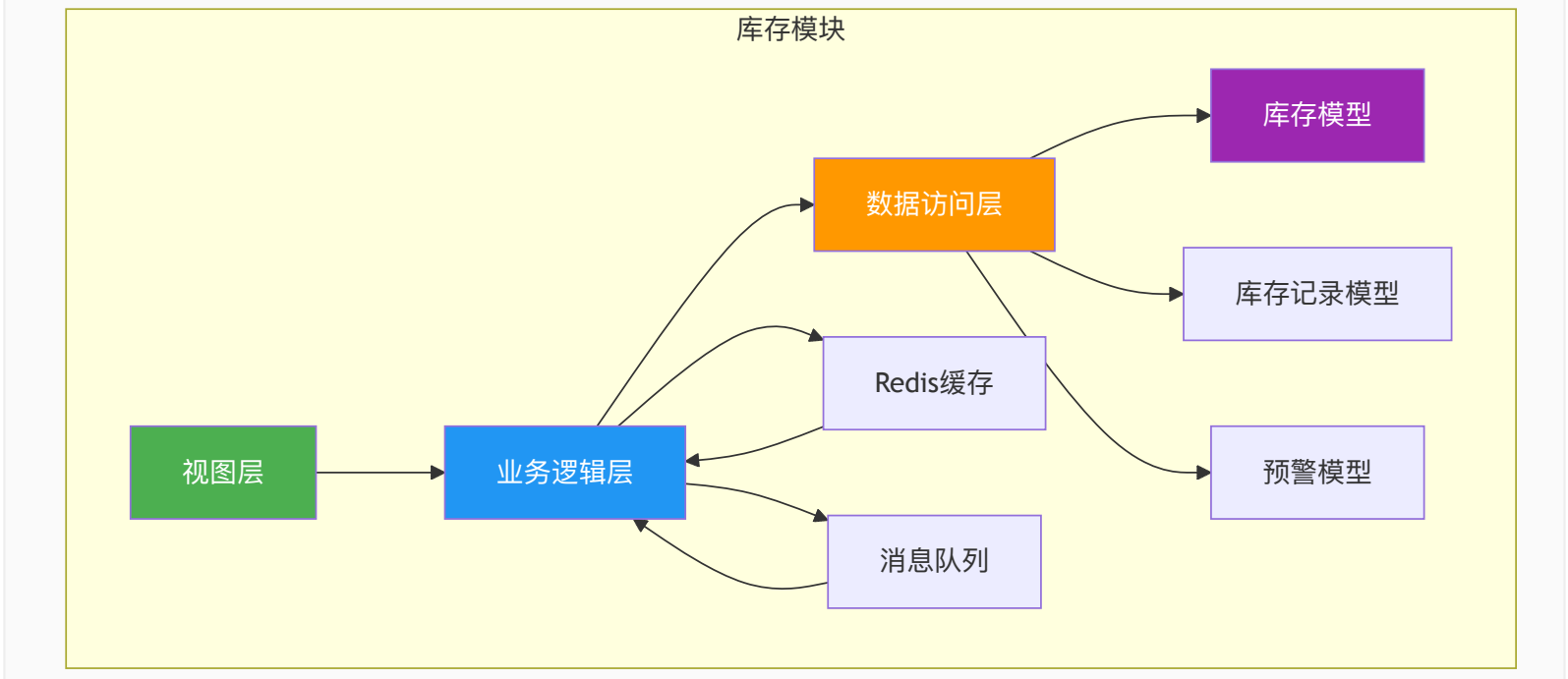
3.9.2 库存模块用例图



3.9.3 库存模块类图

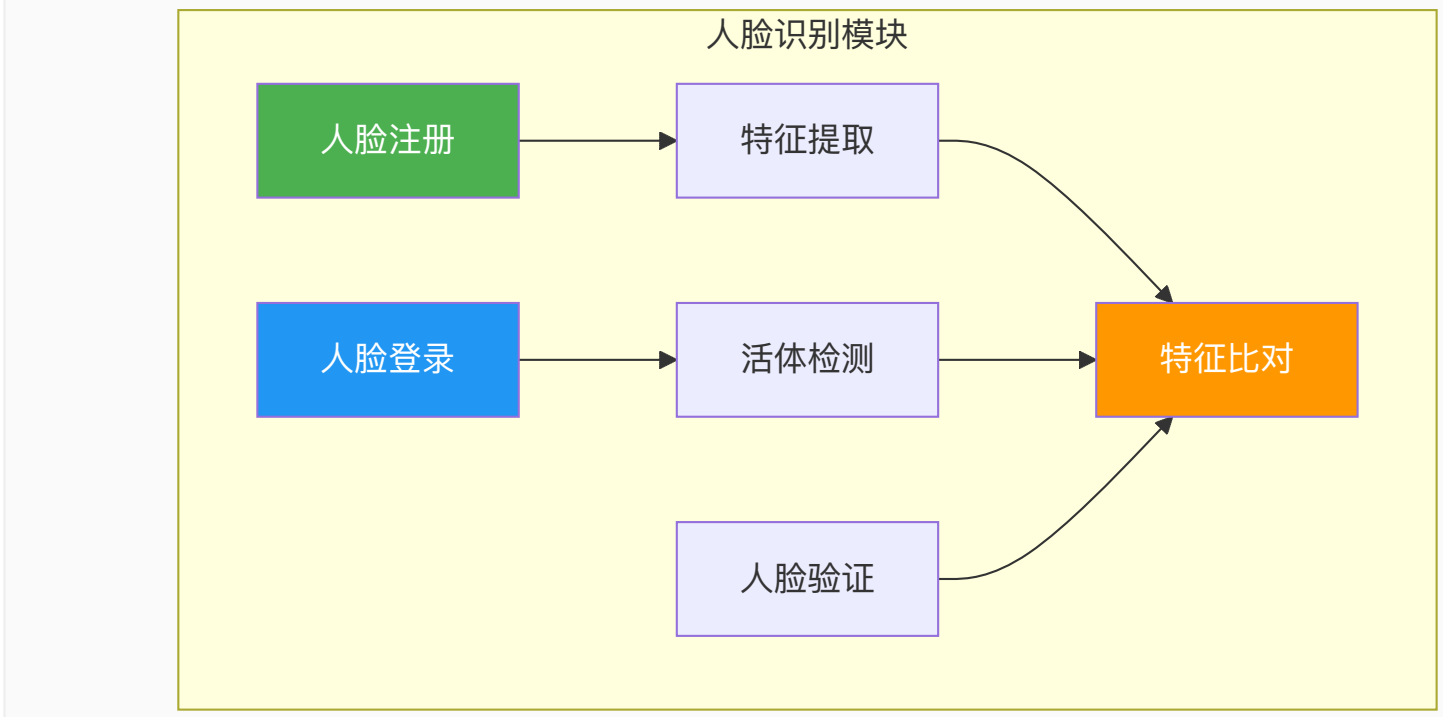


3.9.4 库存模块内部交互拓扑图

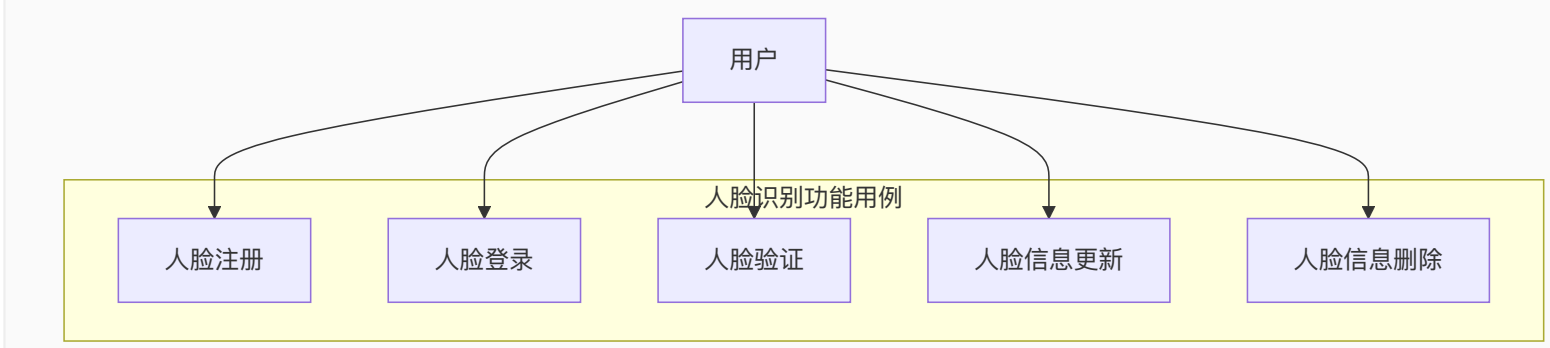


3.10 人脸识别模块

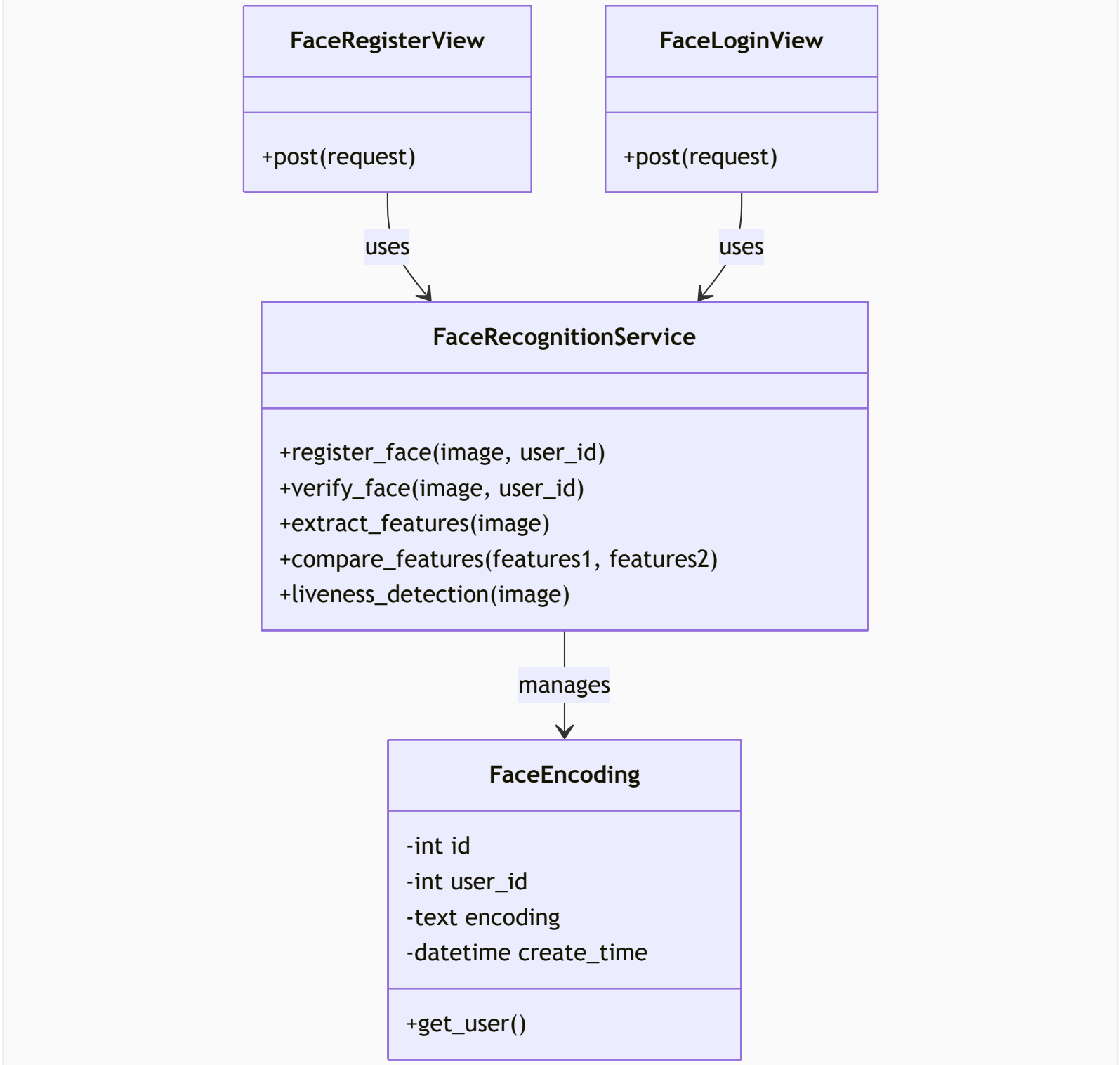
3.10.1 人脸识别模块功能图



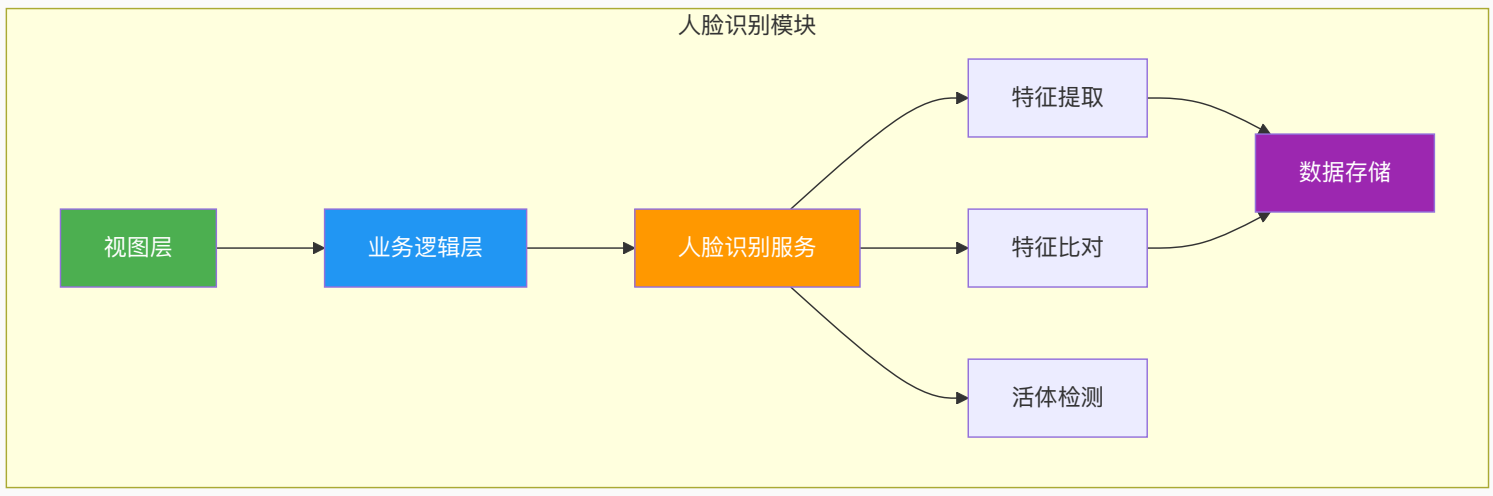
3.10.2 人脸识别模块用例图



3.10.3 人脸识别模块类图

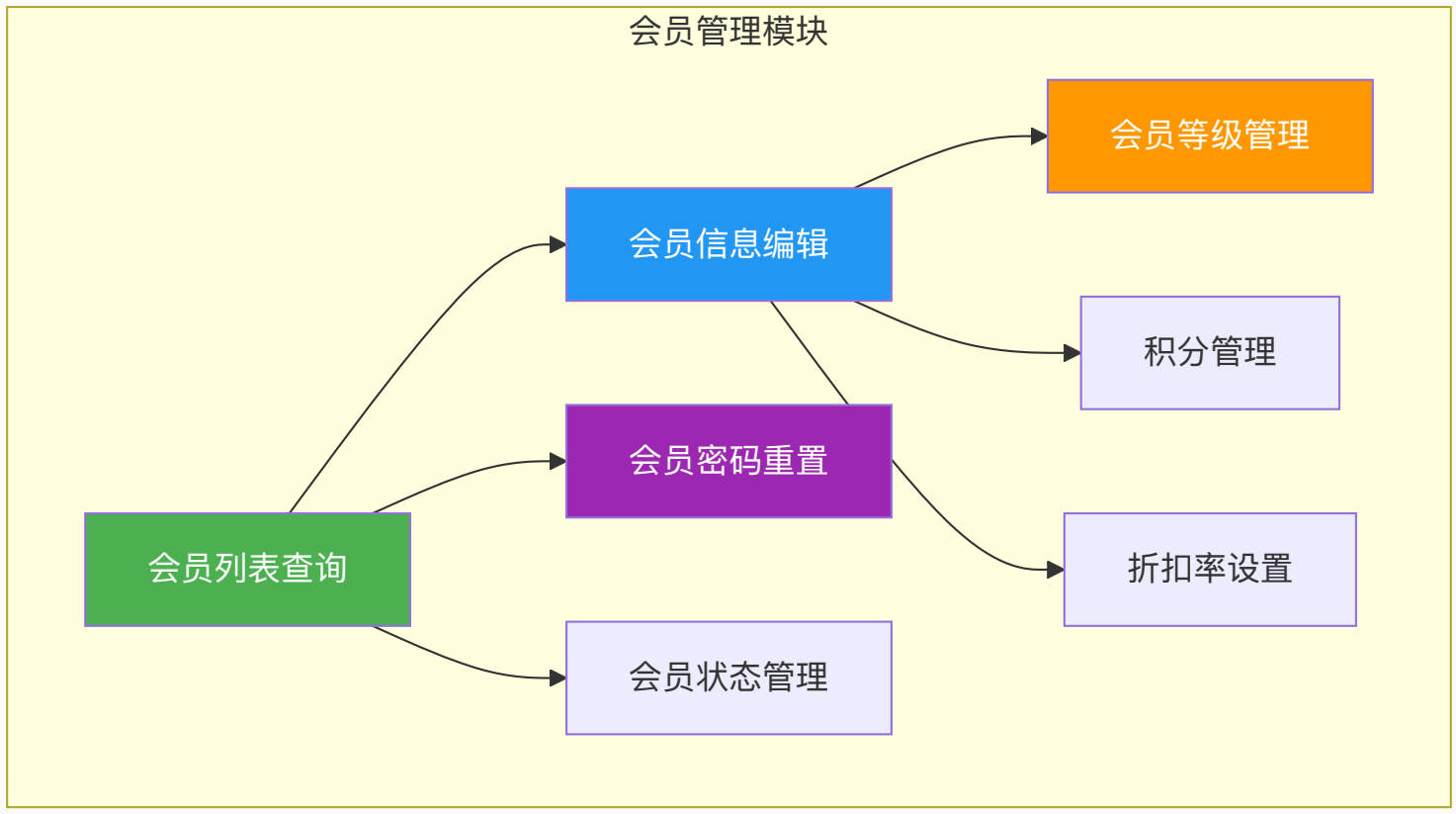


3.10.4 人脸识别模块内部交互拓扑图

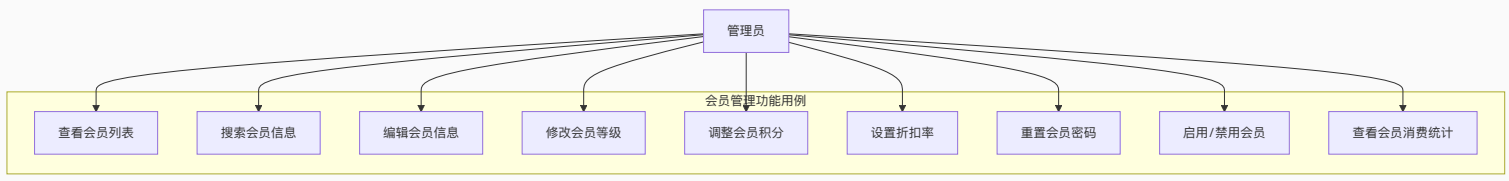


3.11 会员管理模块

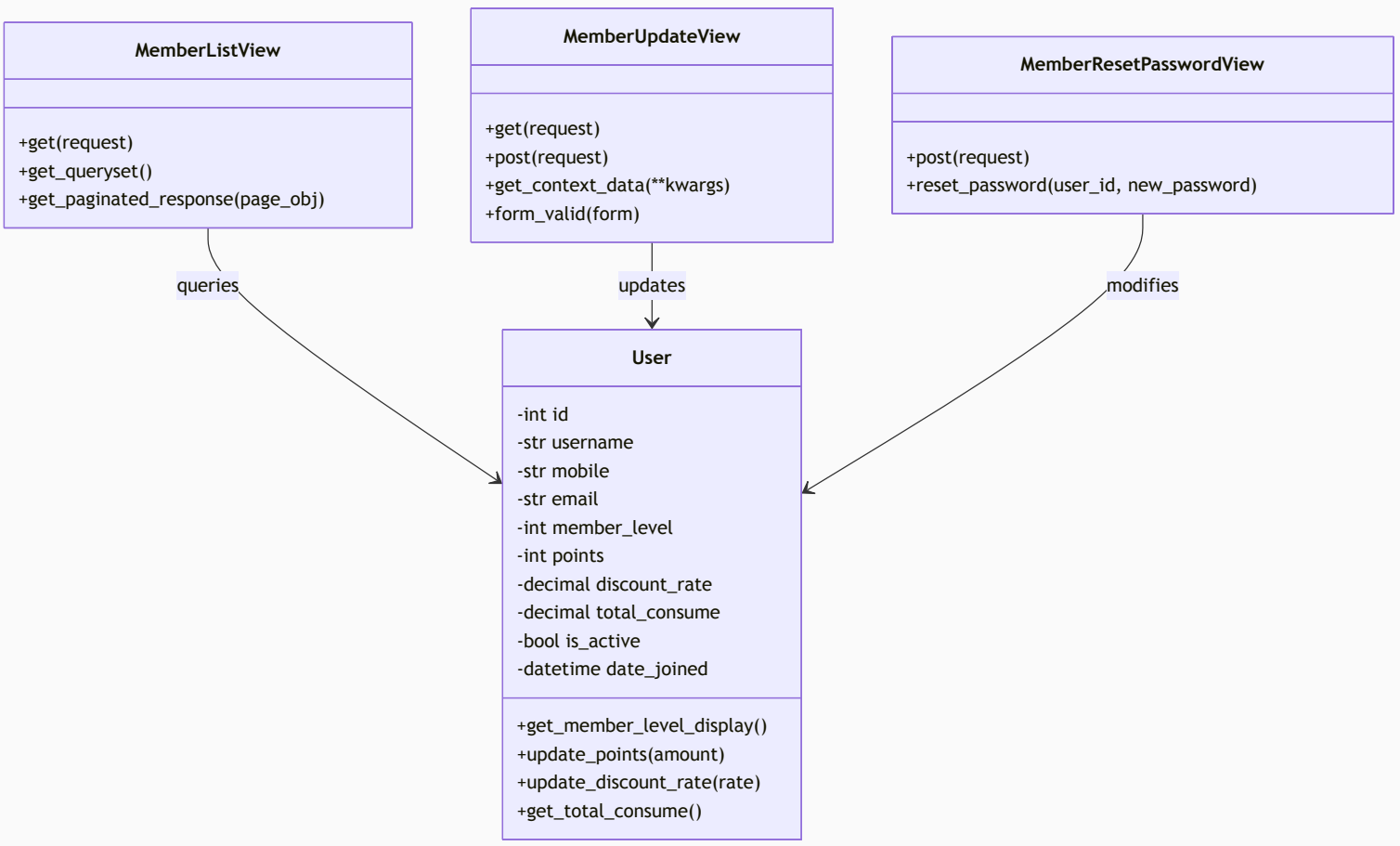
3.11.1 会员管理模块功能图



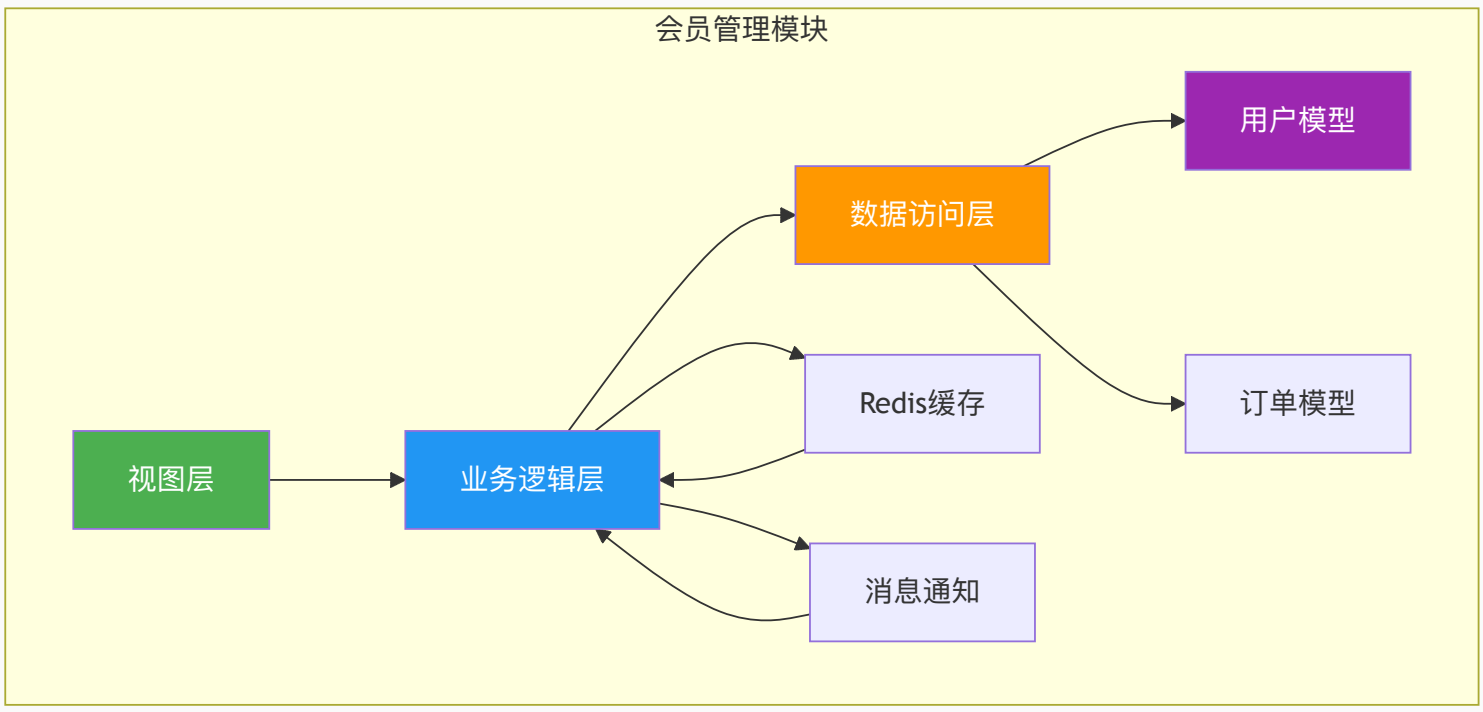
3.11.2 会员管理模块用例图



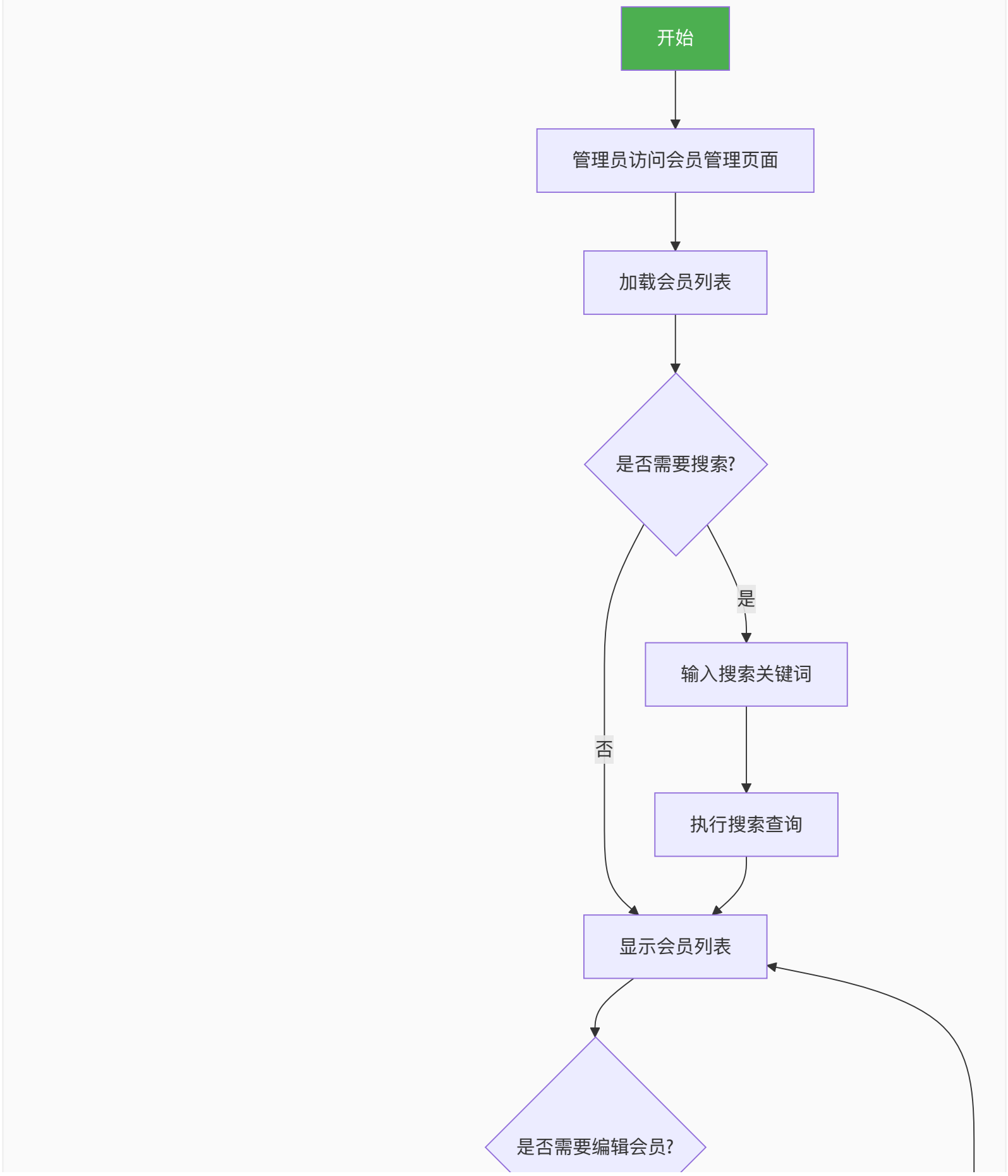
3.11.3 会员管理模块类图

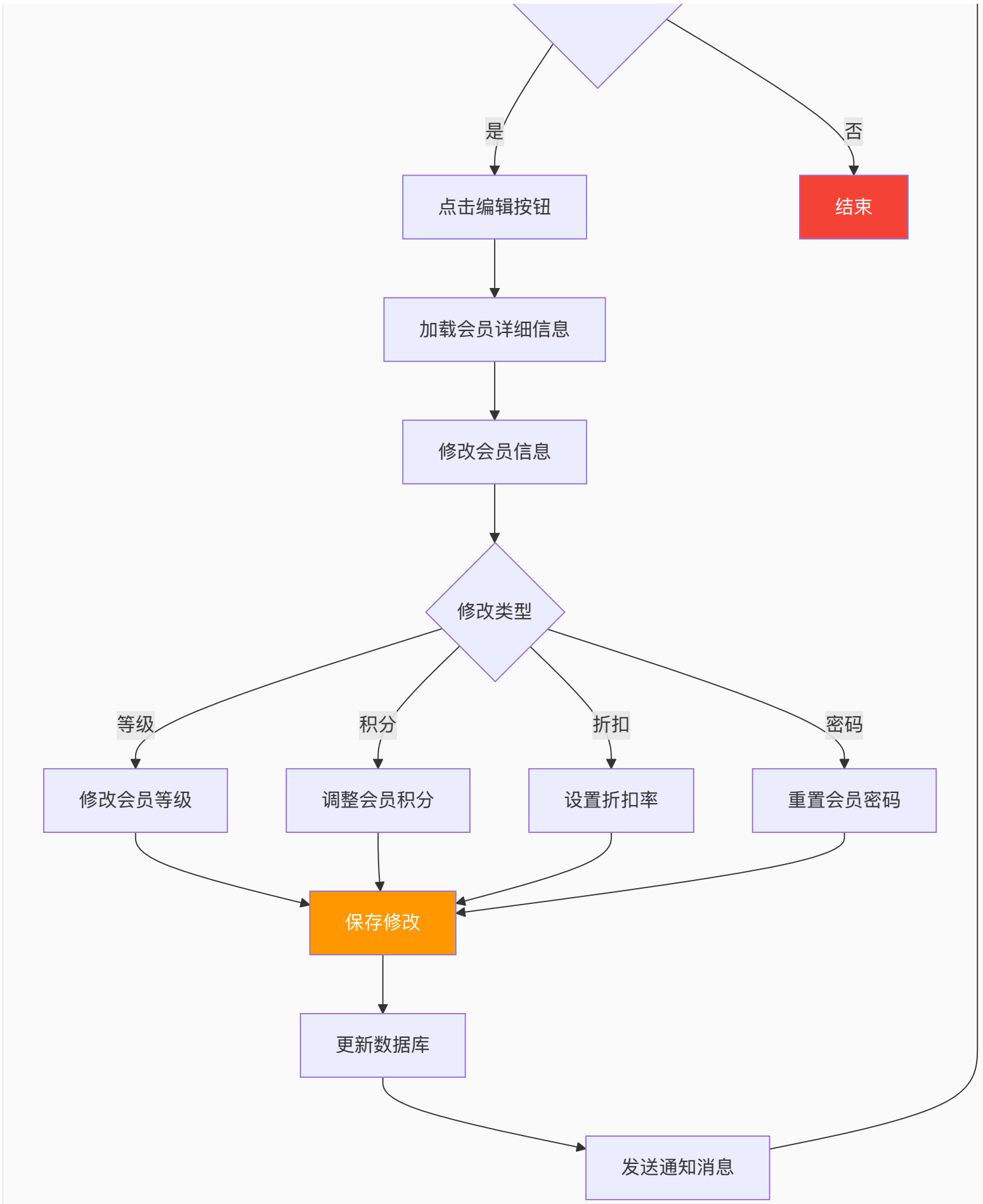


3.11.4 会员管理模块内部交互拓扑图



3.11.5 会员管理模块业务流程图





3.11.6 会员等级设计

会员等级分类

等级	等级值	名称	折扣率	积分倍数	说明
普通会员	0	普通会员	1.00	1.0	默认注册用户
银卡会员	1	银卡会员	0.95	1.2	消费满1000元
金卡会员	2	金卡会员	0.90	1.5	消费满5000元
钻石会员	3	钻石会员	0.85	2.0	消费满10000元

会员等级升级规则

- 普通会员 → 银卡会员：累计消费满1000元
- 银卡会员 → 金卡会员：累计消费满5000元
- 金卡会员 → 钻石会员：累计消费满10000元

- 会员等级只升不降

积分获取规则

- 普通会员：消费1元获得1积分
- 银卡会员：消费1元获得1.2积分
- 金卡会员：消费1元获得1.5积分
- 钻石会员：消费1元获得2积分

3.11.7 会员管理模块API接口

会员列表查询接口

- **接口路径**: `/members/api/`
- **请求方法**: GET
- **请求参数**:
  - `page`: 页码
  - `keyword`: 搜索关键词（用户名/手机号）
- **返回数据**: 会员列表JSON

会员信息编辑接口

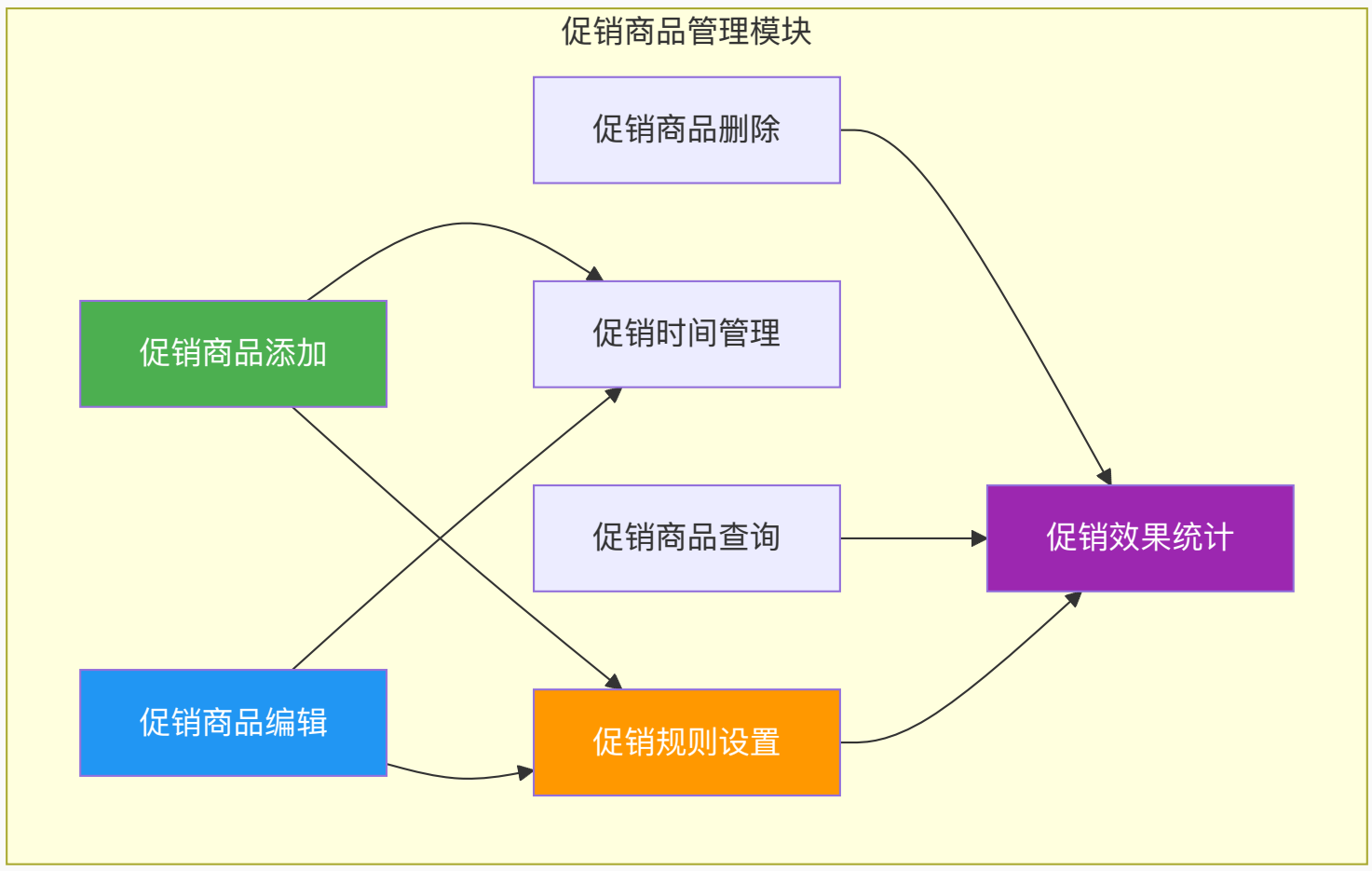
- **接口路径**: `/members/edit/{id}/`
- **请求方法**: POST
- **请求参数**:
  - `member_level`: 会员等级
  - `points`: 积分
  - `discount_rate`: 折扣率
- **返回数据**: 编辑结果

会员密码重置接口

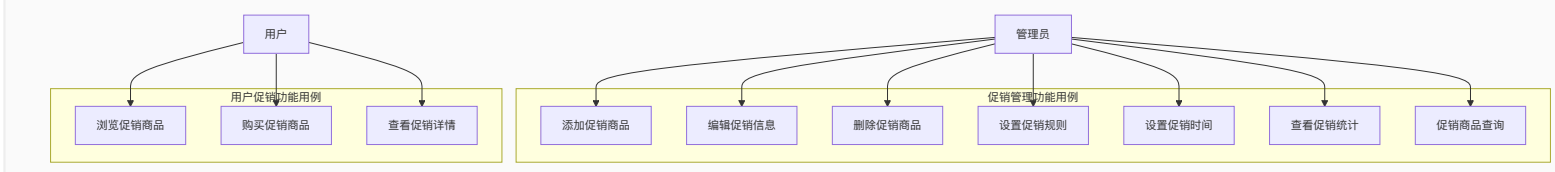
- **接口路径**: `/members/reset-password/{id}/`
- **请求方法**: POST
- **请求参数**:
  - `new_password`: 新密码
- **返回数据**: 重置结果

3.12 促销商品管理模块

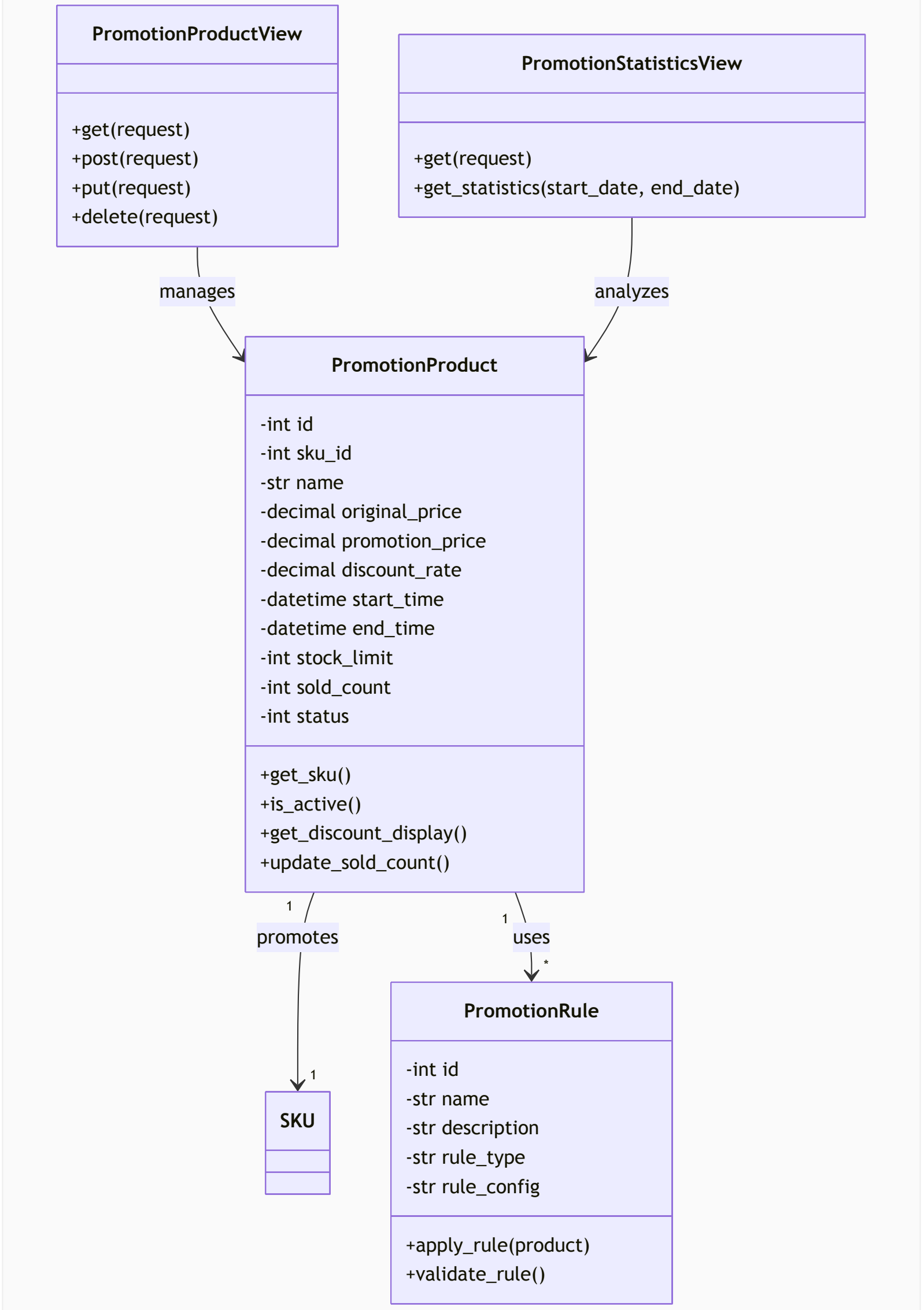
3.12.1 促销商品管理模块功能图



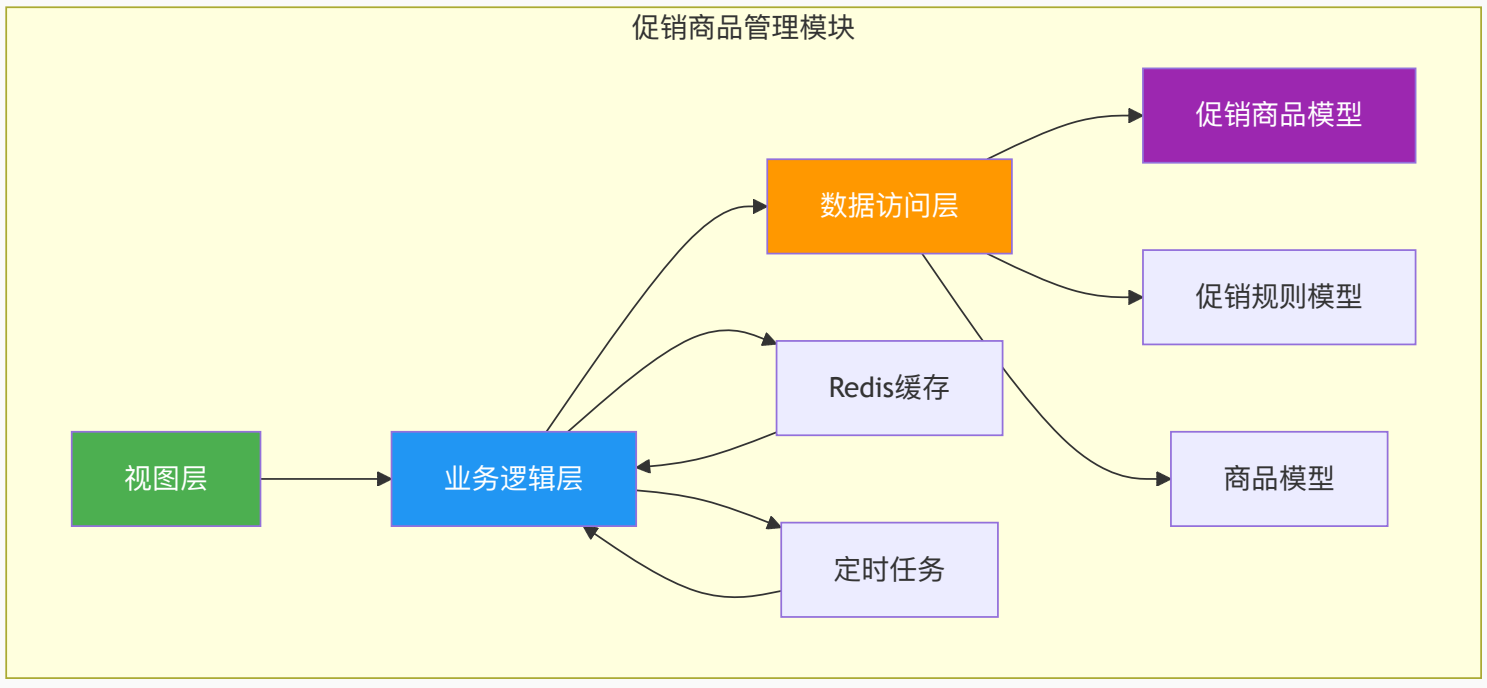
3.12.2 促销商品管理模块用例图



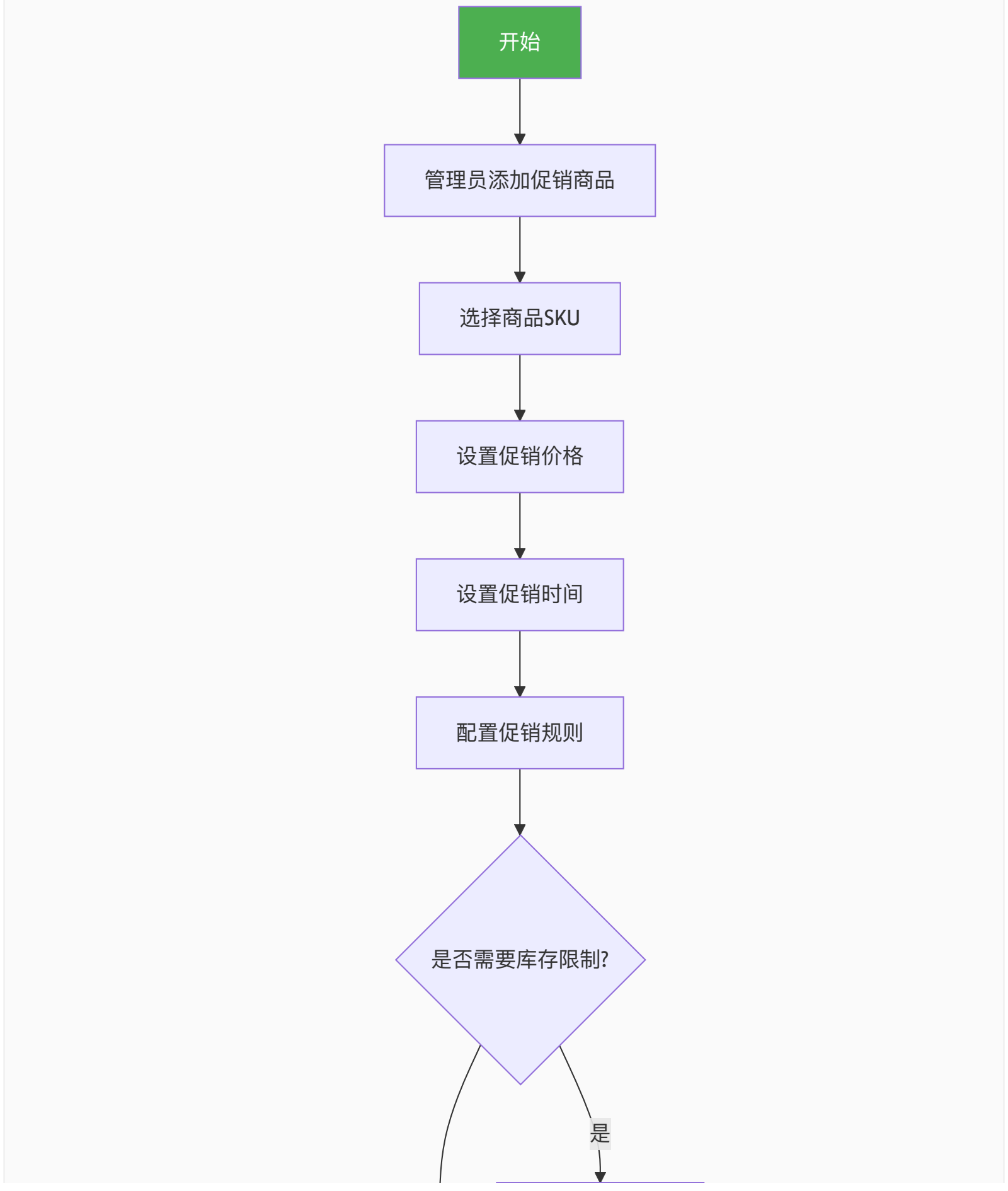
3.12.3 促销商品管理模块类图



3.12.4 促销商品管理模块内部交互拓扑图



3.12.5 促销商品管理流程图



否

设置库存限制

保存促销商品

更新促销商品列表

缓存促销信息

促销开始

当前时间是否在促销期内?

是

显示促销价格

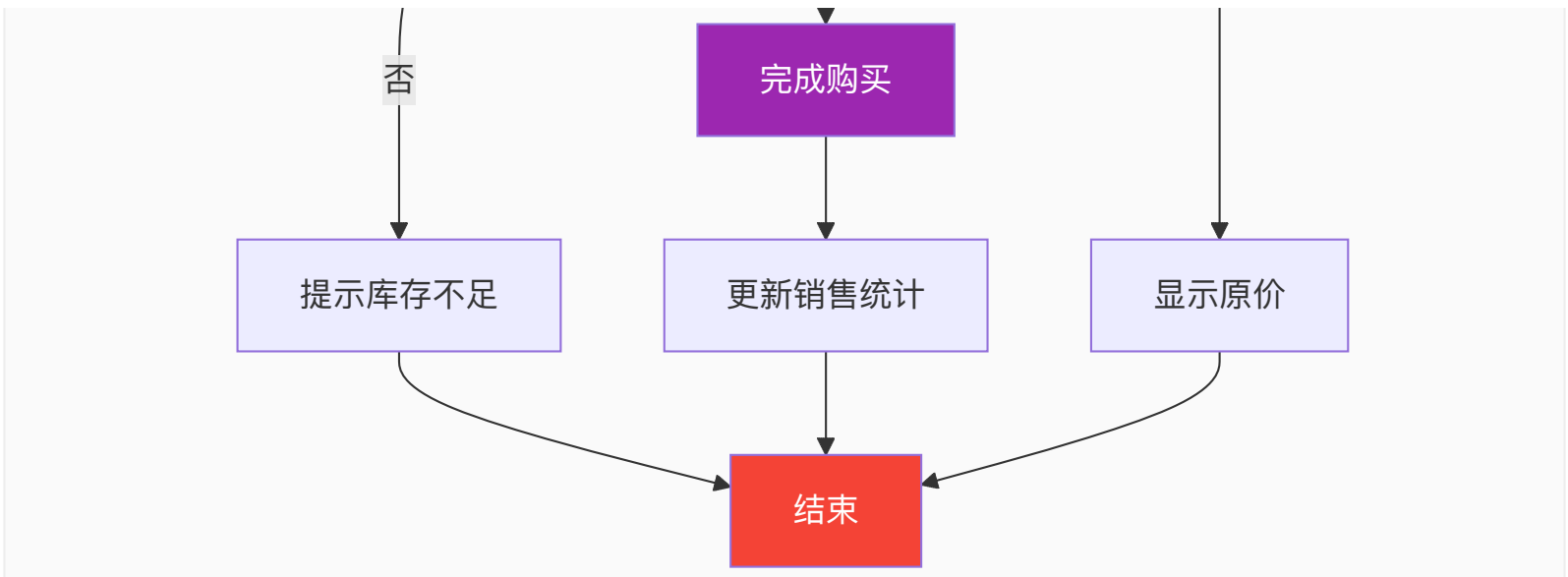
用户购买促销商品

更新销售数量

库存是否充足?

是

否



3.12.6 促销规则设计

促销类型

促销类型	代码	说明	适用场景
直接折扣	DISCOUNT	直接设置促销价格	日常促销
百分比折扣	PERCENTAGE	按百分比打折	节日促销
满减优惠	THRESHOLD	满额减免	大促活动
买赠活动	BOGO	买一赠一	清仓促销
限时秒杀	FLASH_SALE	限时抢购	爆款促销

促销状态

状态值	状态名称	说明
0	未开始	促销尚未开始
1	进行中	促销正在进行
2	已结束	促销已结束
3	已下架	促销商品已下架

3.12.7 促销商品数据表设计

tb_promotion_product (促销商品表)

字段名	类型	长度	允许空	默认值	说明
id	INT	11	NO	AUTO_INCREMENT	主键ID
sku_id	INT	11	NO	-	商品SKU ID
name	VARCHAR	200	NO	-	促销商品名称
original_price	DECIMAL	10,2	NO	-	原价
promotion_price	DECIMAL	10,2	NO	-	促销价格
discount_rate	DECIMAL	5,4	NO	-	折扣率
start_time	DATETIME	-	NO	-	促销开始时间
end_time	DATETIME	-	NO	-	促销结束时间
stock_limit	INT	11	YES	NULL	库存限制
sold_count	INT	11	NO	0	已售数量
status	SMALLINT	6	NO	0	促销状态
description	TEXT	-	YES	NULL	促销描述
create_time	DATETIME	-	NO	CURRENT_TIMESTAMP	创建时间
update_time	DATETIME	-	NO	CURRENT_TIMESTAMP	更新时间

索引设计

- PRIMARY KEY: `id`
- INDEX: `sku_id`
- INDEX: `status`
- INDEX: `start_time`
- INDEX: `end_time`

tb_promotion_rule (促销规则表)

字段名	类型	长度	允许空	默认值	说明
id	INT	11	NO	AUTO_INCREMENT	主键ID
name	VARCHAR	100	NO	-	规则名称
rule_type	VARCHAR	50	NO	-	规则类型
rule_config	JSON	-	NO	-	规则配置
description	VARCHAR	500	YES	NULL	规则描述
is_active	BOOLEAN	-	NO	True	是否启用
create_time	DATETIME	-	NO	CURRENT_TIMESTAMP	创建时间
update_time	DATETIME	-	NO	CURRENT_TIMESTAMP	更新时间

索引设计

- PRIMARY KEY: `id`
- INDEX: `rule_type`
- INDEX: `is_active`

3.12.8 促销商品管理模块API接口

促销商品列表接口

- **接口路径**: `/promotions/api/`
- **请求方法**: GET
- **请求参数**:
  - `page`: 页码
  - `status`: 促销状态
  - `keyword`: 搜索关键词
- **返回数据**: 促销商品列表JSON

添加促销商品接口

- **接口路径**: `/promotions/add/`
- **请求方法**: POST
- **请求参数**:
  - `sku_id`: 商品SKU ID
  - `promotion_price`: 促销价格
  - `start_time`: 开始时间
  - `end_time`: 结束时间
  - `stock_limit`: 库存限制
  - `description`: 促销描述
- **返回数据**: 添加结果

编辑促销商品接口

- **接口路径**: `/promotions/edit/{id}/`
- **请求方法**: POST
- **请求参数**:
  - `promotion_price`: 促销价格
  - `start_time`: 开始时间
  - `end_time`: 结束时间
  - `stock_limit`: 库存限制
  - `status`: 促销状态
- **返回数据**: 编辑结果

促销统计接口

- **接口路径**: `/promotions/statistics/`
- **请求方法**: GET
- **请求参数**:
  - `start_date`: 开始日期
  - `end_date`: 结束日期
- **返回数据**: 促销统计数据JSON

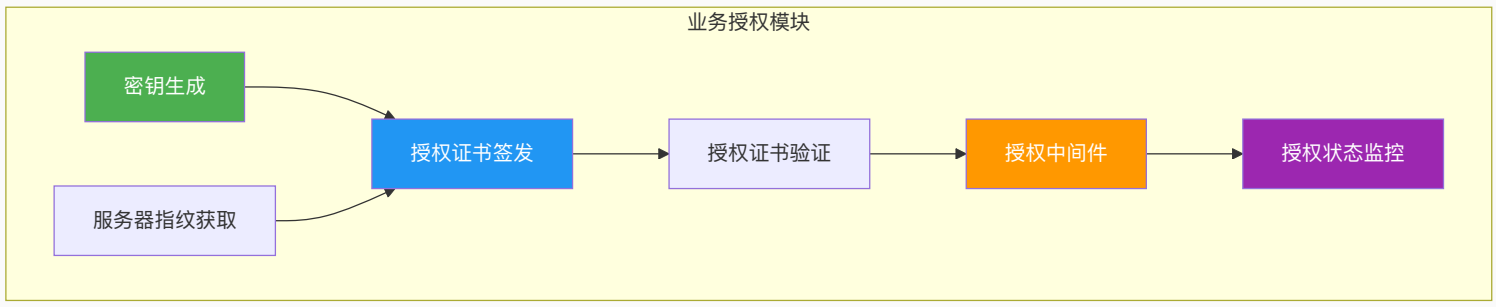
3.12.9 促销商品功能特点

1. **灵活配置**: 支持多种促销类型和规则配置
2. **时间控制**: 精确控制促销开始和结束时间
3. **库存管理**: 支持促销商品库存限制
4. **实时统计**: 提供促销效果实时统计分析
5. **缓存优化**: 使用Redis缓存促销信息，提高访问速度
6. **自动下架**: 促销结束后自动下架促销商品

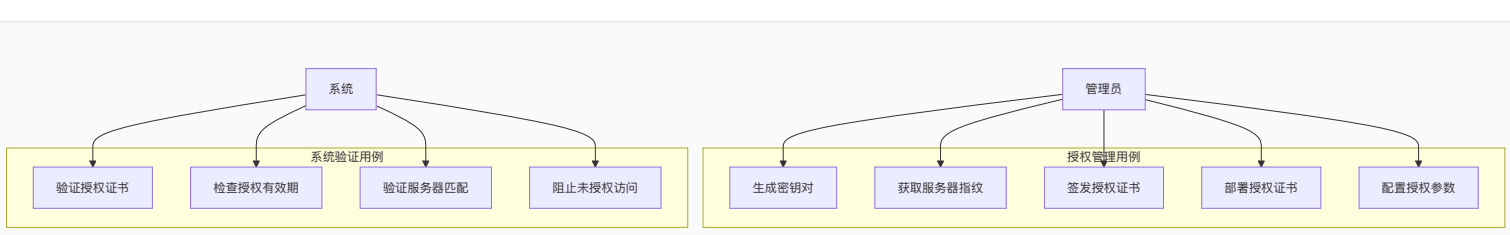
7. **销售追踪**: 实时追踪促销商品销售情况

3.13 业务授权模块

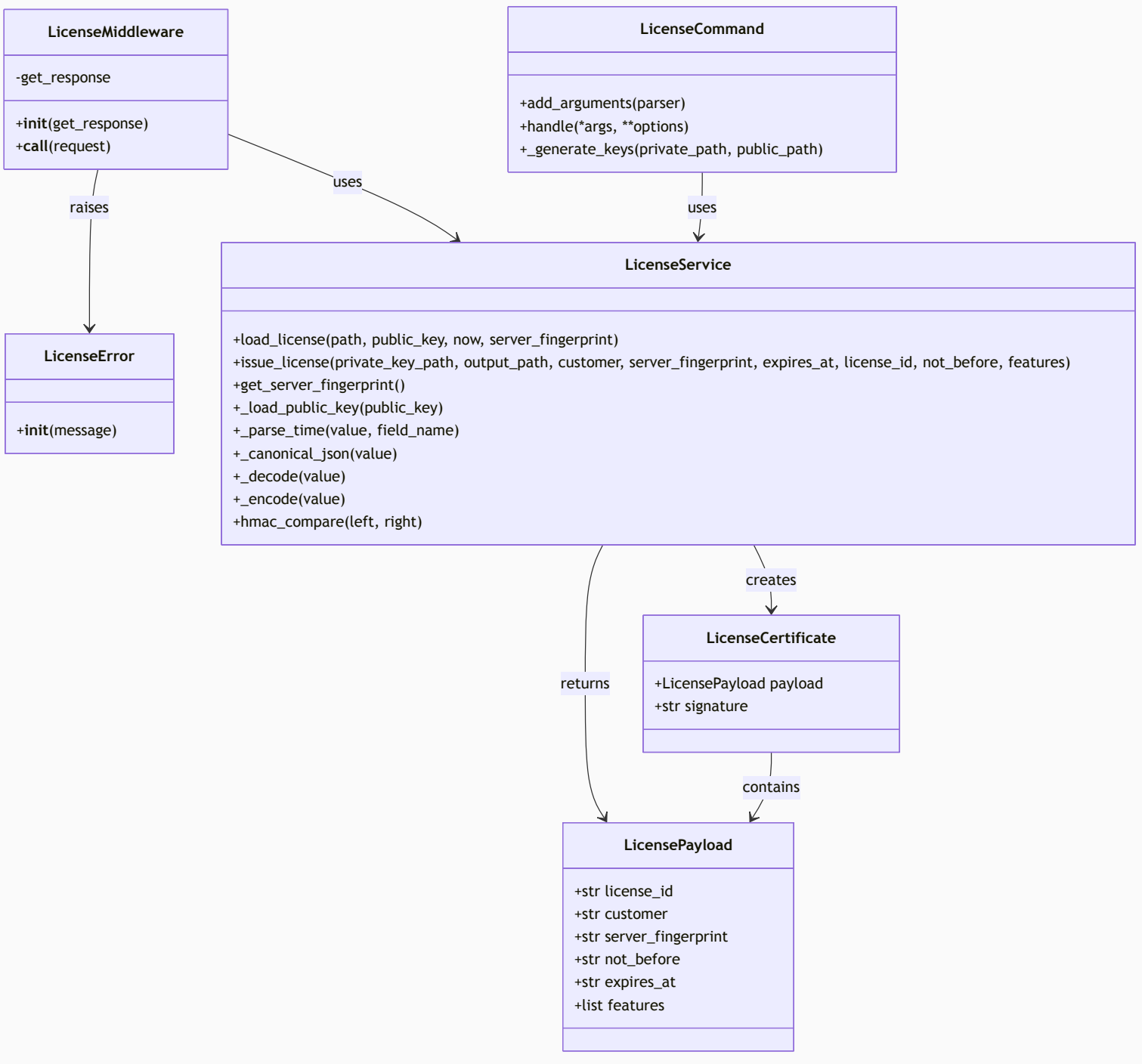
3.13.1 业务授权模块功能图



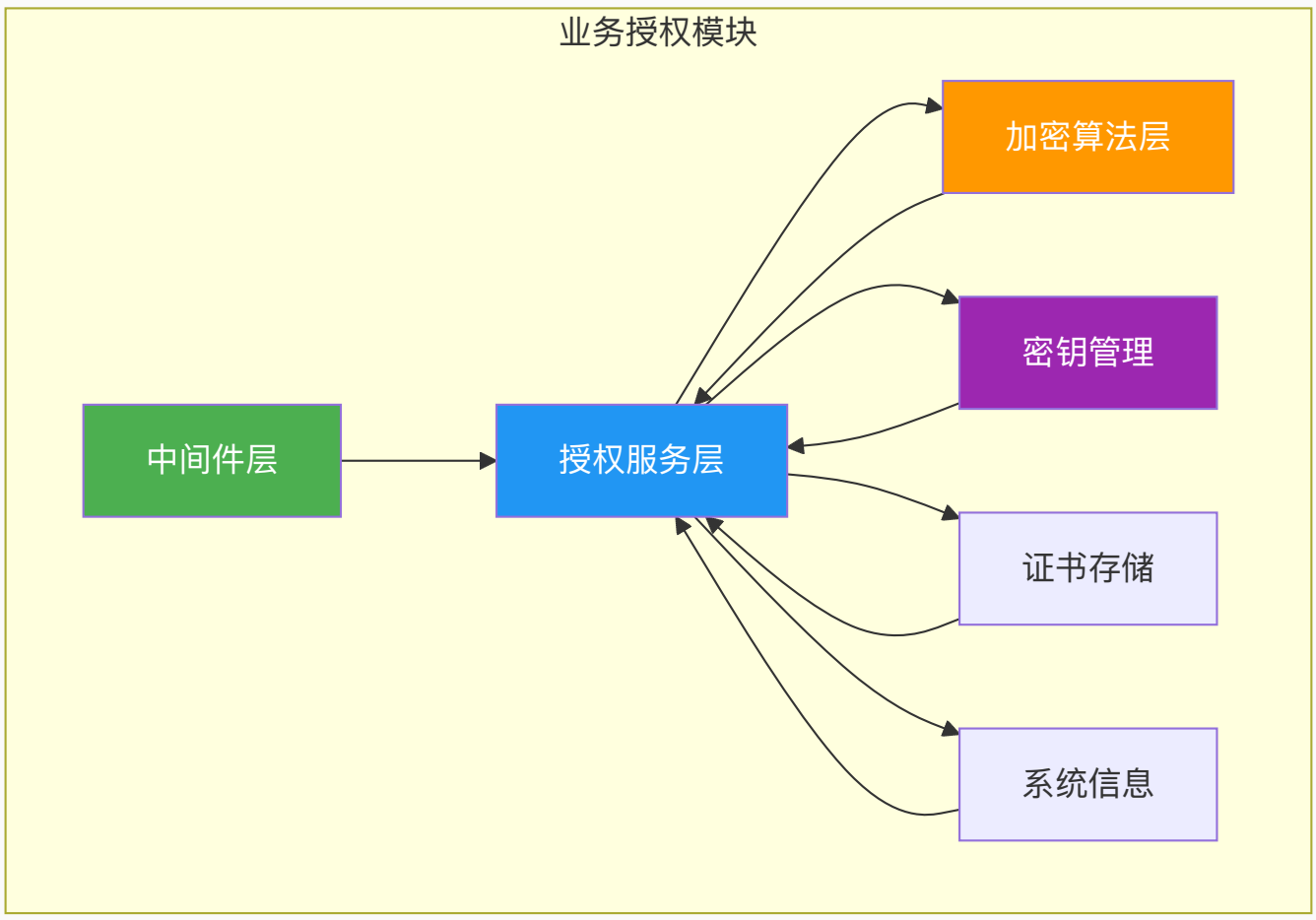
3.13.2 业务授权模块用例图



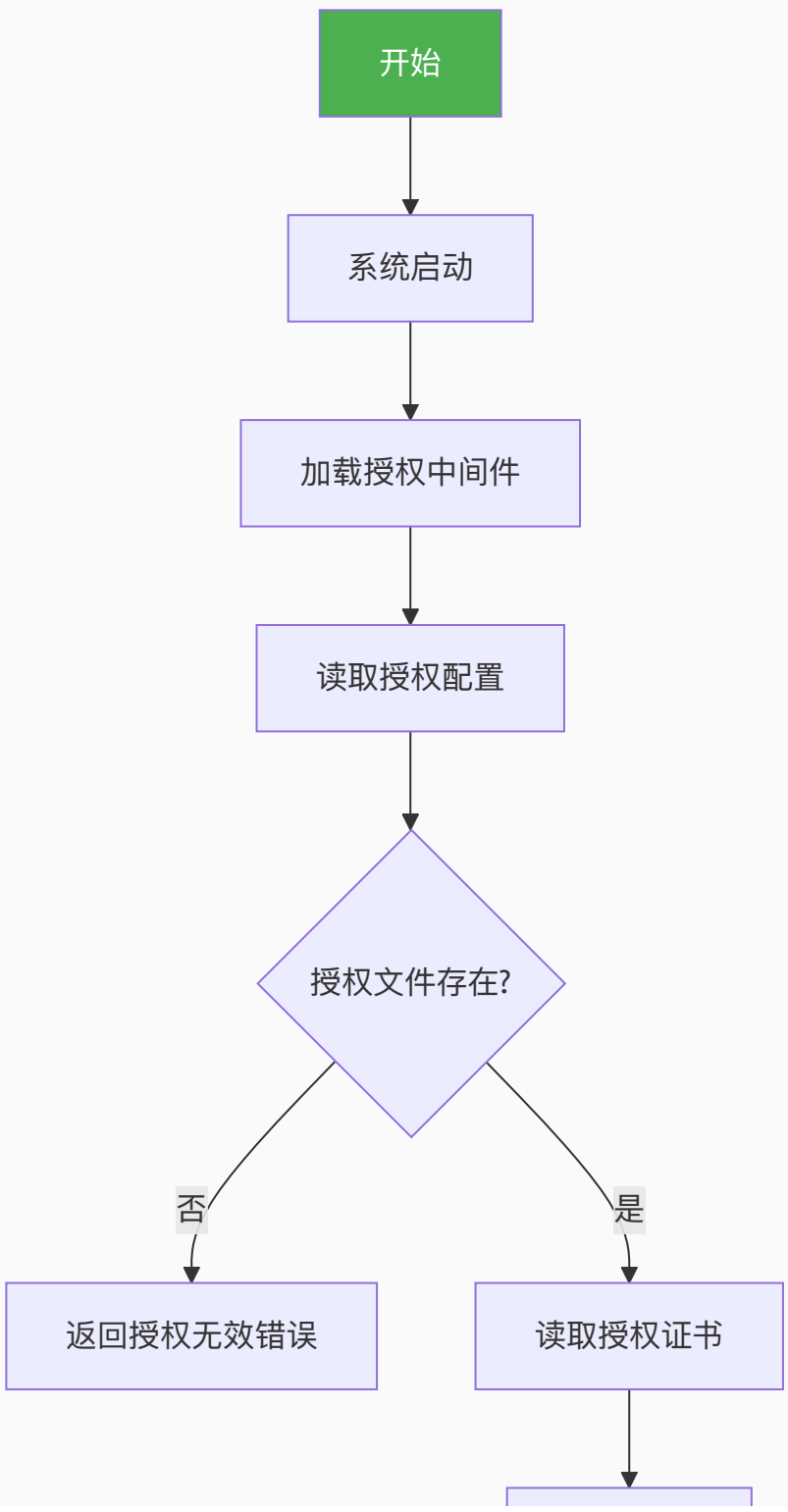
3.13.3 业务授权模块类图

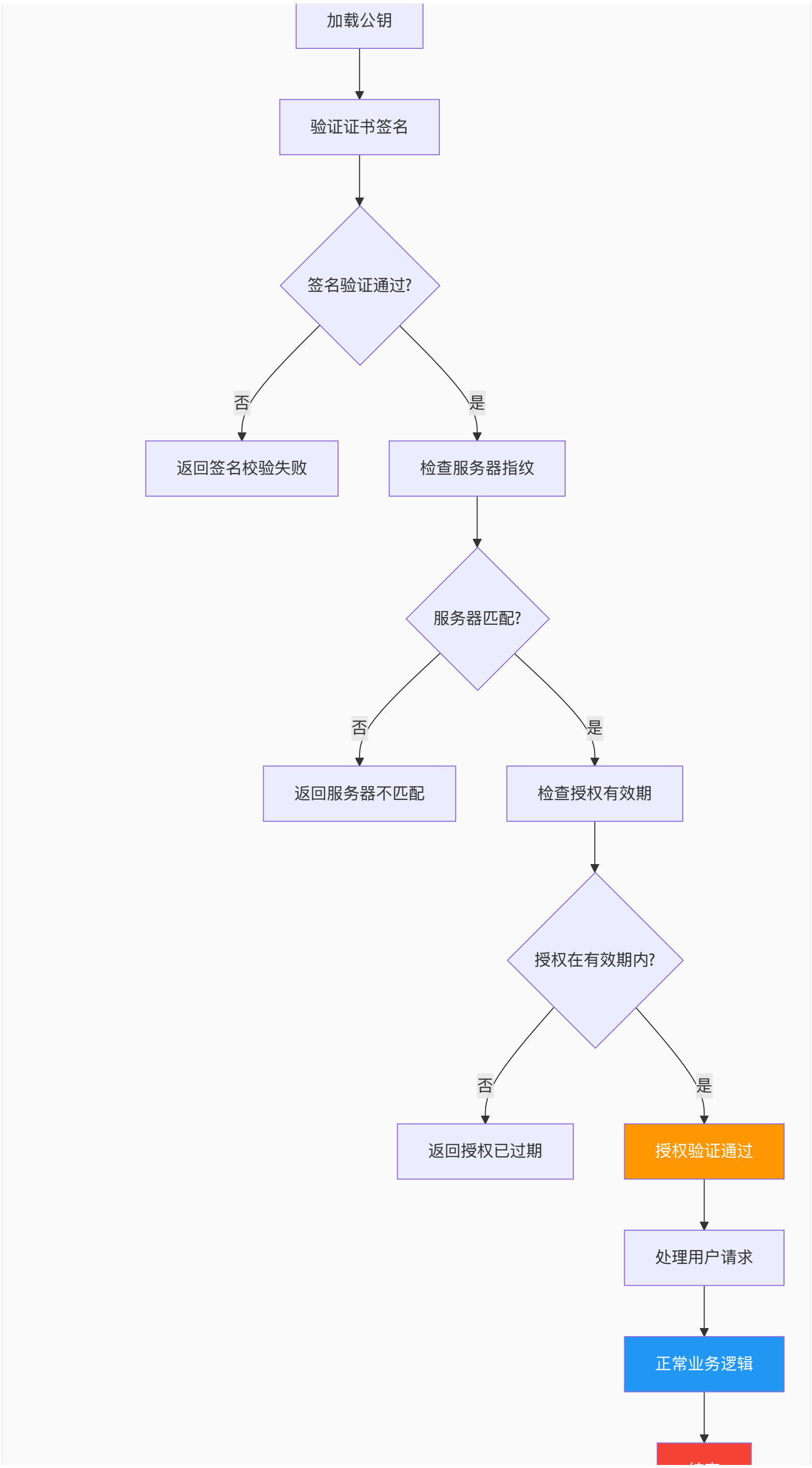


3.13.4 业务授权模块内部交互拓扑图



3.13.5 业务授权模块业务流程图




加载公钥
验证证书签名
签名验证通过?
否
是
返回签名校验失败
检查服务器指纹
服务器匹配?
否
是
返回服务器不匹配
检查授权有效期
授权在有效期内?
否
是
返回授权已过期
授权验证通过
处理用户请求
正常业务逻辑

结束

3.13.6 业务授权模块详细设计说明

3.13.6.1 设计目标

业务授权模块旨在为肥猫商城系统提供安全的商业授权保护机制，确保系统只能在获得有效授权的情况下运行。该模块采用数字签名技术，实现授权证书的安全签发和验证，防止未授权使用和盗版。

3.13.6.2 核心功能

1. 密钥管理

- 支持Ed25519非对称加密算法
- 自动生成公私钥对
- 私钥用于签发授权证书，公钥用于验证授权证书
- 私钥仅保存在许可方签发环境，不部署到客户服务器

2. 服务器指纹识别

- 基于服务器硬件特征生成唯一指纹
- 综合考虑机器ID、操作系统、CPU架构、主机名、MAC地址等信息
- 使用SHA256哈希算法生成稳定的指纹标识
- 授权证书与特定服务器绑定，防止证书迁移

3. 授权证书签发

- 支持自定义客户信息
- 可设置授权有效期（起止时间）
- 支持功能特性授权（可选功能模块）
- 使用私钥对证书内容进行数字签名
- 证书格式为JSON，包含payload和signature两部分

4. 授权证书验证

- 验证证书签名的有效性
- 检查服务器指纹匹配
- 验证授权时间范围（not_before到expires_at）
- 检查必要字段完整性
- 使用恒定时间比较防止时序攻击

5. 授权中间件

- 在Django中间件层进行授权拦截
- 每个请求都进行授权验证
- 授权失败时返回403错误
- 将授权信息注入到request对象中供业务使用

3.13.6.3 技术实现

加密算法选择

- 采用Ed25519椭圆曲线数字签名算法
- 具有高安全性和高性能特点
- 签名长度固定，便于处理
- 抗量子计算攻击能力强

证书结构

```
{
  "payload": {
    "license_id": "uuid",
    "customer": "客户名称",
    "server_fingerprint": "服务器指纹哈希",
    "not_before": "2026-01-01T00:00:00+00:00",
    "expires_at": "2027-12-31T23:59:59+00:00",
    "features": ["feature1", "feature2"]
  },
  "signature": "base64编码的数字签名"
}
```

服务器指纹生成算法

```
def get_server_fingerprint():
    machine_id = 读取系统机器ID
    parts = [
        machine_id,
        platform.system(),      # 操作系统
        platform.machine(),     # CPU架构
        socket.gethostname(),   # 主机名
        str(uuid.getnode())     # MAC地址
    ]
    return hashlib.sha256('|'.join(parts).encode('utf-8')).hexdigest()
```

签名验证流程

1. 读取授权证书JSON文件
2. 提取payload和signature字段
3. 使用公钥验证signature对payload的签名
4. 检查payload的必要字段
5. 验证服务器指纹匹配
6. 检查时间有效性

3.13.6.4 安全特性

1. 防篡改

- 证书内容经过数字签名，任何修改都会导致验证失败
- 使用canonical JSON格式确保序列化一致性

2. 防重放

- 证书与特定服务器指纹绑定
- 服务器指纹基于硬件特征，难以伪造

3. 防时序攻击

- 使用hmac.compare_digest进行恒定时间比较
- 防止通过时间差推断比较结果

4. 密钥安全

- 私钥仅存在于签发环境
- 私钥文件权限设置为600
- 公钥可安全部署到客户服务器

5. 时间控制

- 支持设置授权生效时间和过期时间
- 防止提前使用证书

- 自动阻止过期证书

3.13.6.5 文件结构

```
hot_mall/
├── license.py                    # 授权核心逻辑
├── license_middleware.py         # Django中间件
└── apps/
    └── users/
        └── management/
            └── commands/
                └── license.py    # Django管理命令
```

3.13.6.6 配置参数

在settings文件中配置授权相关参数：

```
# 商业授权配置
LICENSE_FILE = os.getenv('LICENSE_FILE', 'path/to/license.json')
LICENSE_PUBLIC_KEY = os.getenv('LICENSE_PUBLIC_KEY', '')
LICENSE_PUBLIC_KEY_FILE = os.getenv('LICENSE_PUBLIC_KEY_FILE', 'path/to/public.pem')
```

3.13.6.7 中间件配置

在Django中间件配置中添加授权中间件：

```
MIDDLEWARE = [
    'django.middleware.security.SecurityMiddleware',
    'hot_mall.license_middleware.LicenseMiddleware',  # 授权中间件
    'django.contrib.sessions.middleware.SessionMiddleware',
    # ... 其他中间件
]
```

3.13.7 业务授权模块使用说明

3.13.7.1 密钥对生成

使用Django管理命令生成密钥对：

```
python manage.py license generate-keys \
    --private-key license-private-key.pem \
    --public-key license-public-key.pem
```

注意事项：

- 私钥文件权限自动设置为600
- 请妥善保管私钥，不要泄露
- 私钥仅用于签发授权证书，不部署到客户服务器

3.13.7.2 获取服务器指纹

获取客户服务器的指纹信息：

```
python manage.py license fingerprint
```

输出示例：

```
a1b2c3d4e5f6...（64位十六进制字符串）
```

3.13.7.3 签发授权证书

在许可方环境使用私钥签发授权证书：

```
python manage.py license issue \
    --private-key license-private-key.pem \
    --output license.json \
    --customer "客户公司名称" \
    --server-fingerprint "客户服务器指纹" \
    --expires-at "2027-12-31T23:59:59+00:00" \
    --license-id "custom-license-id" \
    --feature feature1 \
    --feature feature2
```

参数说明：

- `--private-key`：签发私钥文件路径
- `--output`：输出证书文件路径
- `--customer`：客户名称
- `--server-fingerprint`：客户服务器指纹（不填则使用当前机器）
- `--expires-at`：授权过期时间（UTC时区）
- `--license-id`：自定义授权ID（可选）
- `--feature`：授权的功能特性（可多次指定）

3.13.7.4 部署授权证书

将生成的以下文件部署到客户服务器：

1. **license.json**: 授权证书文件
2. **license-public-key.pem**: 验证公钥文件

配置环境变量或修改settings文件：

```
export LICENSE_FILE=/path/to/license.json
export LICENSE_PUBLIC_KEY_FILE=/path/to/license-public-key.pem
```

3.13.7.5 授权验证

系统启动时会自动验证授权：

1. 中间件拦截每个HTTP请求
2. 读取并验证授权证书
3. 检查服务器指纹匹配
4. 验证授权时间有效性
5. 验证通过则正常处理请求
6. 验证失败则返回403错误

3.13.7.6 授权错误处理

常见授权错误及处理方法：

错误1: 授权证书不存在或格式无效

- 检查LICENSE_FILE路径是否正确
- 确认证书文件格式为有效的JSON

错误2: 授权证书签名校验失败

- 确认使用正确的公钥文件
- 检查证书是否被篡改
- 确认私钥和公钥匹配

错误3: 授权证书与当前服务器不匹配

- 检查服务器指纹是否正确
- 确认证书是否在正确的服务器上部署
- 重新获取服务器指纹并签发新证书

错误4: 授权证书已过期

- 检查系统时间是否正确
- 联系许可方续期授权
- 使用新的过期时间签发新证书

错误5: 授权证书尚未生效

- 检查not_before时间设置
- 确认系统时间是否正确
- 调整证书生效时间

3.13.8 业务授权模块API接口

业务授权模块主要通过中间件自动处理，不提供直接的HTTP接口。所有接口都通过Django管理命令调用。

3.13.8.1 管理命令接口

获取服务器指纹

```
GET /management/命令行接口
python manage.py license fingerprint
```

生成密钥对

```
python manage.py license generate-keys [options]
```

签发授权证书

```
python manage.py license issue [options]
```

3.13.9 业务授权模块功能特点

1. **高安全性**: 采用Ed25519数字签名算法，确保授权证书不可伪造
2. **服务器绑定**: 基于硬件特征的服务器指纹，防止证书迁移
3. **时间控制**: 精确控制授权生效和过期时间
4. **功能授权**: 支持按功能模块进行授权控制
5. **自动化验证**: 中间件自动拦截验证，无需业务代码干预
6. **易于部署**: 通过环境变量配置，支持灵活部署
7. **错误友好**: 提供清晰的错误信息，便于问题排查
8. **性能优化**: 验证过程高效，对系统性能影响小

3.13.10 业务授权模块监控与维护

3.13.10.1 授权状态监控

建议定期检查授权状态：

1. **日志监控**: 关注授权相关的错误日志

2. **时间提醒**: 在授权过期前30天设置提醒
3. **服务器变更**: 服务器硬件变更时需要重新获取指纹

3.13.10.2 授权续期流程

1. 获取新的服务器指纹（如服务器变更）
2. 联系许可方提供续期信息
3. 许可方使用私钥签发新的授权证书
4. 部署新的授权证书到服务器
5. 重启应用服务使新授权生效

3.13.10.3 应急处理

授权过期应急处理:

- 临时授权：联系许可方获取临时授权证书
- 系统时间：检查系统时间是否正确
- 回滚处理：如证书部署错误，回滚到之前的版本

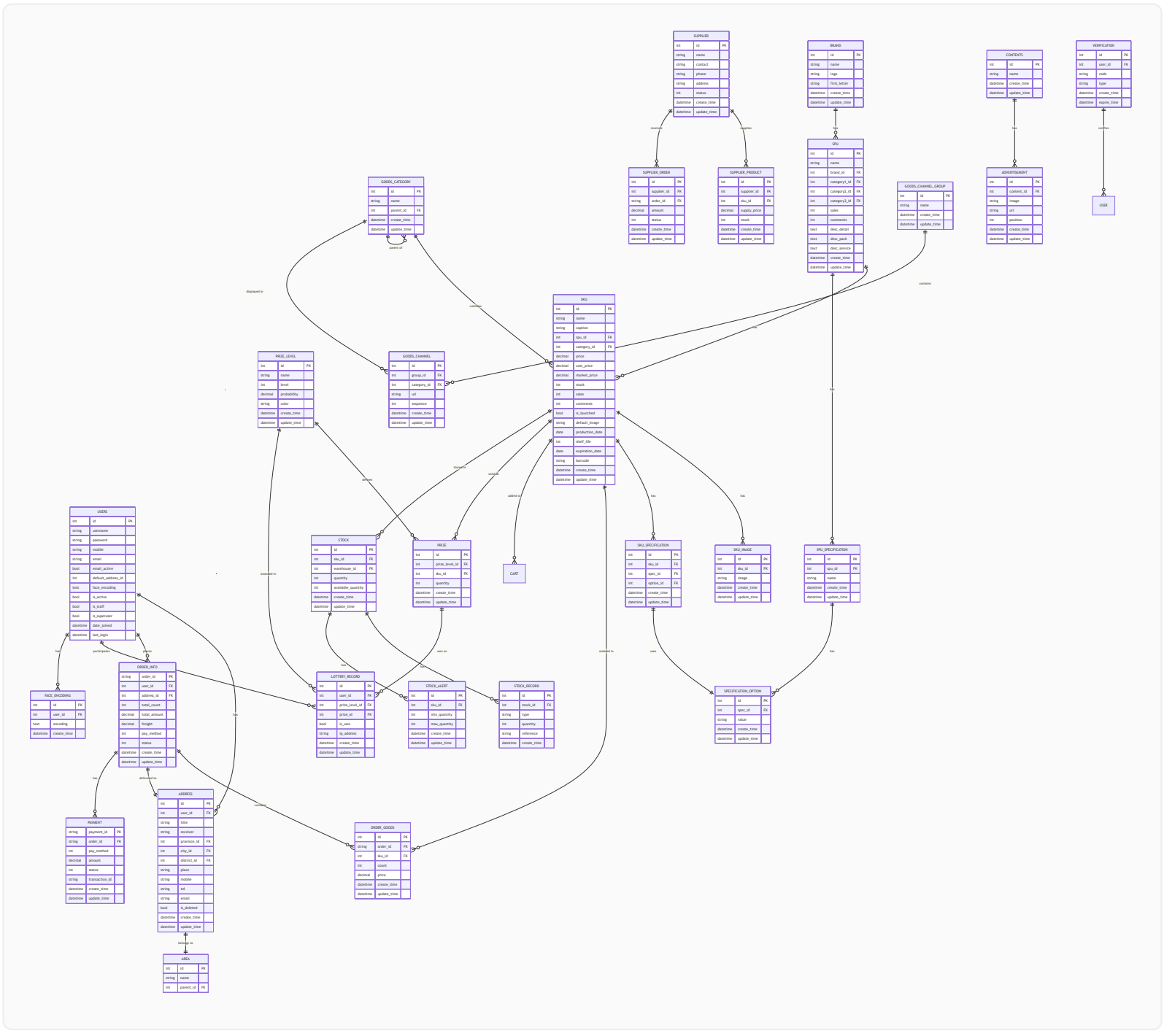
服务器变更处理:

- 提前规划：在服务器变更前获取新指纹
- 测试验证：在测试环境验证新证书
- 平滑迁移：在低峰期进行证书部署

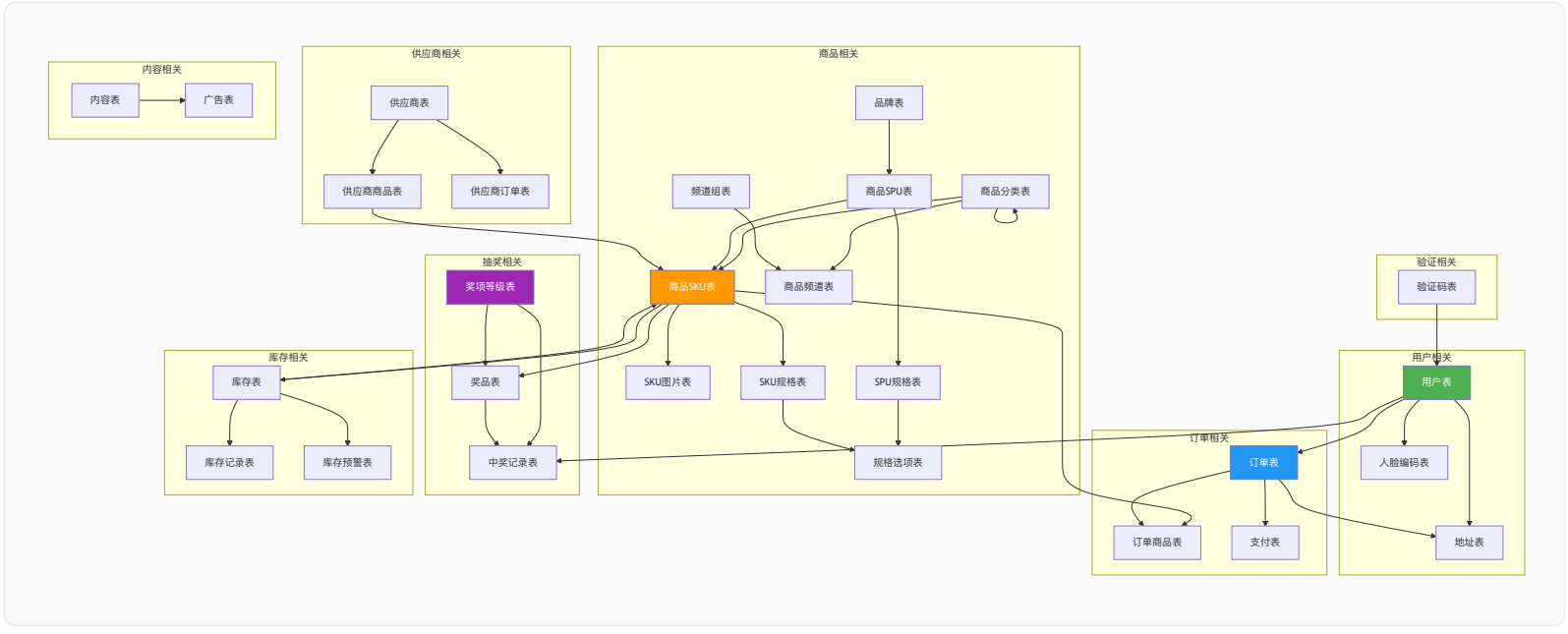
---

4. 数据库设计

4.1 数据库ER图



4.2 数据表关系图



4.3 核心数据表结构

4.3.1 用户相关表

tb_users (用户表)

字段名	类型	长度	允许空	默认值	说明
id	INT	11	NO	AUTO_INCREMENT	主键ID
username	VARCHAR	150	NO	-	用户名
password	VARCHAR	128	NO	-	密码
mobile	VARCHAR	11	NO	-	手机号
email	VARCHAR	254	YES	NULL	邮箱
email_active	BOOLEAN	-	NO	False	邮箱验证状态
default_address_id	INT	11	YES	NULL	默认地址ID
face_encoding	TEXT	-	YES	NULL	人脸特征编码
is_active	BOOLEAN	-	NO	True	是否激活
is_staff	BOOLEAN	-	NO	False	是否员工
is_superuser	BOOLEAN	-	NO	False	是否超级管理员
date_joined	DATETIME	-	NO	CURRENT_TIMESTAMP	注册时间
last_login	DATETIME	-	YES	NULL	最后登录时间

索引设计

- PRIMARY KEY: `id`
- UNIQUE KEY: `username`
- UNIQUE KEY: `mobile`
- INDEX: `email`

tb_address (用户地址表)

字段名	类型	长度	允许空	默认值	说明
id	INT	11	NO	AUTO_INCREMENT	主键ID
user_id	INT	11	NO	-	用户ID
title	VARCHAR	20	NO	-	地址名称
receiver	VARCHAR	20	NO	-	收货人
province_id	INT	11	NO	-	省ID
city_id	INT	11	NO	-	市ID
district_id	INT	11	NO	-	区ID
place	VARCHAR	50	NO	-	详细地址
mobile	VARCHAR	11	NO	-	手机号
tel	VARCHAR	20	YES	-	固定电话
email	VARCHAR	30	YES	-	电子邮箱
is_deleted	BOOLEAN	-	NO	False	逻辑删除
create_time	DATETIME	-	NO	CURRENT_TIMESTAMP	创建时间
update_time	DATETIME	-	NO	CURRENT_TIMESTAMP	更新时间

索引设计

- PRIMARY KEY: `id`
- INDEX: `user_id`
- INDEX: `province_id`, `city_id`, `district_id`

4.3.2 商品相关表

tb_goods_category (商品分类表)

字段名	类型	长度	允许空	默认值	说明
id	INT	11	NO	AUTO_INCREMENT	主键ID
name	VARCHAR	10	NO	-	分类名称
parent_id	INT	11	YES	NULL	父分类ID
create_time	DATETIME	-	NO	CURRENT_TIMESTAMP	创建时间
update_time	DATETIME	-	NO	CURRENT_TIMESTAMP	更新时间

索引设计

- PRIMARY KEY: `id`
- INDEX: `parent_id`
- INDEX: `name`

tb_sku (商品SKU表)

字段名	类型	长度	允许空	默认值	说明
id	INT	11	NO	AUTO_INCREMENT	主键ID
name	VARCHAR	50	NO	-	商品名称
caption	VARCHAR	100	NO	-	副标题
spu_id	INT	11	NO	-	SPU ID
category_id	INT	11	NO	-	分类ID
price	DECIMAL	10,2	NO	-	单价
cost_price	DECIMAL	10,2	NO	-	进价
market_price	DECIMAL	10,2	NO	-	市场价
stock	INT	11	NO	0	库存
sales	INT	11	NO	0	销量
comments	INT	11	NO	0	评价数
is_launched	BOOLEAN	-	NO	True	是否上架
default_image	VARCHAR	200	YES	NULL	默认图片
production_date	DATE	-	YES	NULL	生产日期
shelf_life	INT	11	NO	0	保质期(天)
expiration_date	DATE	-	YES	NULL	过期日期
barcode	VARCHAR	30	YES	NULL	商品条码
create_time	DATETIME	-	NO	CURRENT_TIMESTAMP	创建时间
update_time	DATETIME	-	NO	CURRENT_TIMESTAMP	更新时间

索引设计

- PRIMARY KEY: `id`
- INDEX: `spu_id`
- INDEX: `category_id`
- INDEX: `barcode`
- INDEX: `is_launched`

4.3.3 订单相关表

tb_order_info (订单信息表)

字段名	类型	长度	允许空	默认值	说明
order_id	VARCHAR	64	NO	-	订单号
user_id	INT	11	NO	-	用户ID
address_id	INT	11	NO	-	收货地址ID
total_count	INT	11	NO	1	商品总数
total_amount	DECIMAL	10,2	NO	-	商品总金额
freight	DECIMAL	10,2	NO	-	运费
pay_method	SMALLINT	6	NO	1	支付方式
status	SMALLINT	6	NO	1	订单状态
create_time	DATETIME	-	NO	CURRENT_TIMESTAMP	创建时间
update_time	DATETIME	-	NO	CURRENT_TIMESTAMP	更新时间

索引设计

- PRIMARY KEY: `order_id`
- INDEX: `user_id`
- INDEX: `status`
- INDEX: `create_time`

tb_order_goods (订单商品表)

字段名	类型	长度	允许空	默认值	说明
id	INT	11	NO	AUTO_INCREMENT	主键ID
order_id	VARCHAR	64	NO	-	订单号
sku_id	INT	11	NO	-	商品SKU ID
count	INT	11	NO	-	商品数量
price	DECIMAL	10,2	NO	-	商品单价
create_time	DATETIME	-	NO	CURRENT_TIMESTAMP	创建时间
update_time	DATETIME	-	NO	CURRENT_TIMESTAMP	更新时间

索引设计

- PRIMARY KEY: `id`
- INDEX: `order_id`
- INDEX: `sku_id`

4.3.4 抽奖相关表

tb_prize_level (奖项表)

字段名	类型	长度	允许空	默认值	说明
id	INT	11	NO	AUTO_INCREMENT	主键ID
name	VARCHAR	50	NO	-	等级名称
level	INT	11	NO	-	等级序号
probability	DECIMAL	5,2	NO	-	中奖概率(%)
color	VARCHAR	20	NO	#FF6B6B	转盘颜色
create_time	DATETIME	-	NO	CURRENT_TIMESTAMP	创建时间
update_time	DATETIME	-	NO	CURRENT_TIMESTAMP	更新时间

索引设计

- PRIMARY KEY: `id`
- UNIQUE KEY: `level`

tb_prize (奖品表)

字段名	类型	长度	允许空	默认值	说明
id	INT	11	NO	AUTO_INCREMENT	主键ID
prize_level_id	INT	11	NO	-	中奖等级ID
sku_id	INT	11	NO	-	商品SKU ID
quantity	INT	11	NO	1	奖品数量
create_time	DATETIME	-	NO	CURRENT_TIMESTAMP	创建时间
update_time	DATETIME	-	NO	CURRENT_TIMESTAMP	更新时间

索引设计

- PRIMARY KEY: `id`
- INDEX: `prize_level_id`
- INDEX: `sku_id`

tb_lottery_record (中奖记录表)

字段名	类型	长度	允许空	默认值	说明
id	INT	11	NO	AUTO_INCREMENT	主键ID
user_id	INT	11	NO	-	用户ID
prize_level_id	INT	11	NO	-	中奖等级ID
prize_id	INT	11	YES	NULL	奖品ID
is_won	BOOLEAN	-	NO	False	是否中奖
ip_address	VARCHAR	15	YES	NULL	IP地址
create_time	DATETIME	-	NO	CURRENT_TIMESTAMP	创建时间
update_time	DATETIME	-	NO	CURRENT_TIMESTAMP	更新时间

索引设计

- PRIMARY KEY: `id`

- INDEX: `user_id`
- INDEX: `prize_level_id`
- INDEX: `create_time`

4.4 数据库优化策略

4.4.1 索引优化

- 为常用查询字段建立索引
- 使用复合索引优化多条件查询
- 定期分析索引使用情况，删除无用索引

4.4.2 查询优化

- 使用 `select_related` 和 `prefetch_related` 减少查询次数
- 避免N+1查询问题
- 使用 `only()` 和 `defer()` 只查询需要的字段

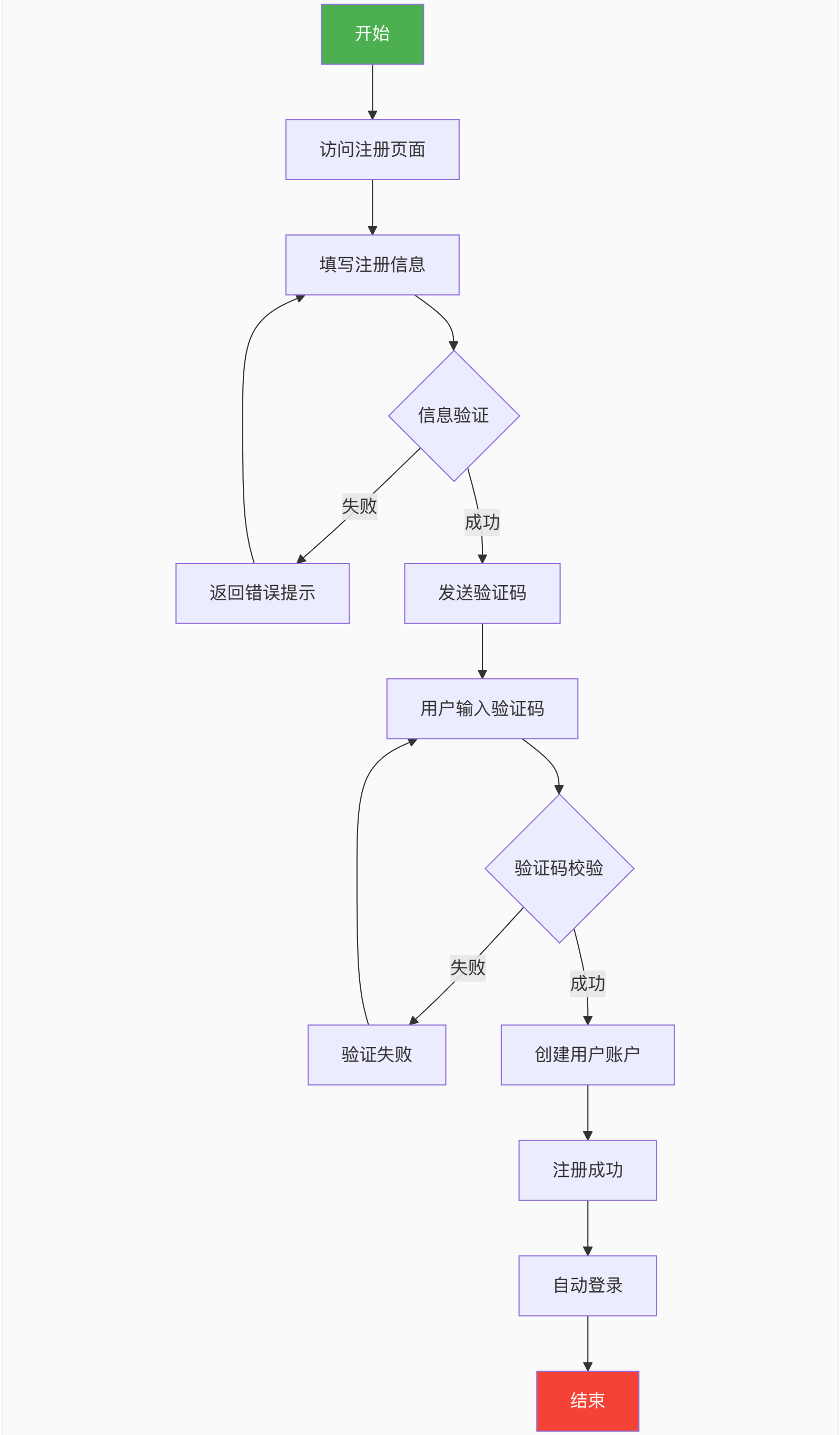
4.4.3 分表分库策略

- 订单表按时间分表
- 日志表按用户ID分表
- 大表考虑垂直拆分

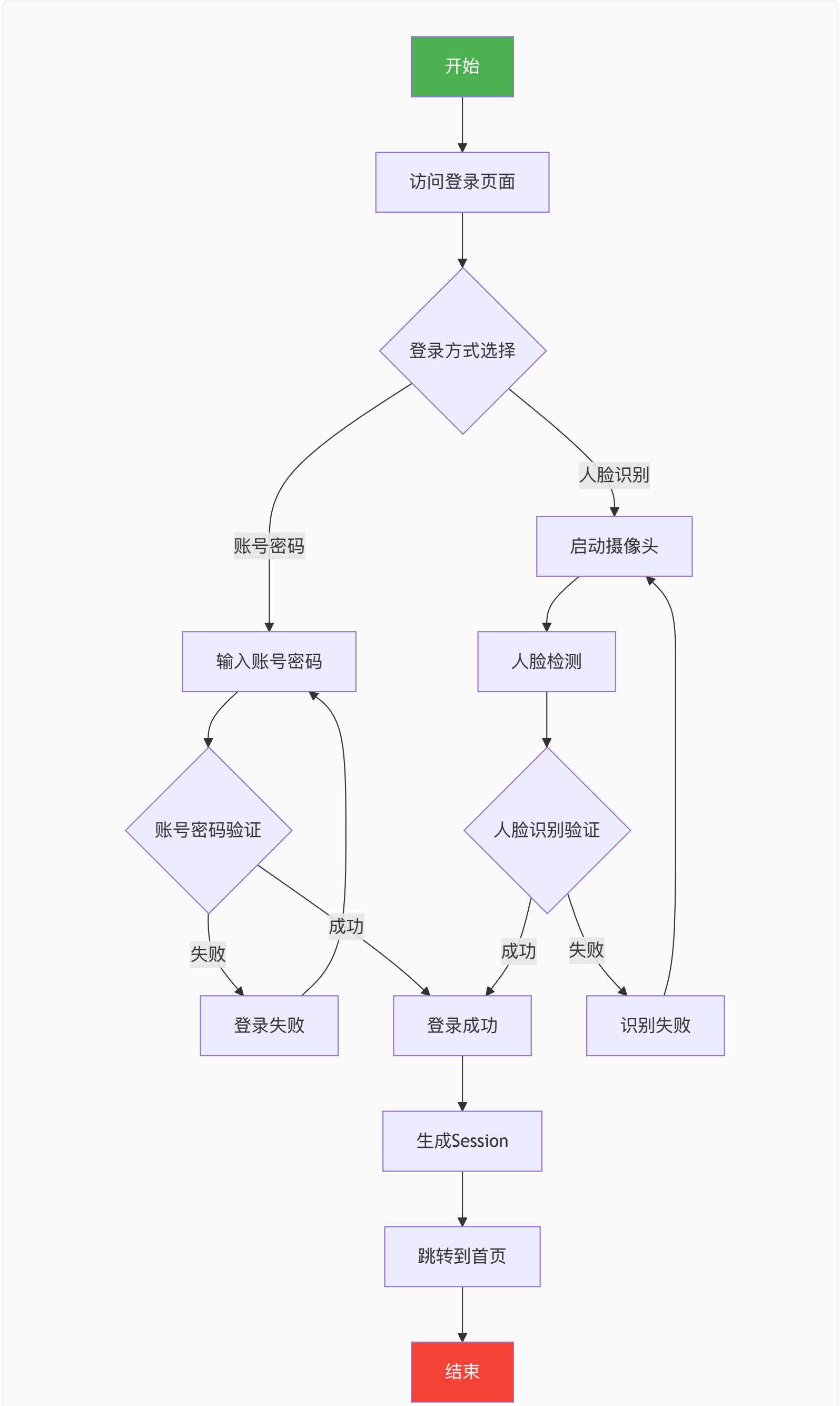
---

5. 业务流程设计

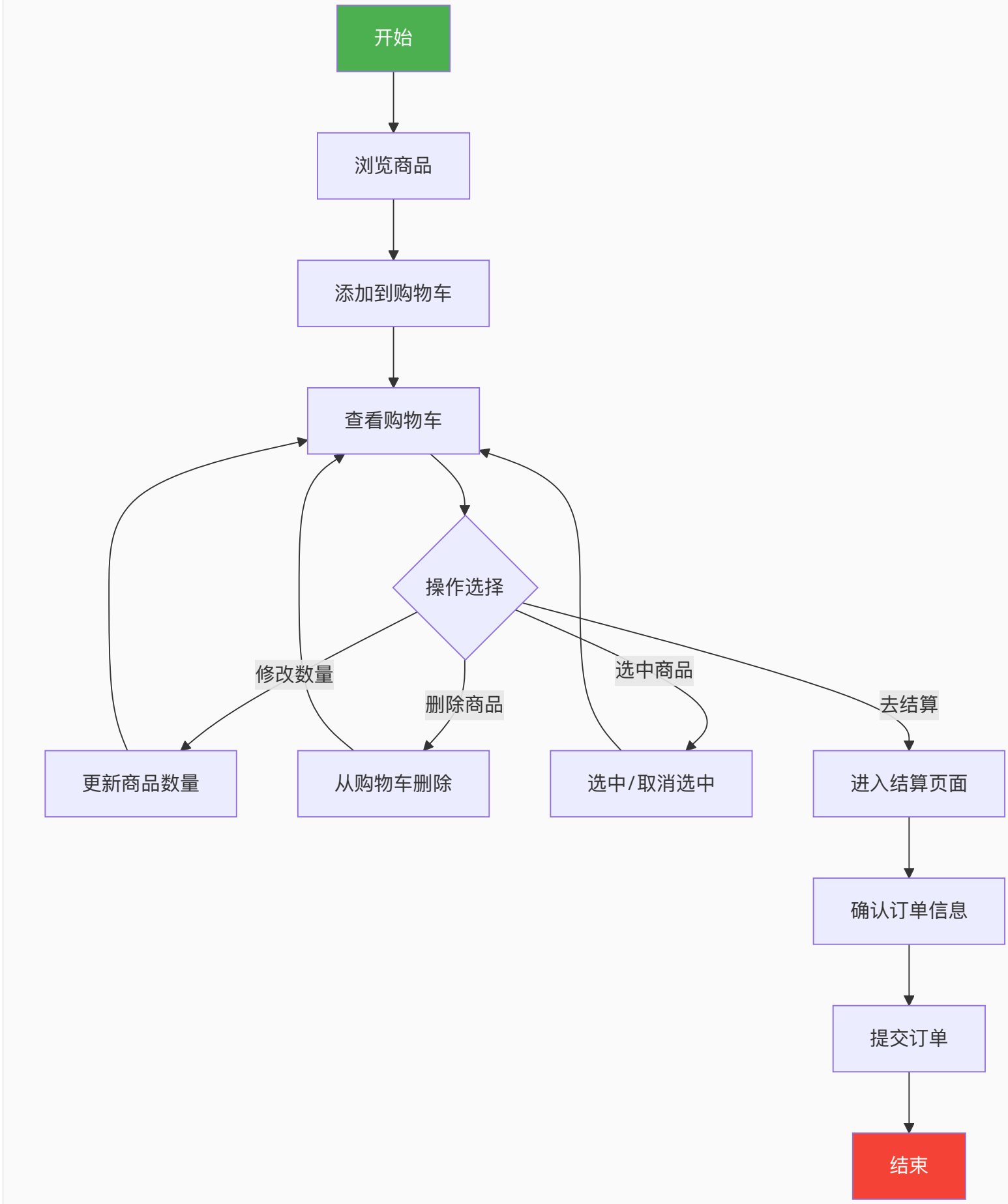
5.1 用户注册流程


开始
访问注册页面
填写注册信息
信息验证
失败
成功
返回错误提示
发送验证码
用户输入验证码
验证码校验
失败
成功
验证失败
创建用户账户
注册成功
自动登录
结束

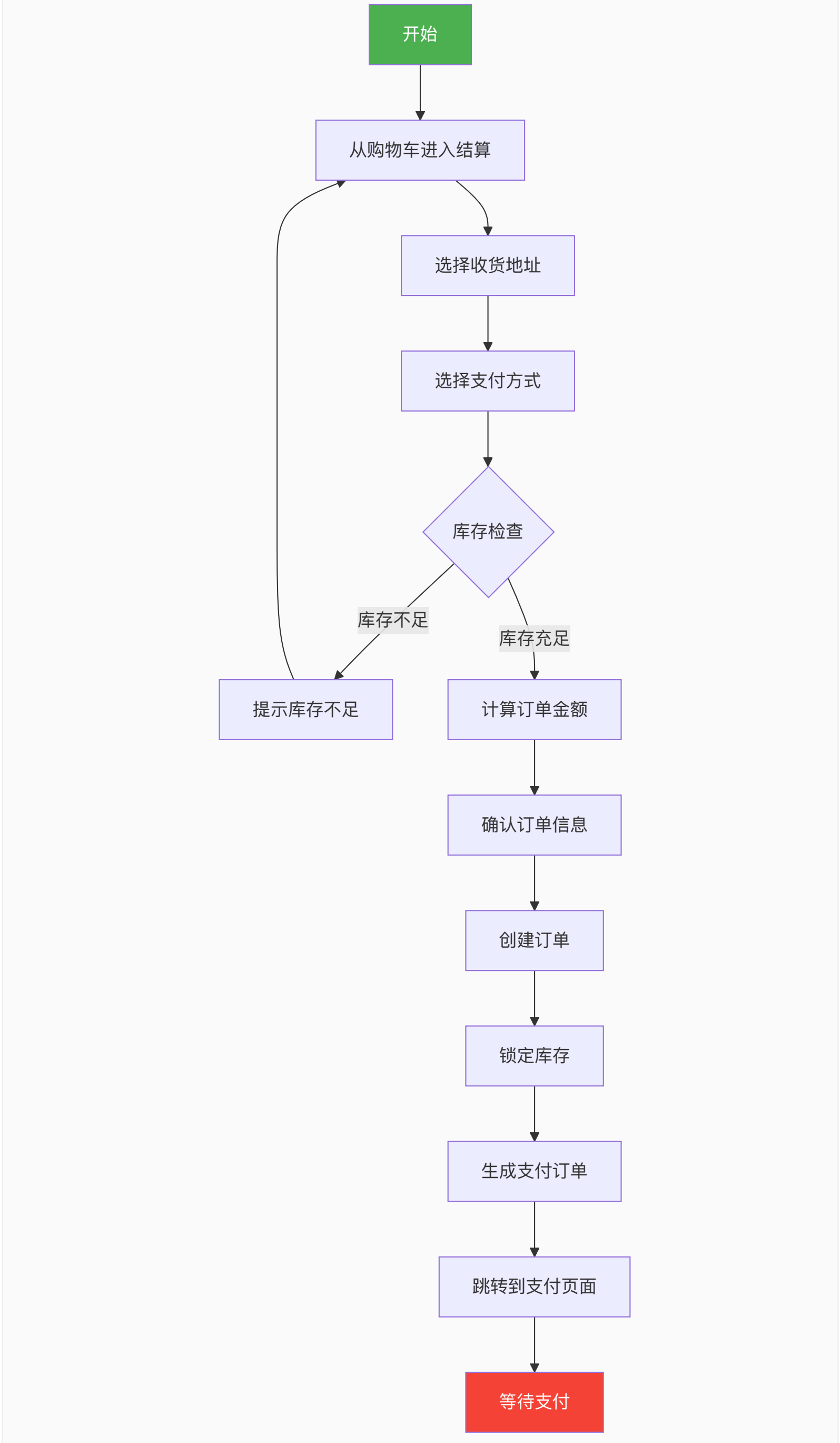
5.2 用户登录流程


开始
访问登录页面
登录方式选择
账号密码
输入账号密码
账号密码验证
失败
成功
登录失败
人脸识别
启动摄像头
人脸检测
人脸识别验证
成功
失败
识别失败
登录成功
生成Session
跳转到首页
结束

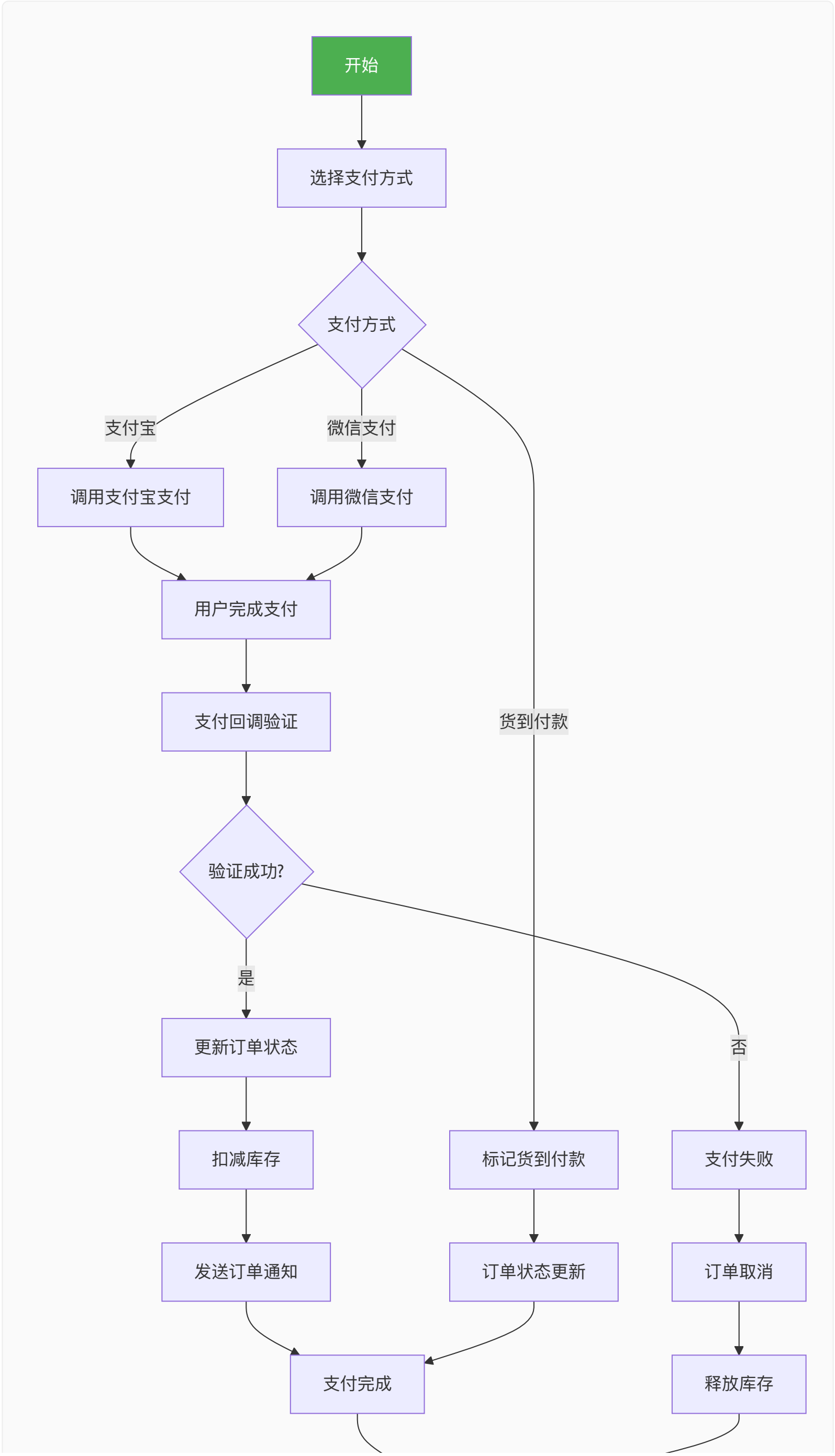
5.3 购物车流程



5.4 下单流程


开始
从购物车进入结算
选择收货地址
选择支付方式
库存检查
库存不足
库存充足
提示库存不足
计算订单金额
确认订单信息
创建订单
锁定库存
生成支付订单
跳转到支付页面
等待支付

5.5 支付流程

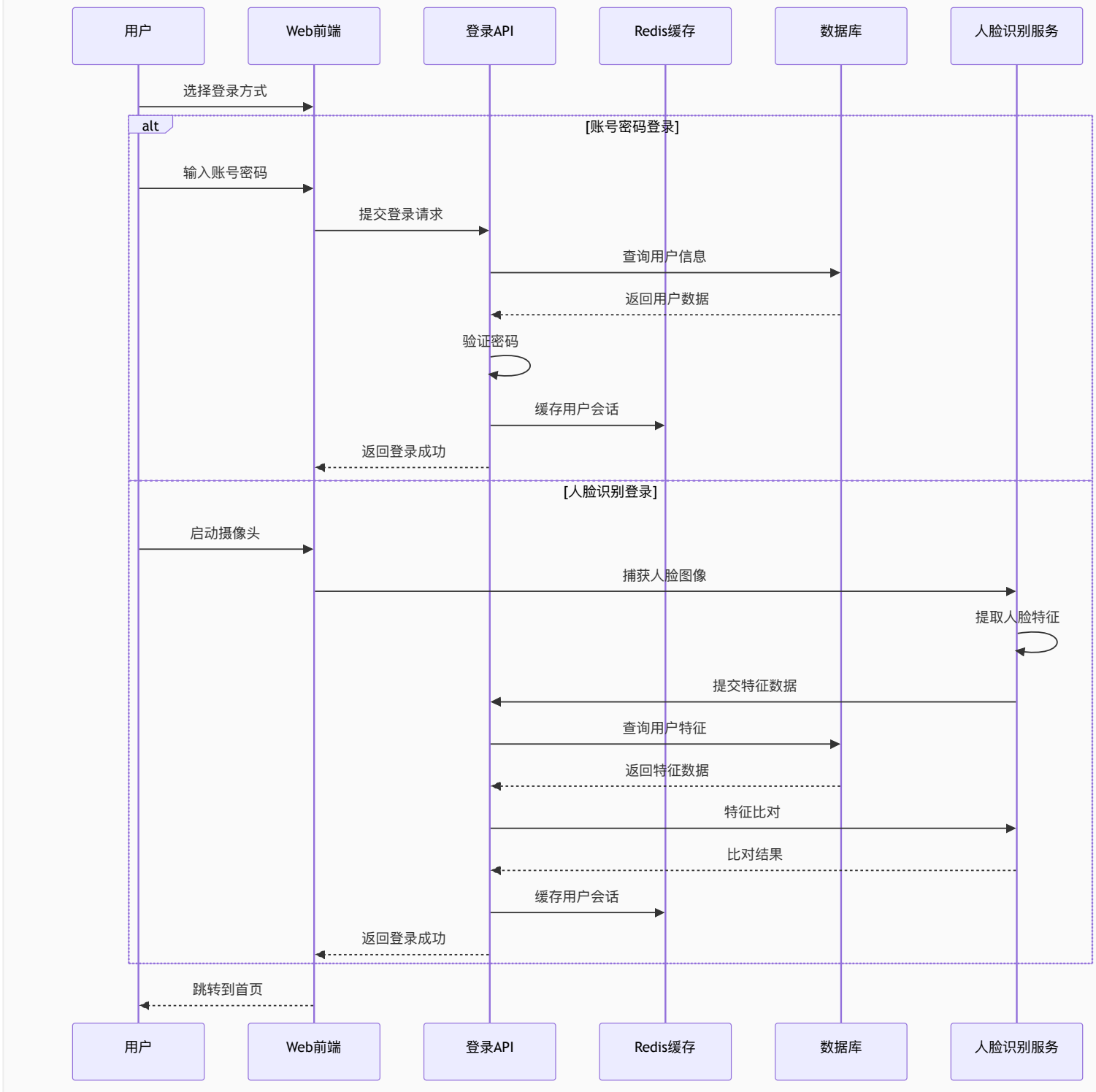

开始
选择支付方式
支付方式
支付宝
微信支付
货到付款
调用支付宝支付
调用微信支付
用户完成支付
支付回调验证
验证成功?
是
否
更新订单状态
扣减库存
发送订单通知
标记货到付款
订单状态更新
支付失败
订单取消
释放库存
支付完成



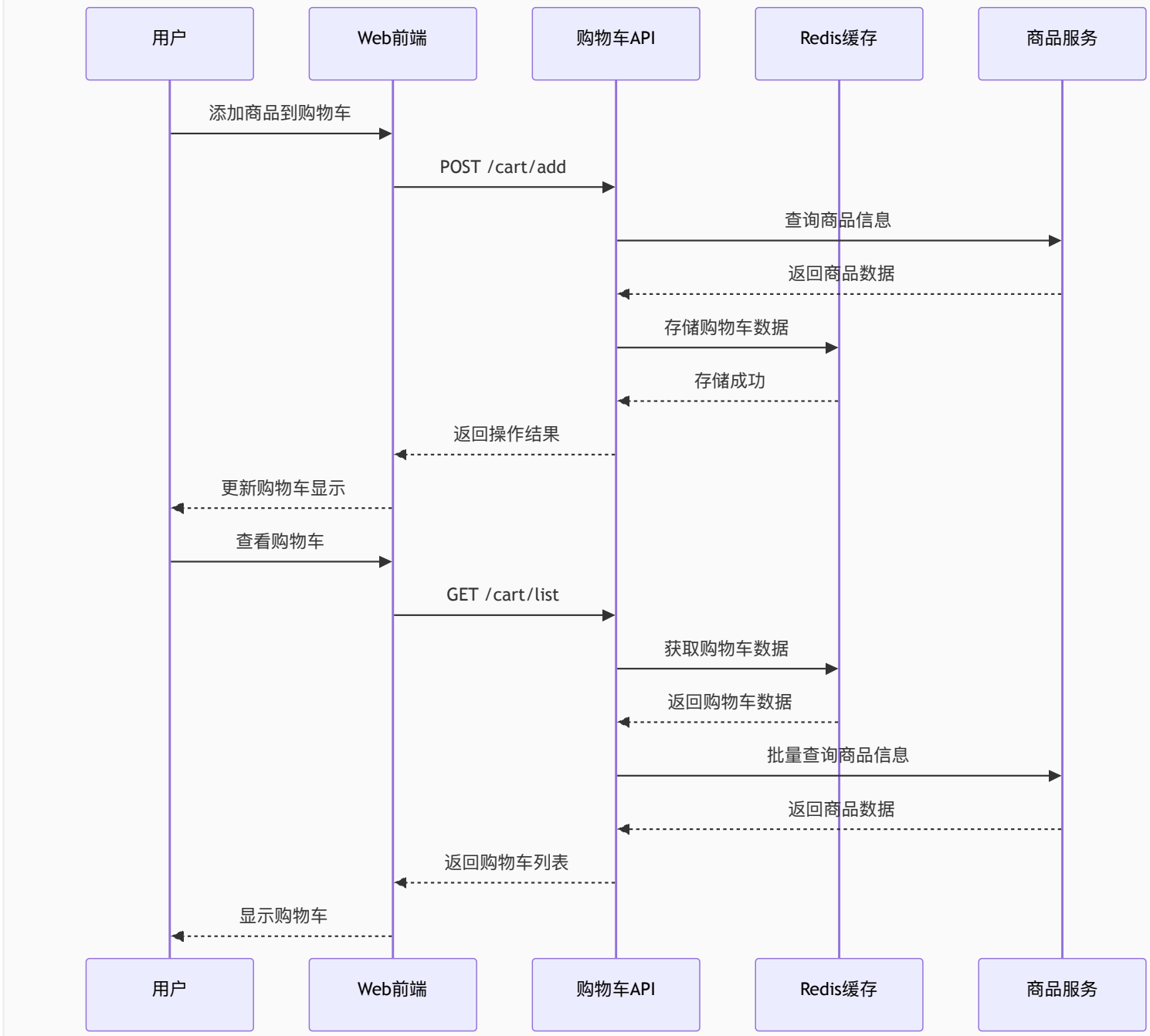
6. 时序图设计

6.1 用户注册时序图

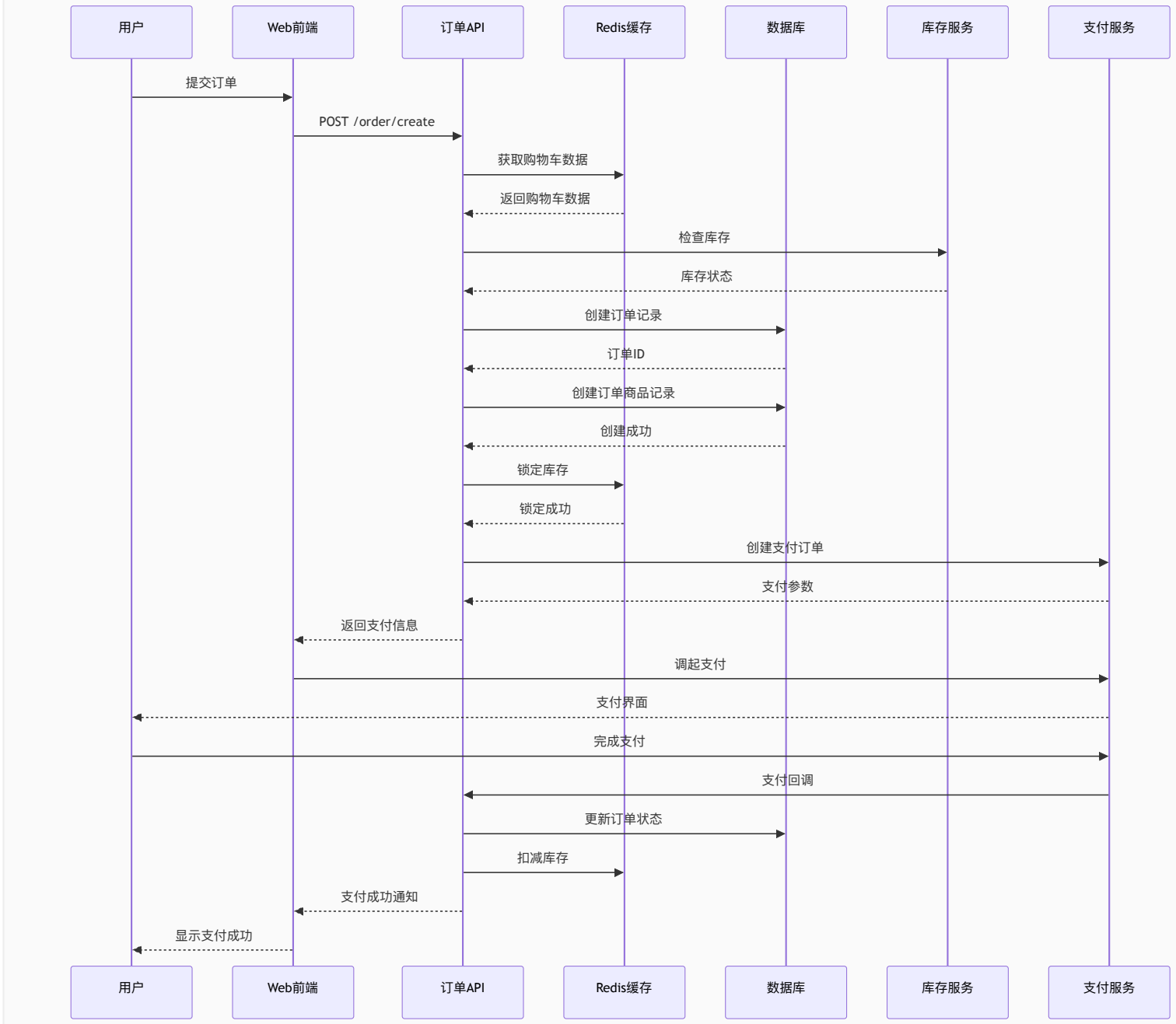
6.2 用户登录时序图


用户
Web前端
登录API
Redis缓存
数据库
人脸识别服务
选择登录方式
alt
[账号密码登录]
输入账号密码
提交登录请求
查询用户信息
返回用户数据
验证密码
缓存用户会话
返回登录成功
[人脸识别登录]
启动摄像头
捕获人脸图像
提取人脸特征
提交特征数据
查询用户特征
返回特征数据
特征比对
比对结果
缓存用户会话
返回登录成功
跳转到首页
用户
Web前端
登录API
Redis缓存
数据库
人脸识别服务

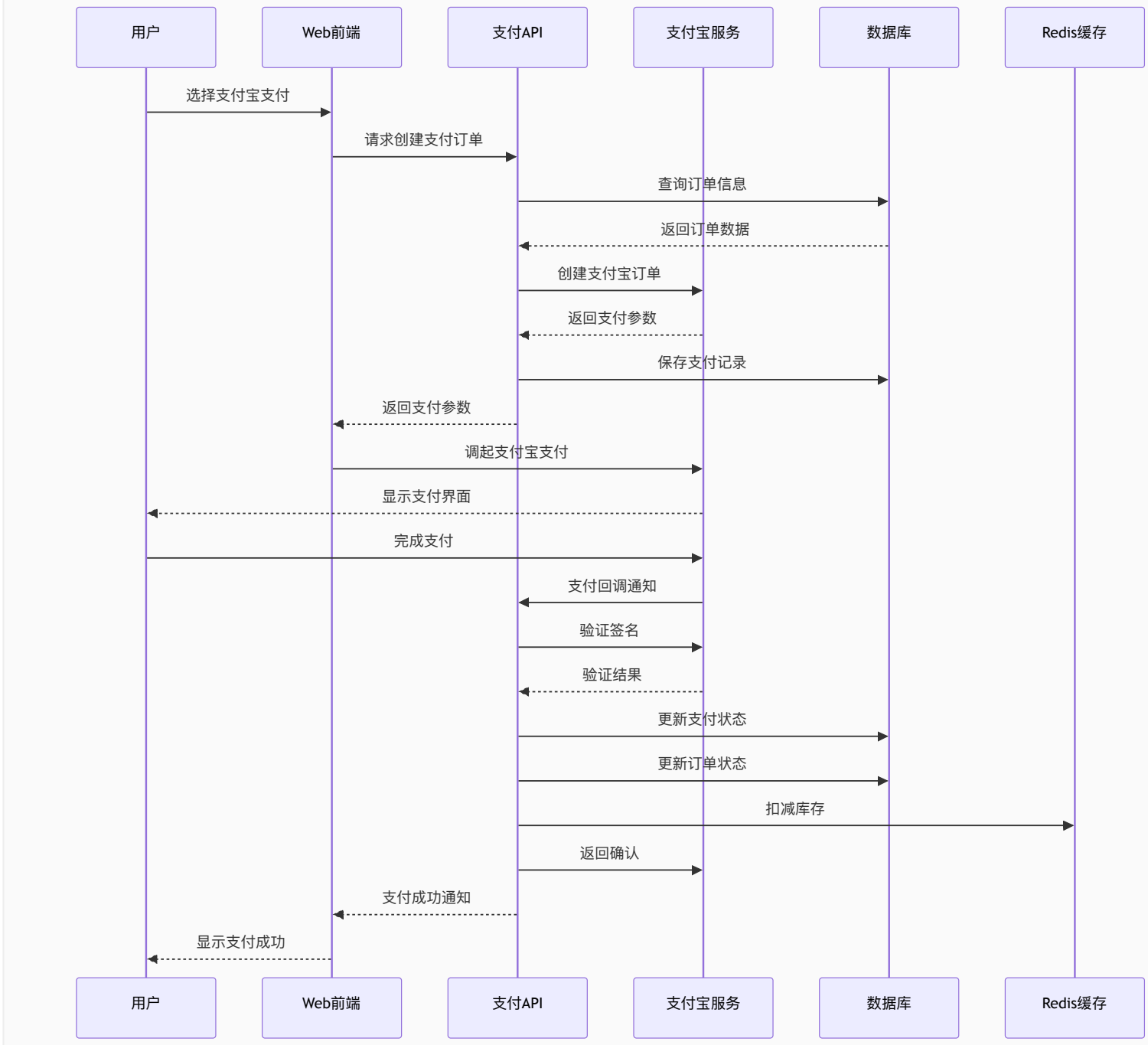
6.3 购物车操作时序图


用户
Web前端
购物车API
Redis缓存
商品服务
添加商品到购物车
POST /cart/add
查询商品信息
返回商品数据
存储购物车数据
存储成功
返回操作结果
更新购物车显示
查看购物车
GET /cart/list
获取购物车数据
返回购物车数据
批量查询商品信息
返回商品数据
返回购物车列表
显示购物车
用户
Web前端
购物车API
Redis缓存
商品服务

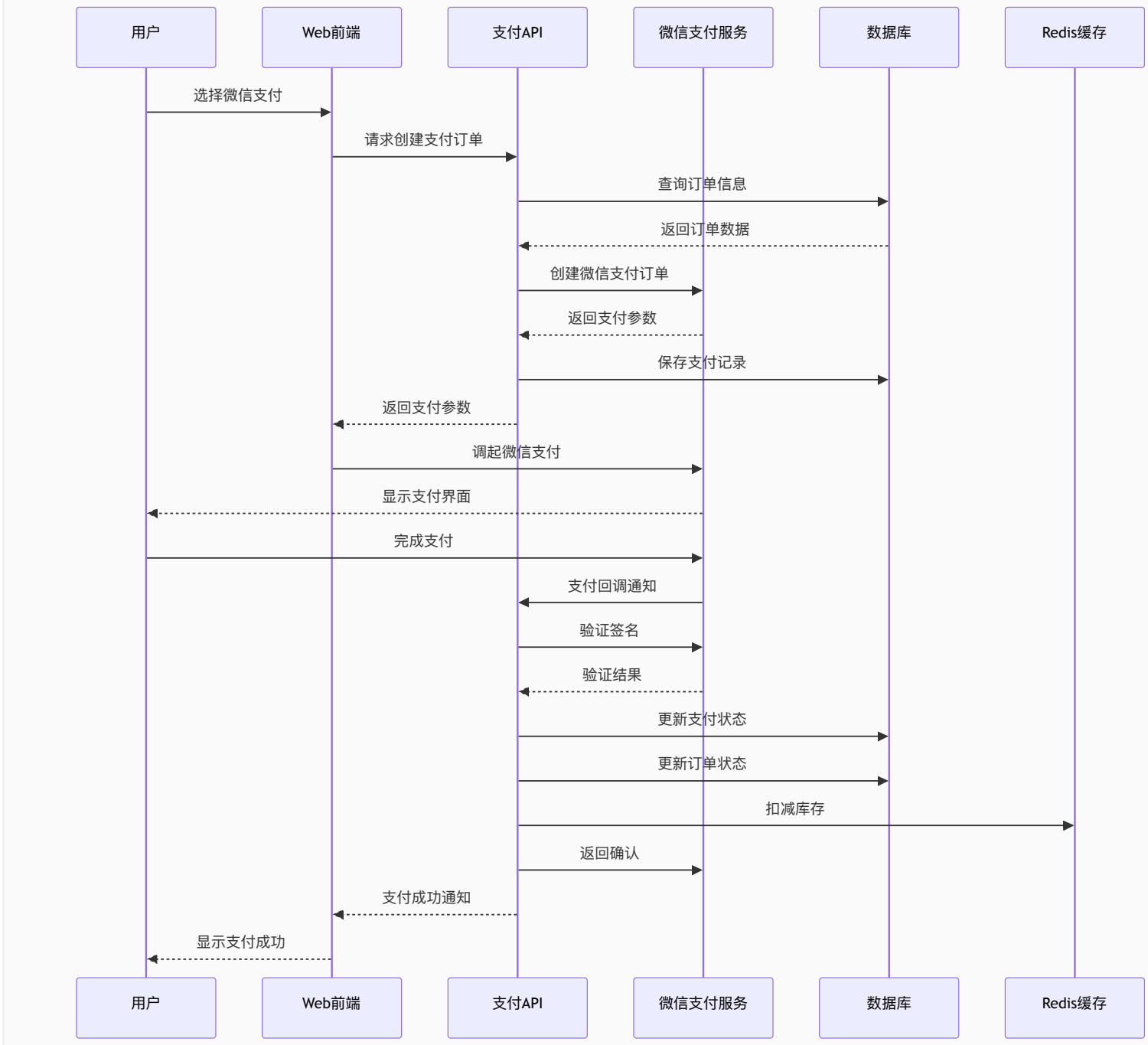
6.4 订单提交时序图


用户
Web前端
订单API
Redis缓存
数据库
库存服务
支付服务
提交订单
POST /order/create
获取购物车数据
返回购物车数据
检查库存
库存状态
创建订单记录
订单ID
创建订单商品记录
创建成功
锁定库存
锁定成功
创建支付订单
支付参数
返回支付信息
调起支付
支付界面
完成支付
支付回调
更新订单状态
扣减库存
支付成功通知
显示支付成功
用户
Web前端
订单API
Redis缓存
数据库
库存服务
支付服务

6.5 支付宝支付时序图


用户
Web前端
支付API
支付宝服务
数据库
Redis缓存
选择支付宝支付
请求创建支付订单
查询订单信息
返回订单数据
创建支付宝订单
返回支付参数
保存支付记录
返回支付参数
调起支付宝支付
显示支付界面
完成支付
支付回调通知
验证签名
验证结果
更新支付状态
更新订单状态
扣减库存
返回确认
支付成功通知
显示支付成功
用户
Web前端
支付API
支付宝服务
数据库
Redis缓存

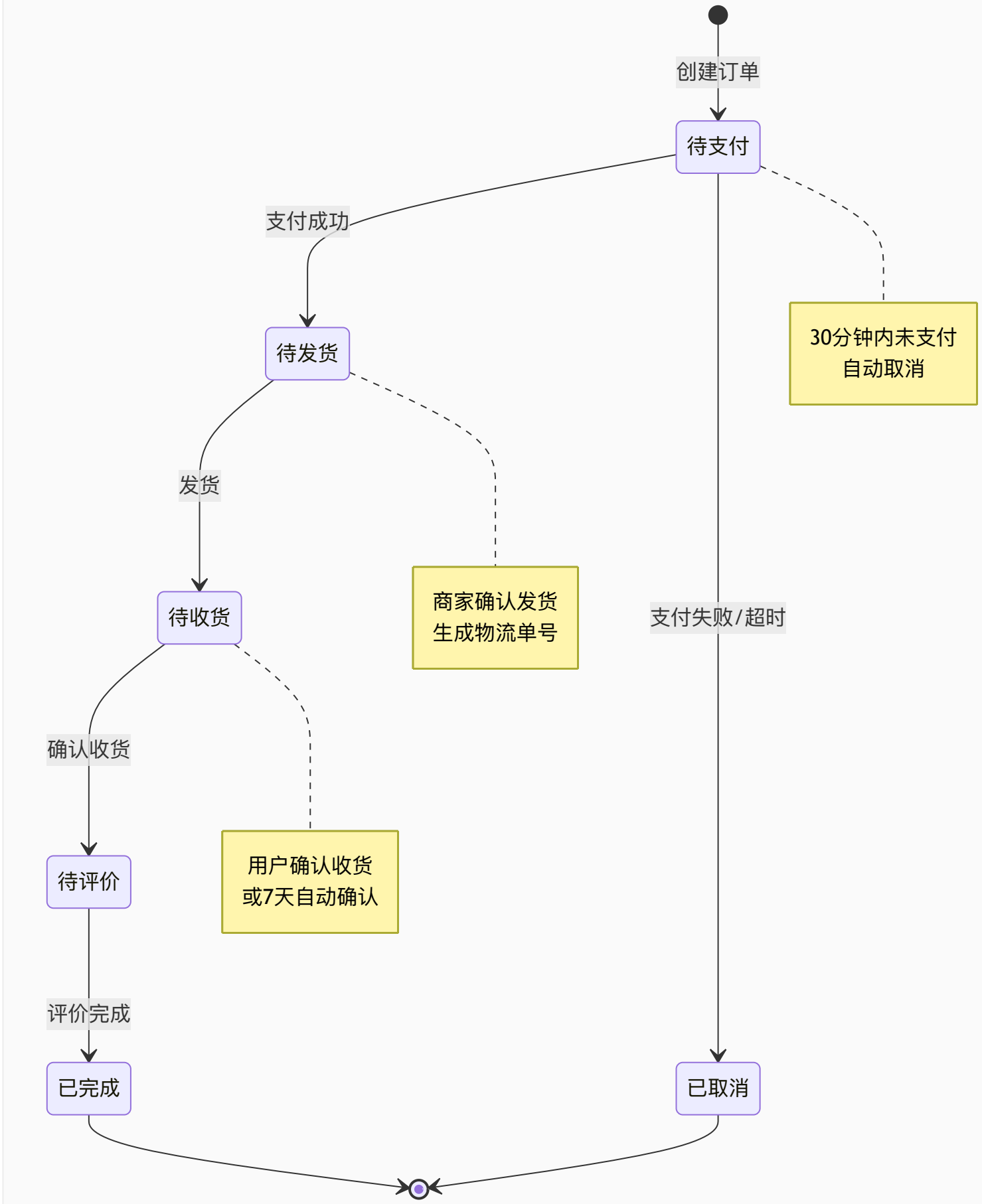
6.6 微信支付时序图



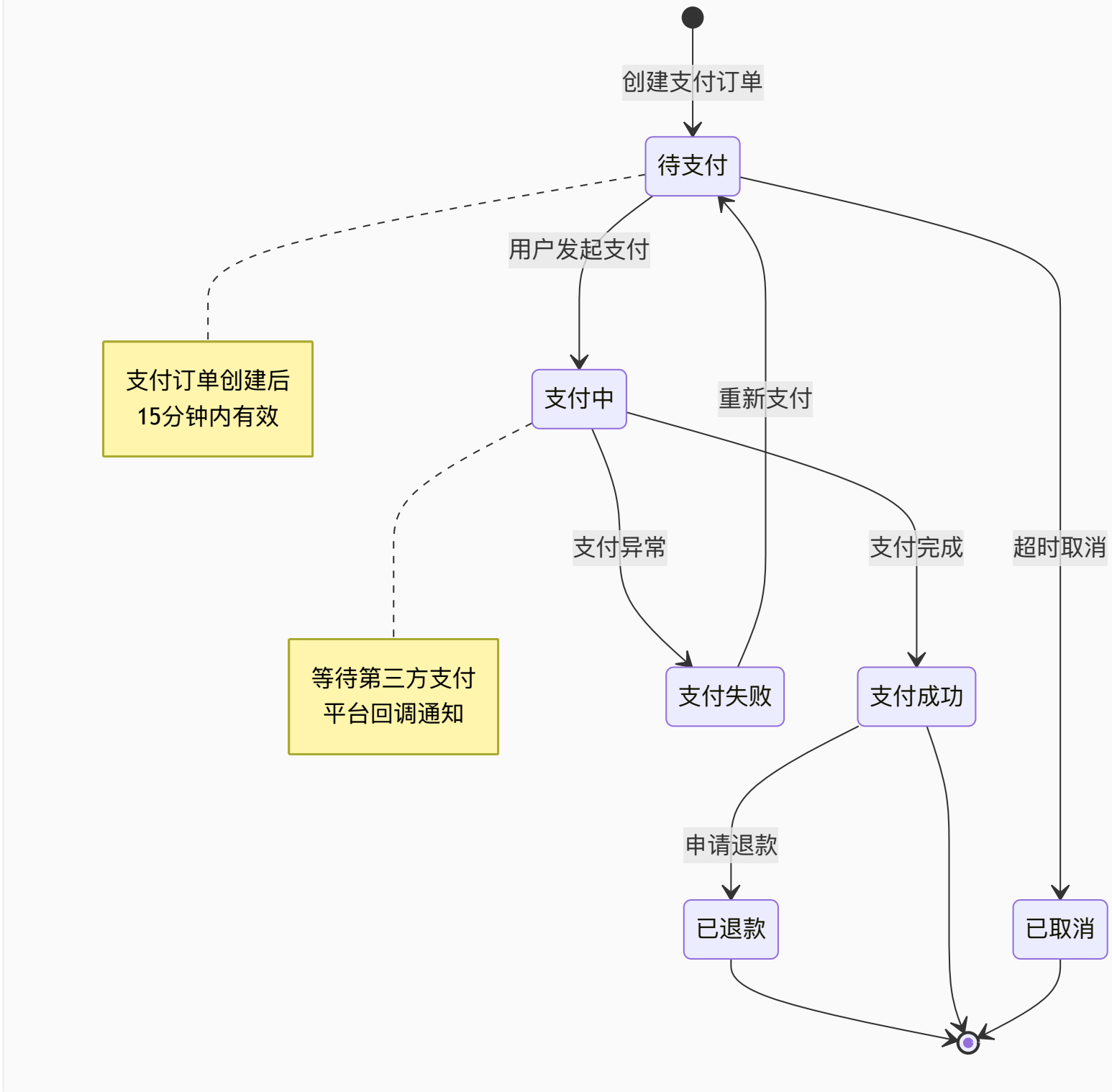
---

7. 状态图设计

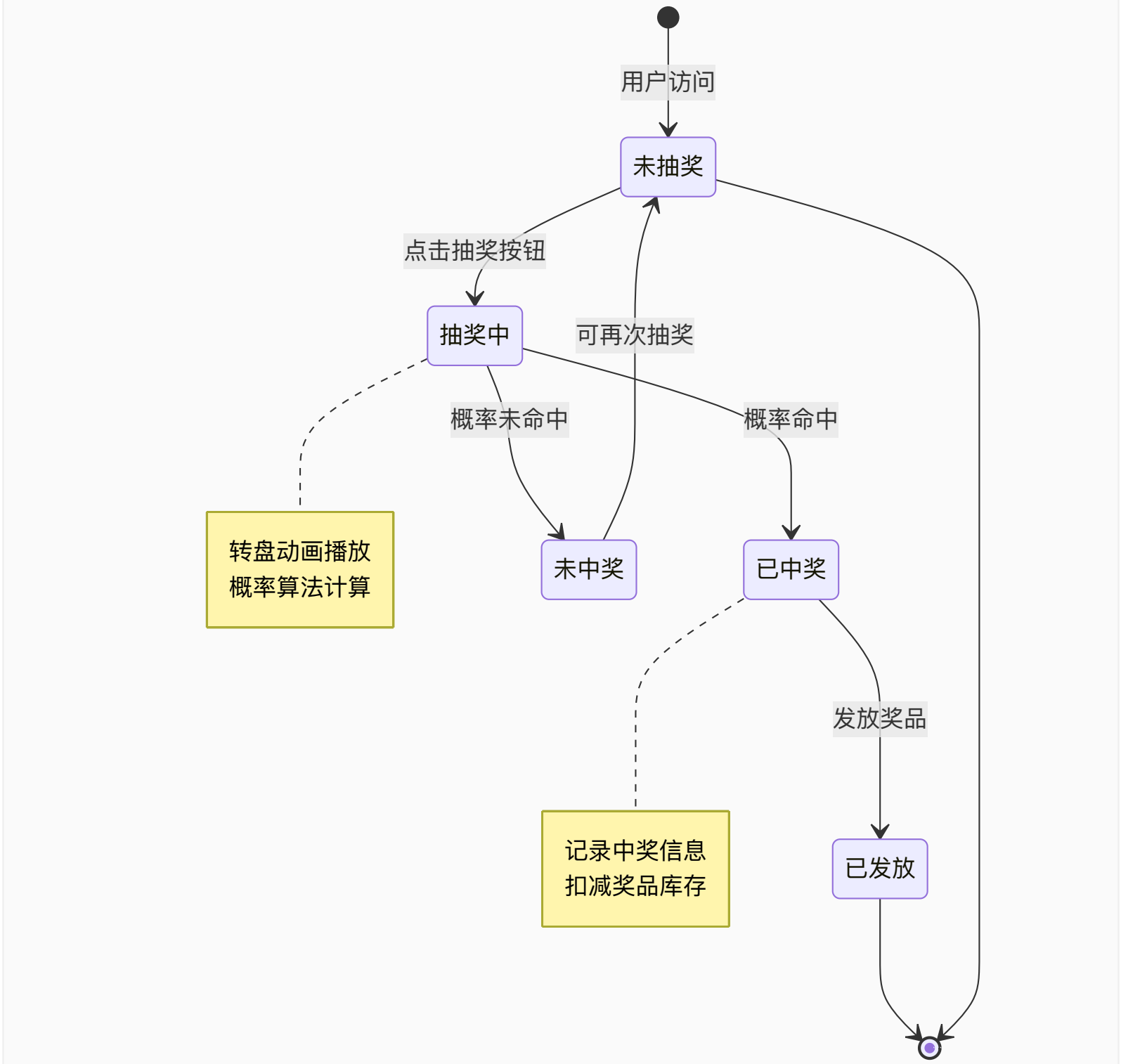
7.1 订单状态流转图


创建订单
待支付
支付成功
30分钟内未支付
自动取消
待发货
发货
商家确认发货
生成物流单号
支付失败/超时
待收货
确认收货
用户确认收货
或7天自动确认
待评价
评价完成
已完成
已取消

7.2 支付状态流转图


创建支付订单
待支付
用户发起支付
支付订单创建后
15分钟内有效
支付中
重新支付
支付异常
支付完成
超时取消
等待第三方支付
平台回调通知
支付失败
支付成功
申请退款
已退款
已取消

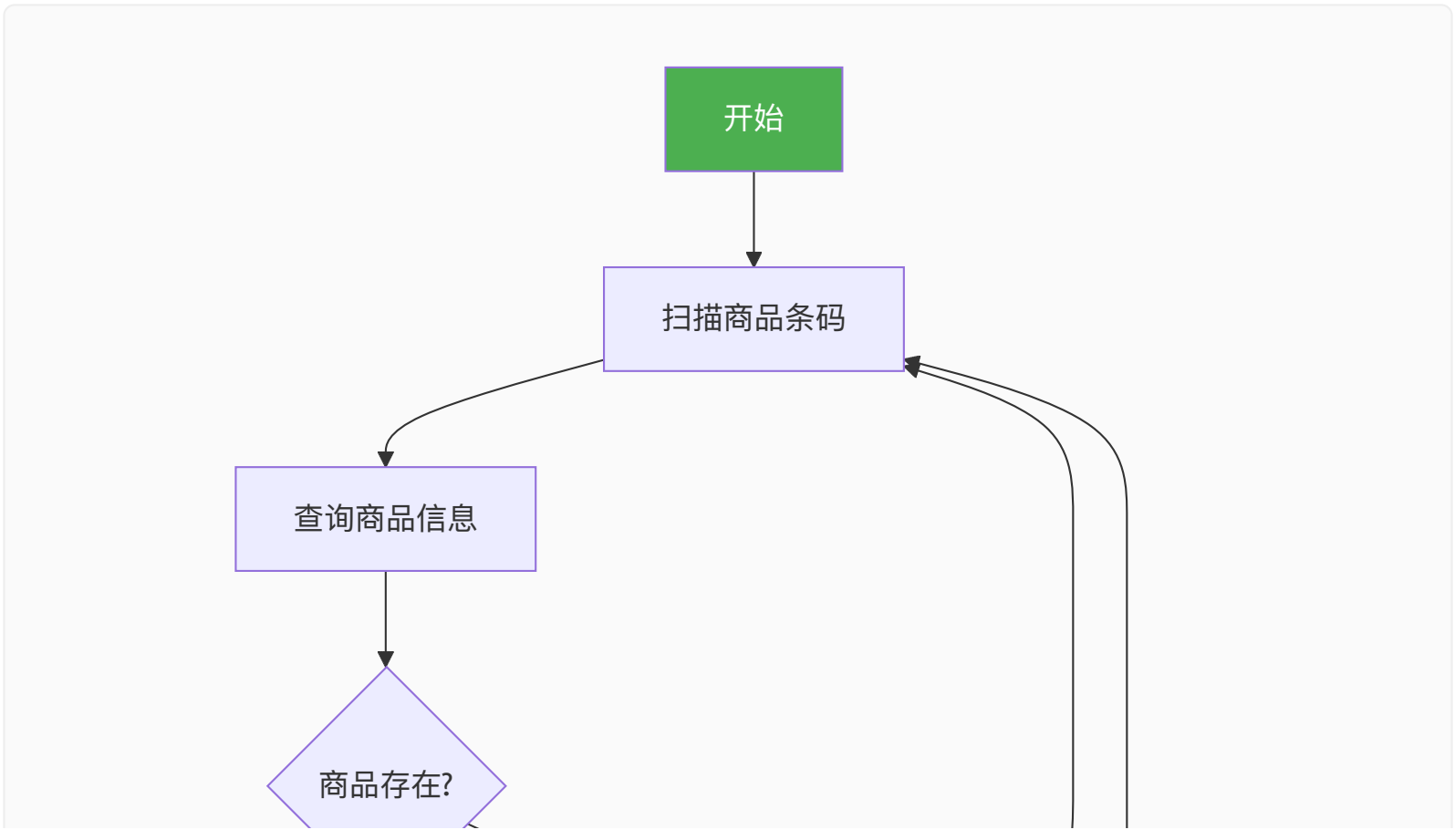
7.3 抽奖状态流转图

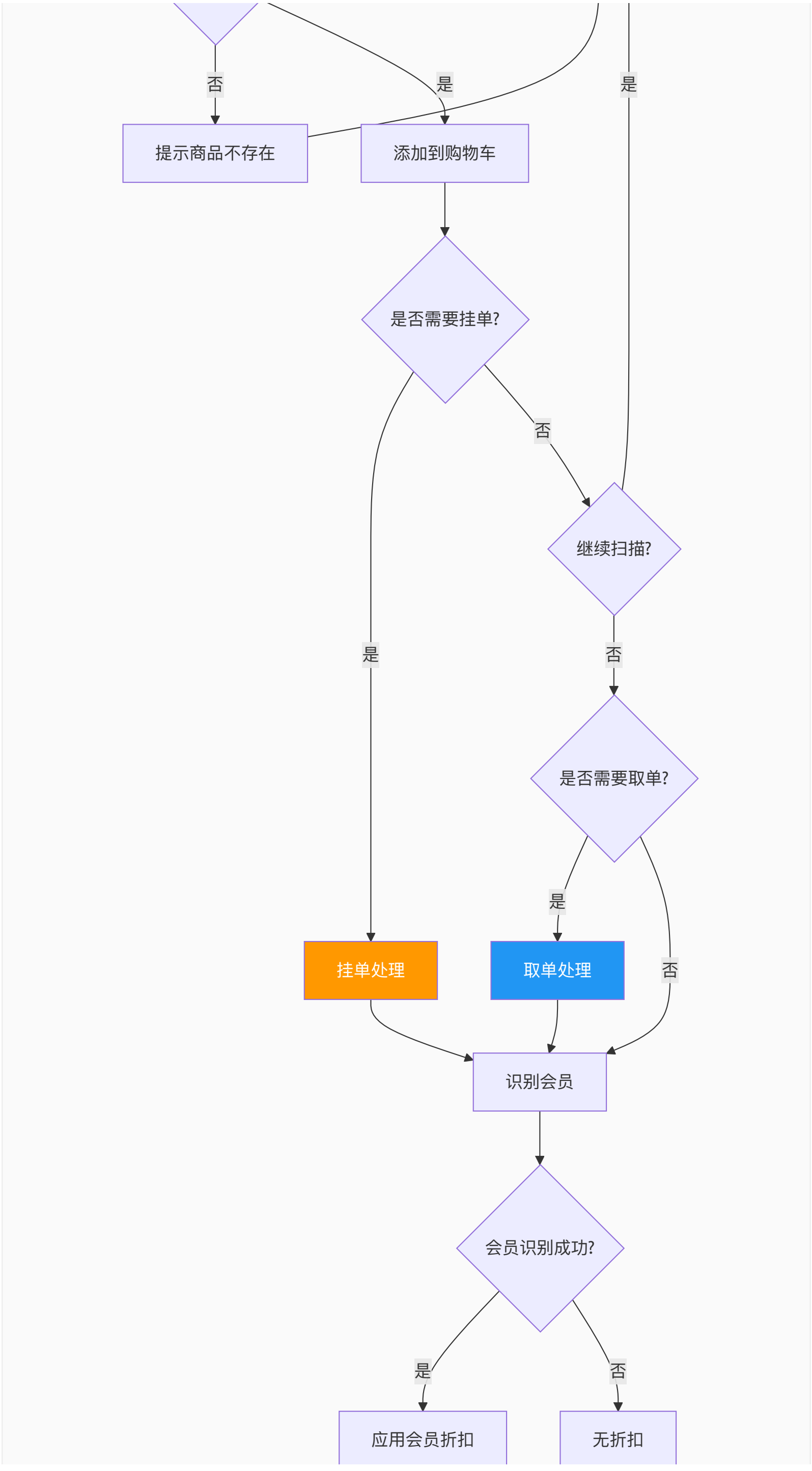


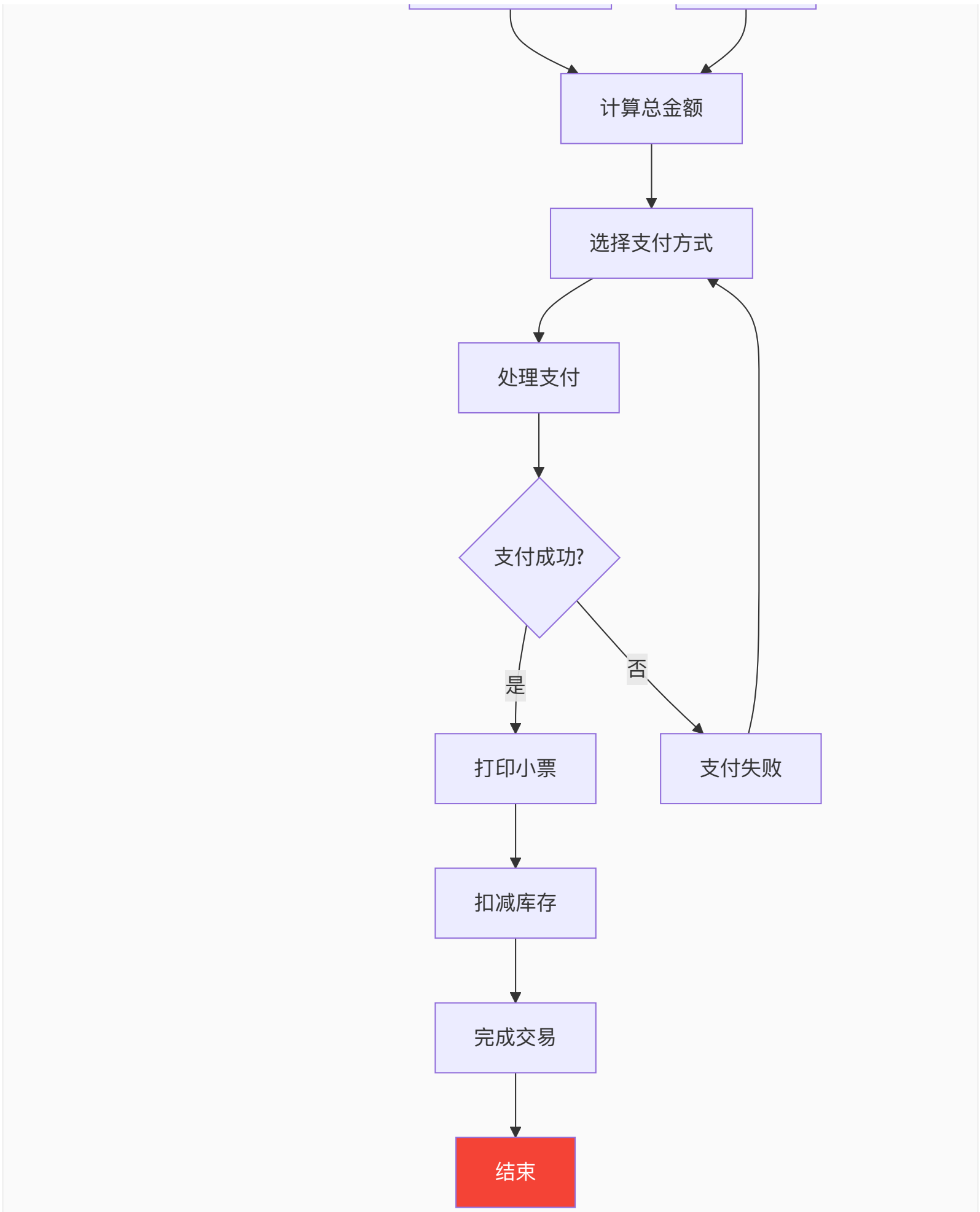
---

8. POS收银模块设计

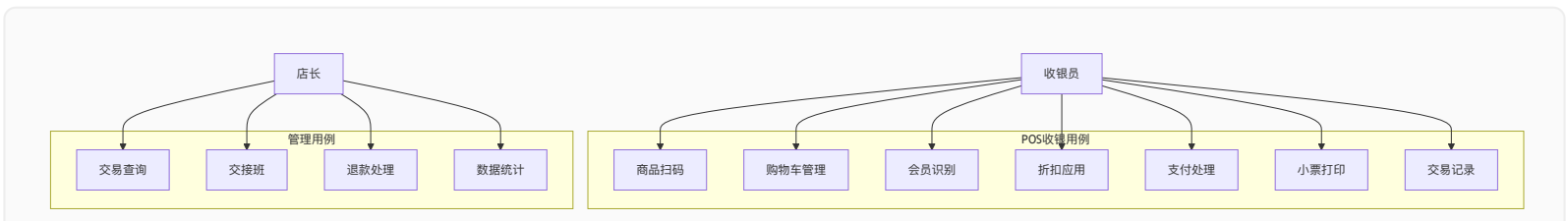
8.1 POS收银流程图



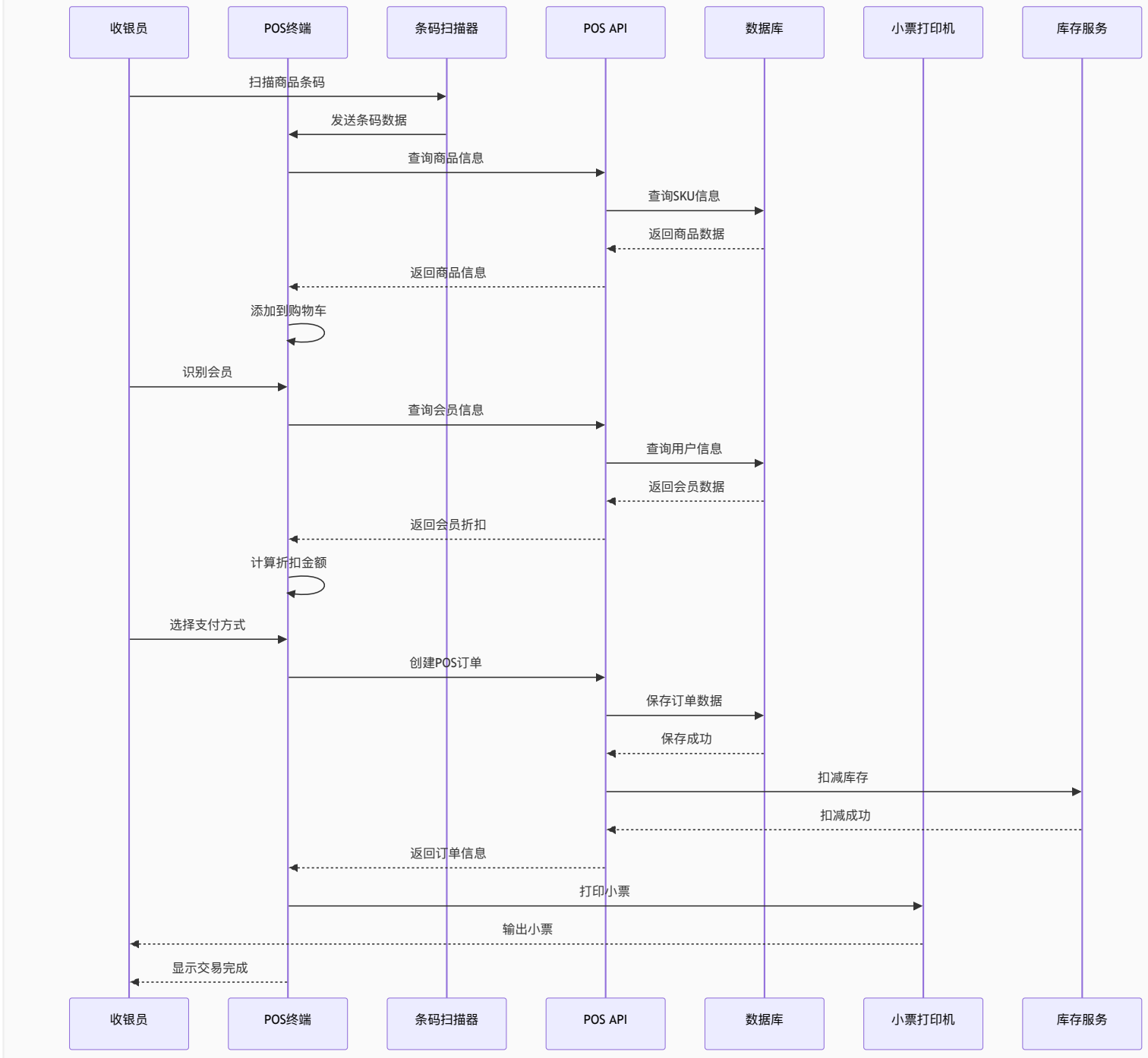

否
是
是
提示商品不存在
添加到购物车
是否需要挂单?
否
是
继续扫描?
否
是否需要取单?
是
否
挂单处理
取单处理
识别会员
会员识别成功?
是
否
应用会员折扣
无折扣



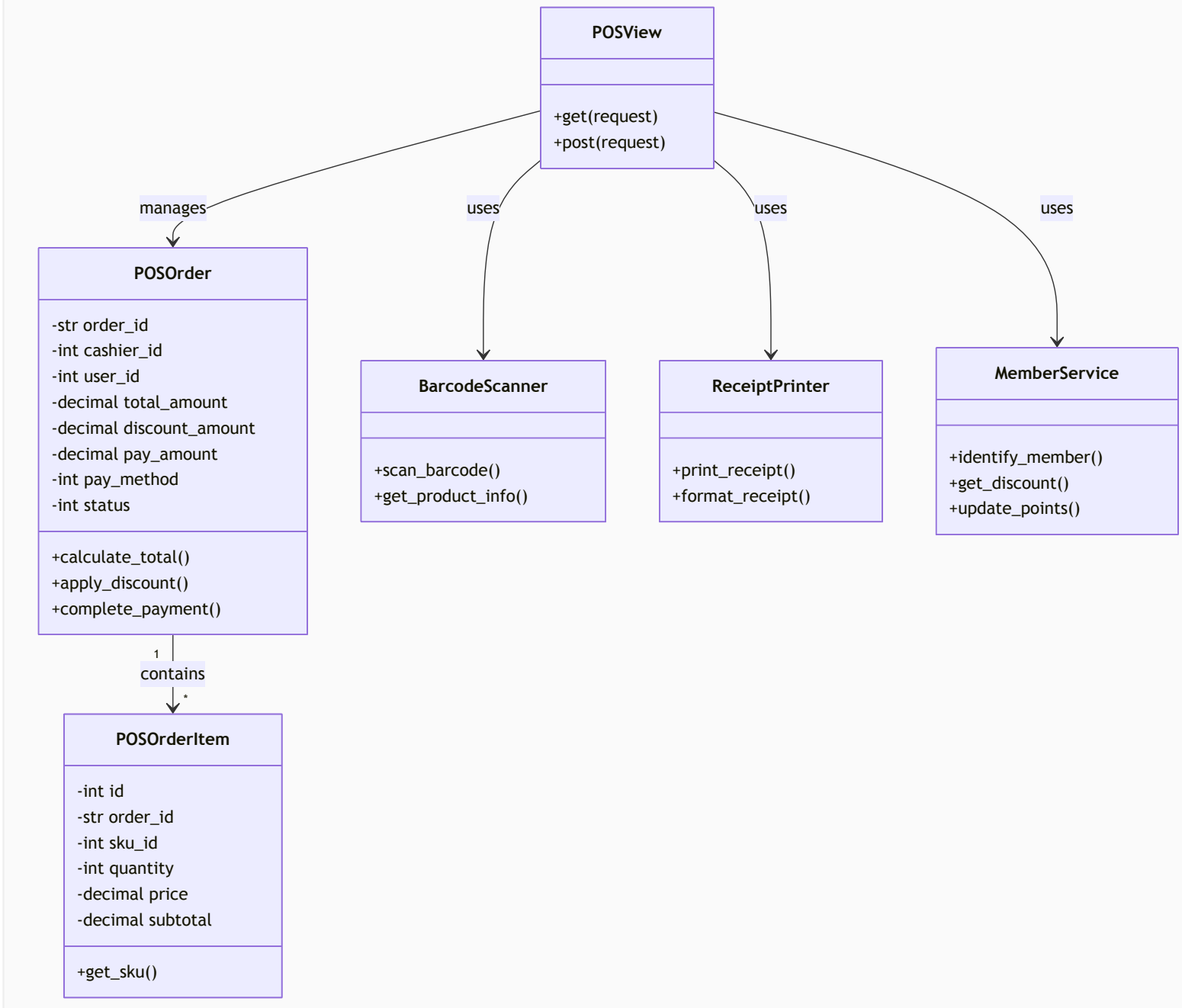
8.2 POS收银用例图



8.3 POS收银时序图


收银员
POS终端
条码扫描器
POS API
数据库
小票打印机
库存服务
扫描商品条码
发送条码数据
查询商品信息
查询SKU信息
返回商品数据
返回商品信息
添加到购物车
识别会员
查询会员信息
查询用户信息
返回会员数据
返回会员折扣
计算折扣金额
选择支付方式
创建POS订单
保存订单数据
保存成功
扣减库存
扣减成功
返回订单信息
打印小票
输出小票
显示交易完成
收银员
POS终端
条码扫描器
POS API
数据库
小票打印机
库存服务

8.4 POS收银类图



8.5 POS挂单取单功能设计

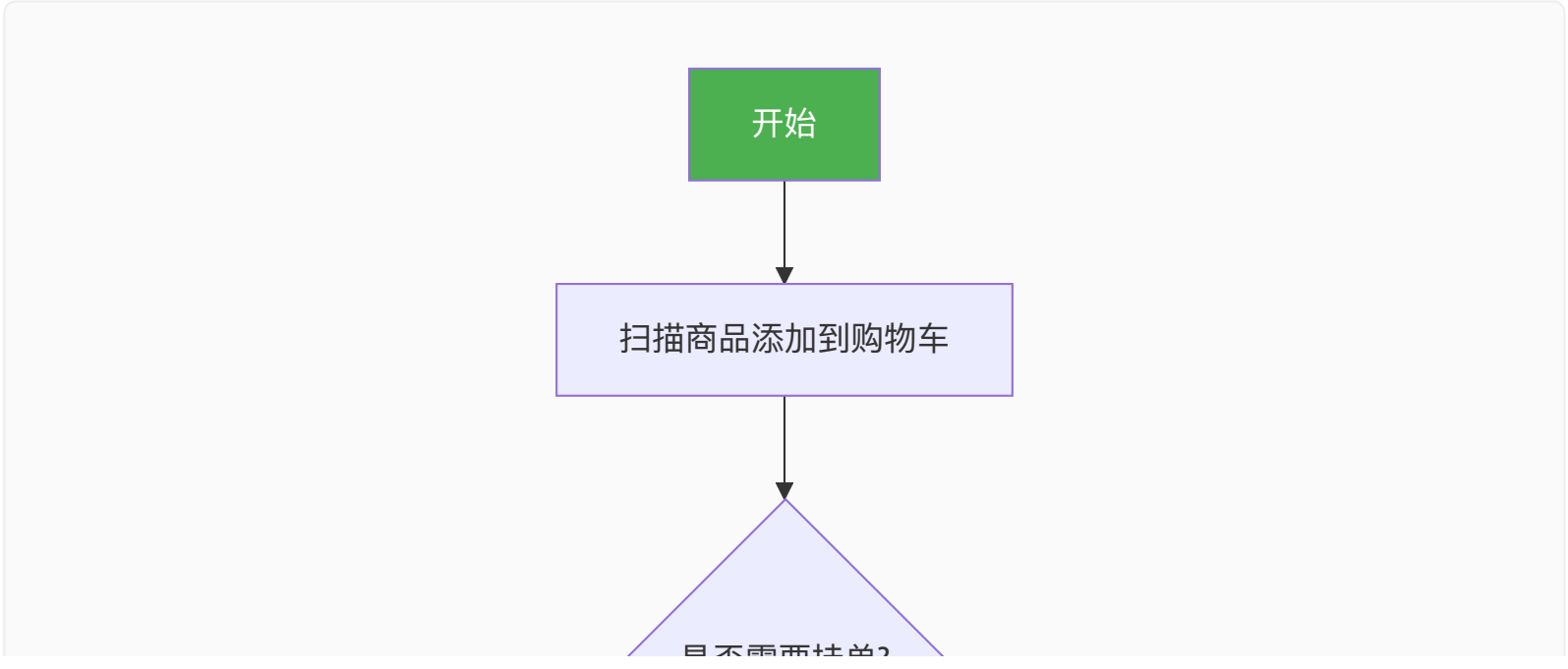
8.5.1 挂单取单业务说明

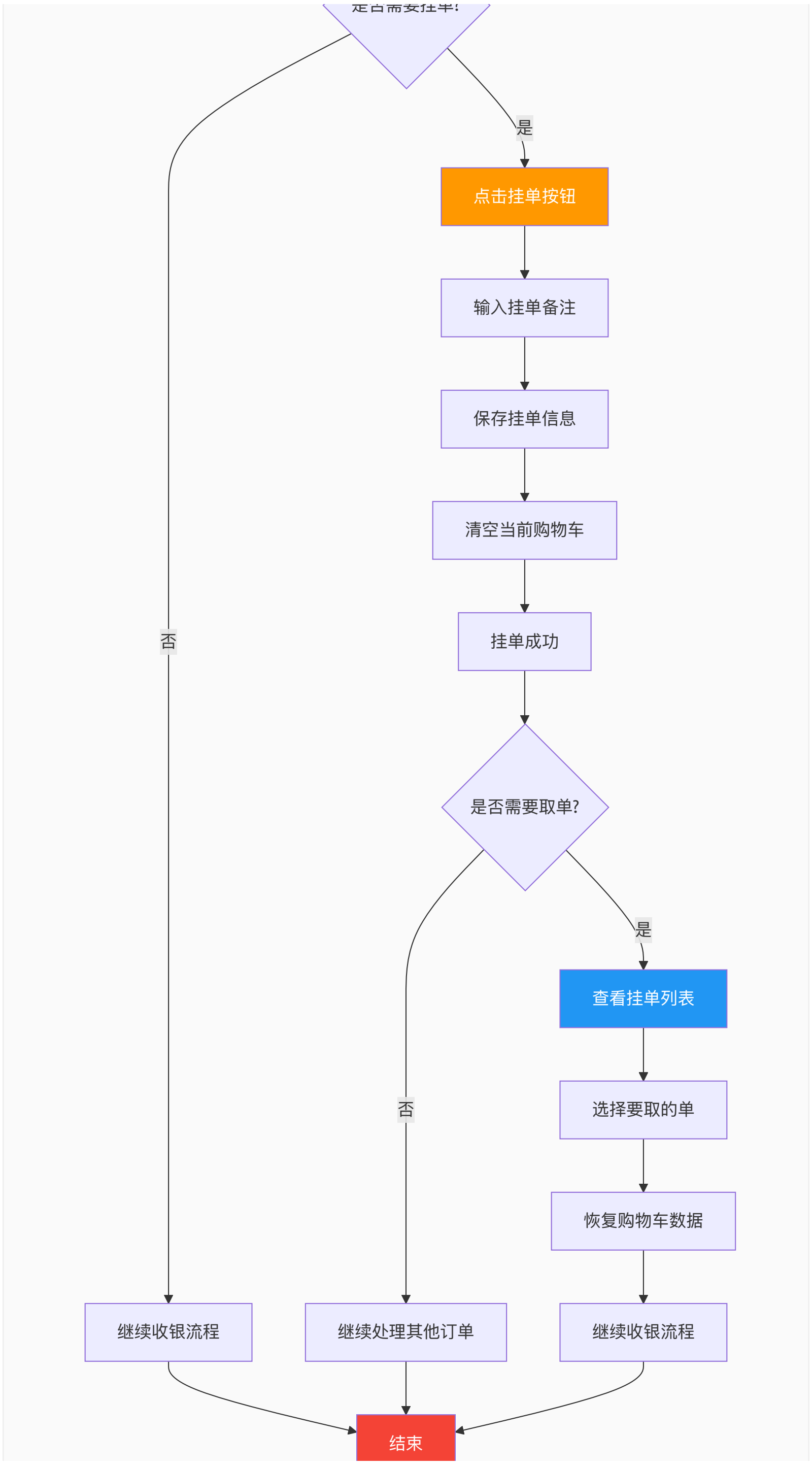
挂单功能允许收银员在收银过程中将当前未完成的订单暂时保存，以便处理其他紧急事务或接待其他顾客。取单功能允许收银员将之前挂起的订单恢复到收银界面继续处理。

业务场景

- 顾客临时离开，需要先处理其他顾客
- 收银员需要交接班或处理其他紧急事务
- 顾客需要补充商品或修改订单
- 系统故障或网络中断时的临时保存

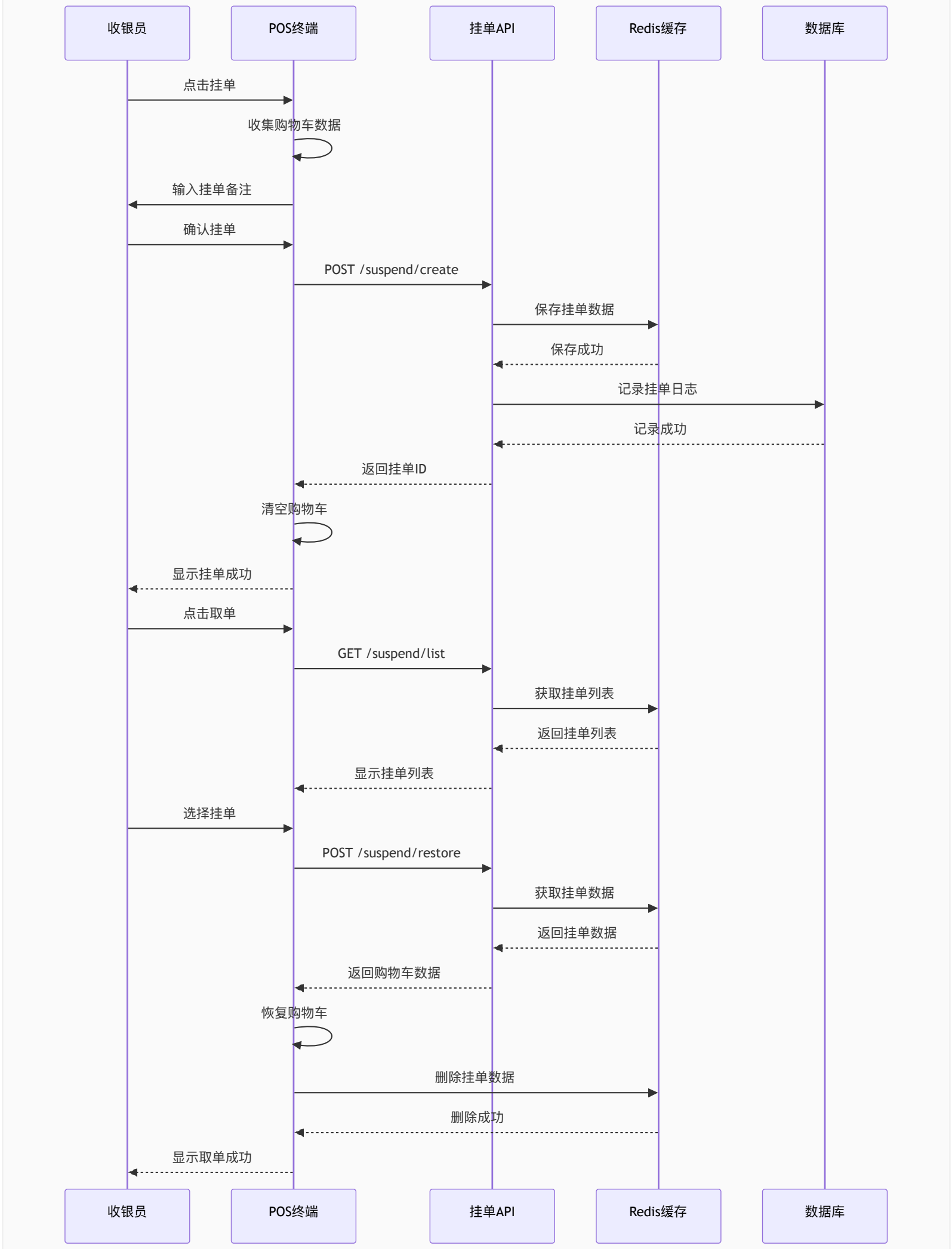
8.5.2 挂单取单流程图



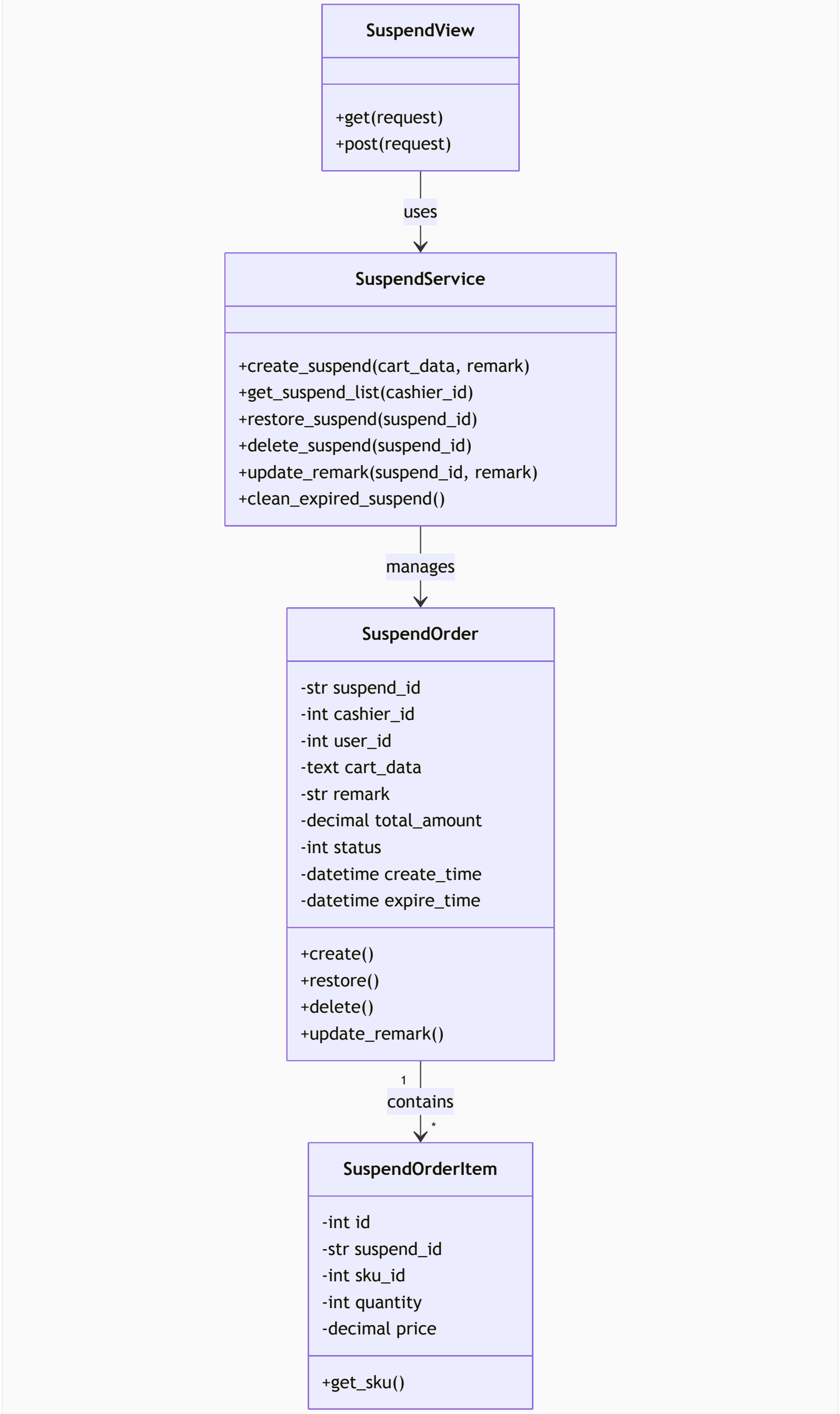

是否需要挂单?
是
点击挂单按钮
输入挂单备注
保存挂单信息
清空当前购物车
挂单成功
是否需要取单?
否
是
查看挂单列表
选择要取的单
恢复购物车数据
否
继续收银流程
继续处理其他订单
继续收银流程
结束

8.5.3 挂单取单用例图

8.5.4 挂单取单时序图


收银员
POS终端
挂单API
Redis缓存
数据库
点击挂单
收集购物车数据
输入挂单备注
确认挂单
POST /suspend/create
保存挂单数据
保存成功
记录挂单日志
记录成功
返回挂单ID
清空购物车
显示挂单成功
点击取单
GET /suspend/list
获取挂单列表
返回挂单列表
显示挂单列表
选择挂单
POST /suspend/restore
获取挂单数据
返回挂单数据
返回购物车数据
恢复购物车
删除挂单数据
删除成功
显示取单成功
收银员
POS终端
挂单API
Redis缓存
数据库

8.5.5 挂单取单类图


SuspendView
+get(request)
+post(request)
uses
SuspendService
+create_suspend(cart_data, remark)
+get_suspend_list(cashier_id)
+restore_suspend(suspend_id)
+delete_suspend(suspend_id)
+update_remark(suspend_id, remark)
+clean_expired_suspend()
manages
SuspendOrder
-str suspend_id
-int cashier_id
-int user_id
-text cart_data
-str remark
-decimal total_amount
-int status
-datetime create_time
-datetime expire_time
+create()
+restore()
+delete()
+update_remark()
1
contains
*
SuspendOrderItem
-int id
-str suspend_id
-int sku_id
-int quantity
-decimal price
+get_sku()

8.5.6 挂单数据表设计

tb_suspended_order (挂单表)

字段名	类型	长度	允许空	默认值	说明
id	INT	11	NO	AUTO_INCREMENT	主键ID
cashier_id	INT	11	NO	-	收银员ID (外键)
member_id	INT	11	YES	NULL	会员ID (外键)
total_count	INT	11	NO	0	商品总数
total_amount	DECIMAL	10,2	NO	0	总金额
cart_data	TEXT	-	NO	-	购物车数据(JSON格式)
remark	TEXT	-	NO	""	挂单备注
create_time	DATETIME	-	NO	CURRENT_TIMESTAMP	创建时间
update_time	DATETIME	-	NO	CURRENT_TIMESTAMP	更新时间

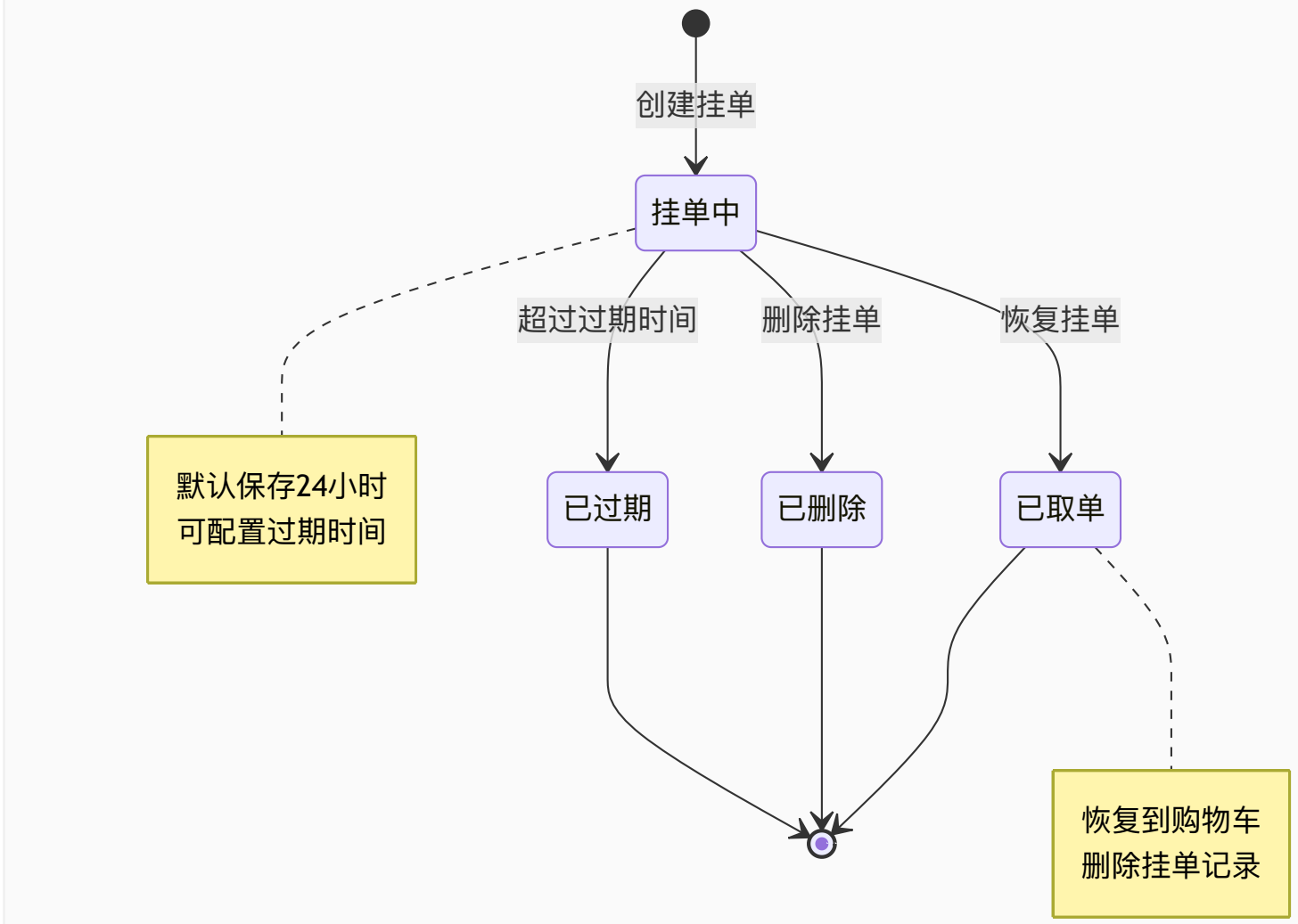
索引设计

- PRIMARY KEY: `id`
- INDEX: `cashier_id`
- INDEX: `member_id`
- INDEX: `create_time`

外键关系

- `cashier_id` → `tb_user.id` (收银员)
- `member_id` → `tb_user.id` (会员，可为空)

8.5.7 挂单取单状态流转图



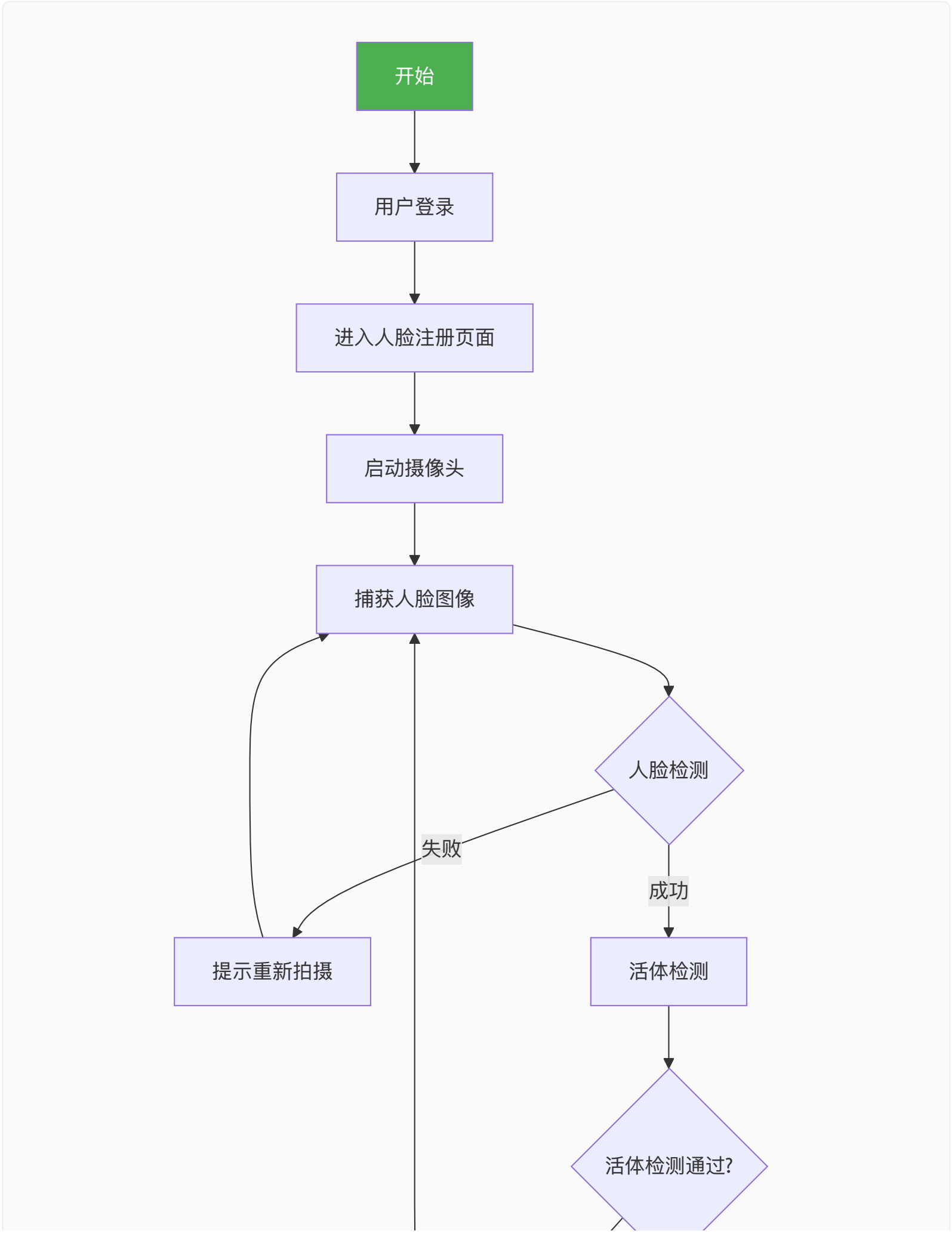
8.5.8 挂单取单功能特点

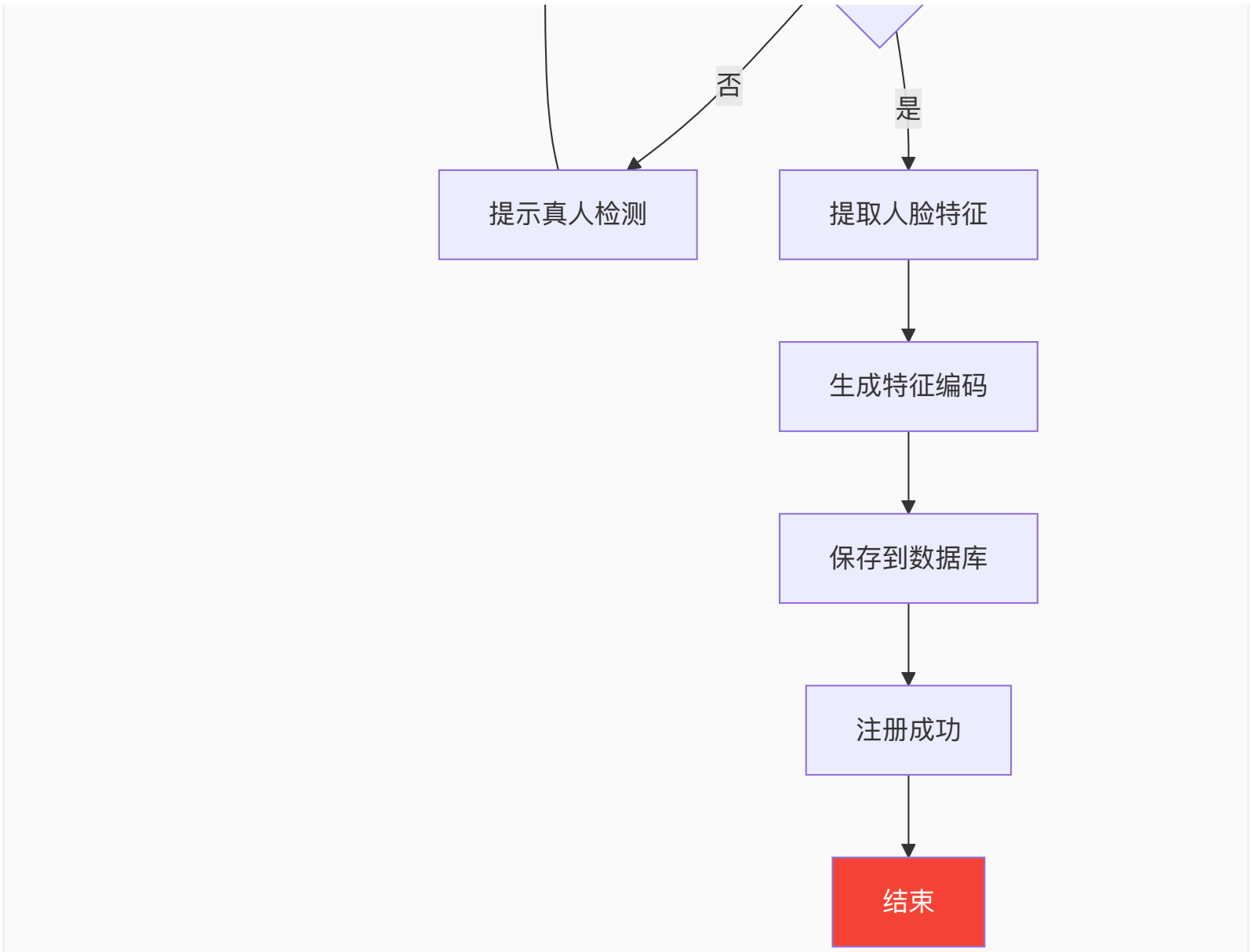
1. **快速挂单**: 一键保存当前购物车状态
2. **备注功能**: 支持添加挂单备注，便于识别
3. **挂单列表**: 查看所有挂单记录
4. **快速取单**: 一键恢复挂单到购物车
5. **自动过期**: 超过设定时间自动清理
6. **权限控制**: 仅收银员可操作自己的挂单
7. **数据持久化**: 使用Redis缓存+数据库双重存储

---

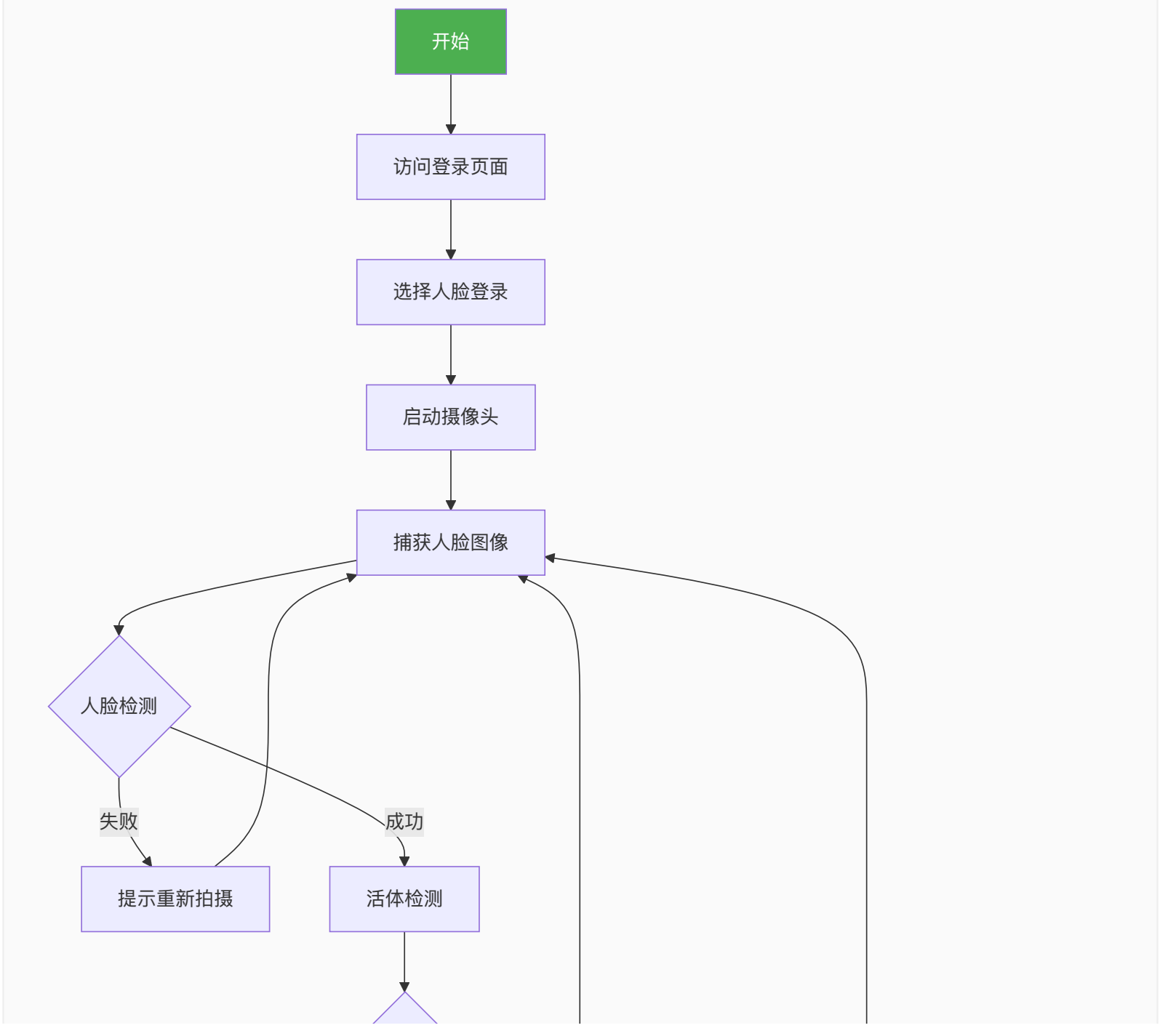
9. 人脸识别模块设计

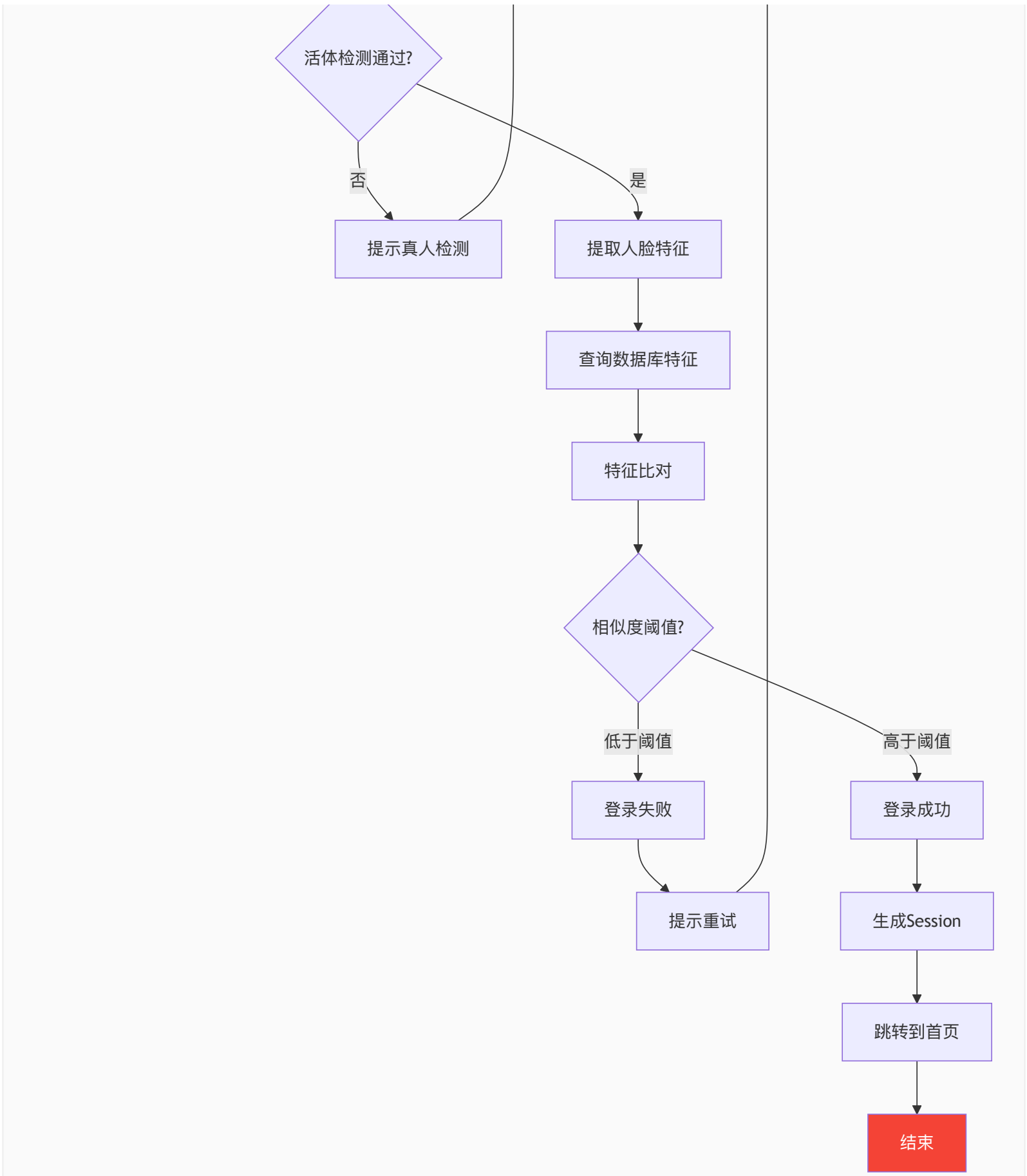
9.1 人脸注册流程图



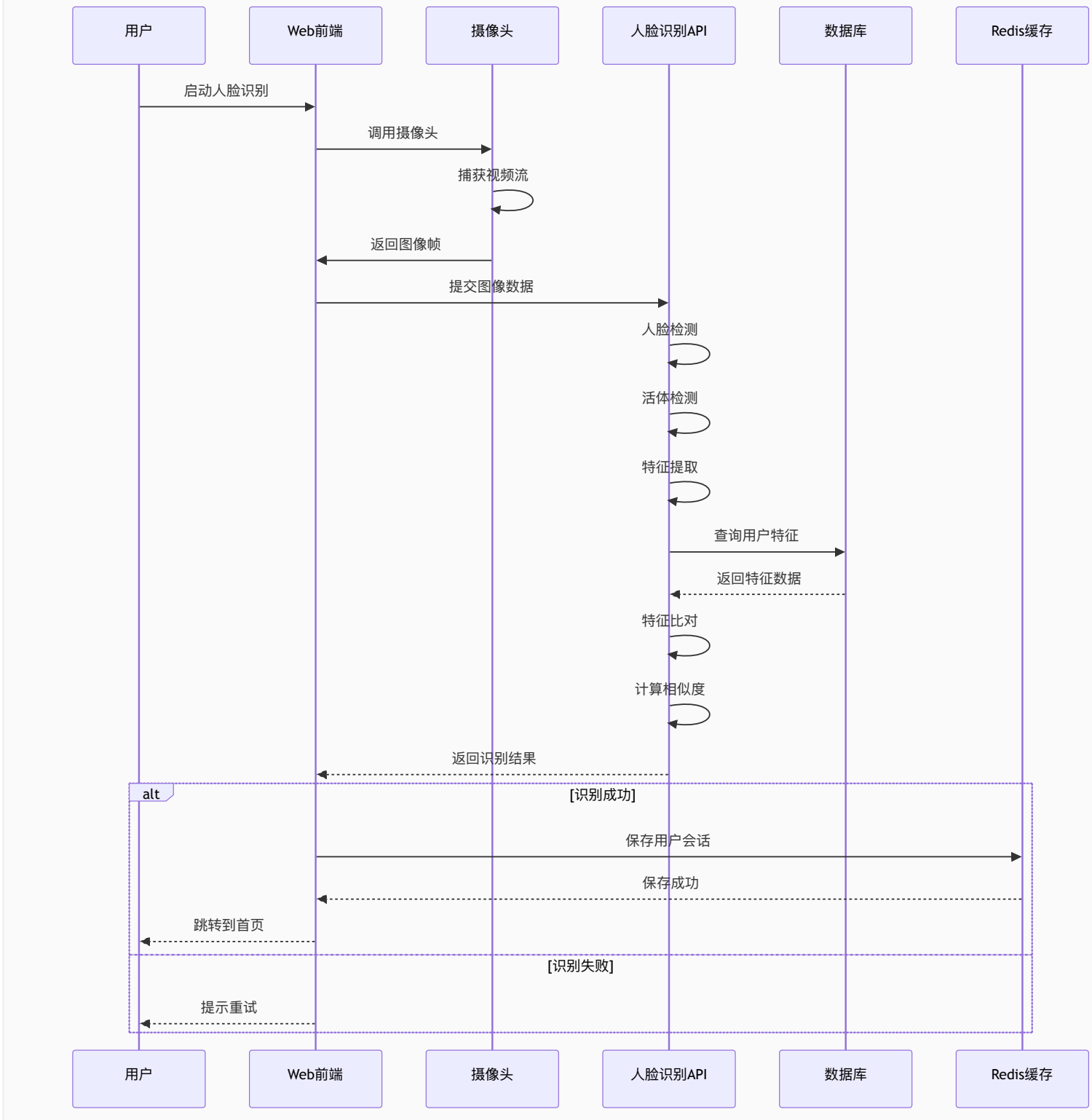

否
是
提示真人检测
提取人脸特征
生成特征编码
保存到数据库
注册成功
结束

9.2 人脸登录流程图

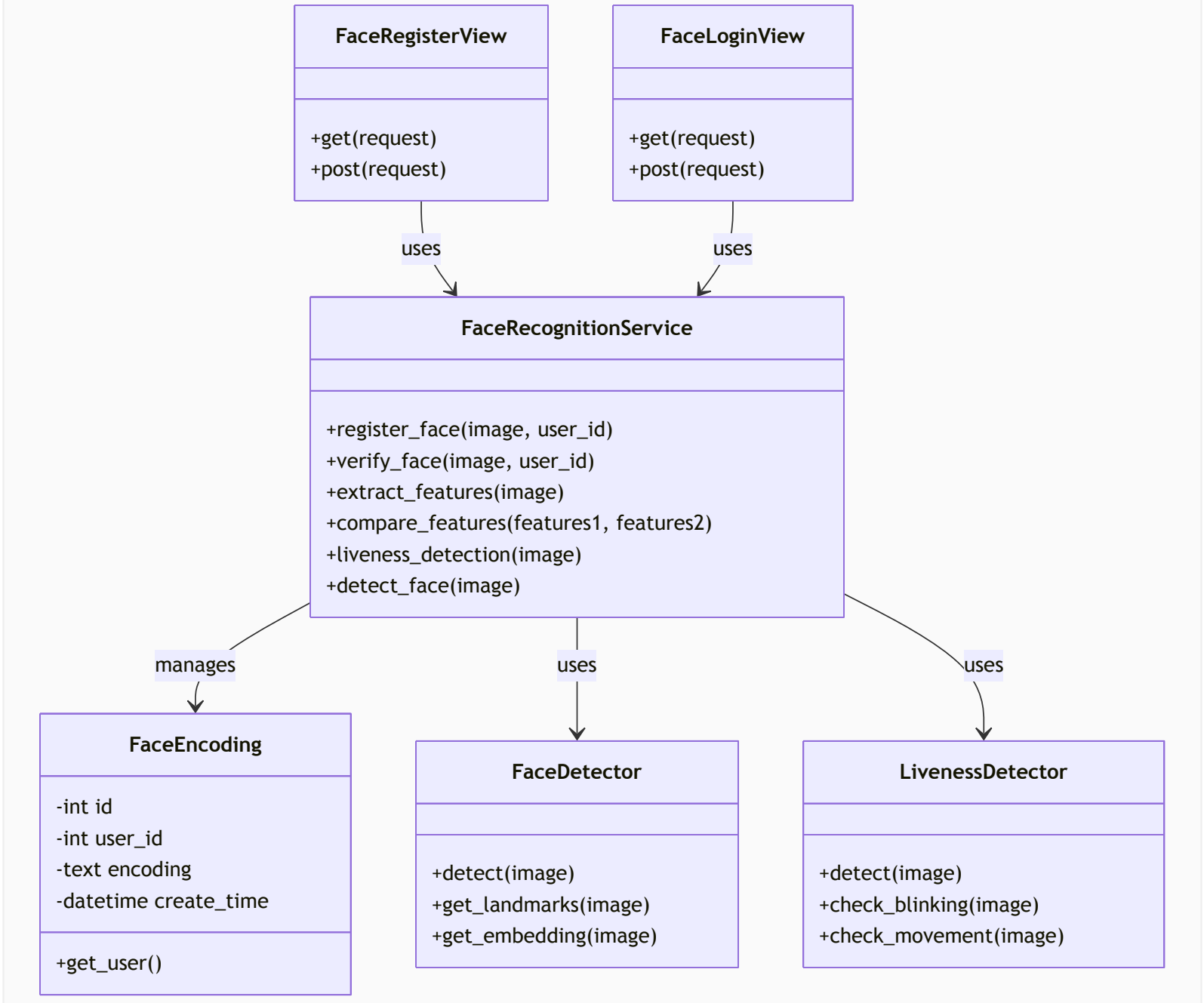

开始
访问登录页面
选择人脸登录
启动摄像头
捕获人脸图像
人脸检测
失败
成功
提示重新拍摄
活体检测


活体检测通过?
否
是
提示真人检测
提取人脸特征
查询数据库特征
特征比对
相似度阈值?
低于阈值
高于阈值
登录失败
登录成功
提示重试
生成Session
跳转到首页
结束

9.3 人脸识别时序图


用户
Web前端
摄像头
人脸识别API
数据库
Redis缓存
启动人脸识别
调用摄像头
捕获视频流
返回图像帧
提交图像数据
人脸检测
活体检测
特征提取
查询用户特征
返回特征数据
特征比对
计算相似度
返回识别结果
alt
[识别成功]
保存用户会话
保存成功
跳转到首页
[识别失败]
提示重试
用户
Web前端
摄像头
人脸识别API
数据库
Redis缓存

9.4 人脸识别类图



---

10. 系统安全设计

10.1 认证与授权

10.1.1 认证机制

- 基于Session的传统认证
- JWT Token认证（可选）
- 人脸识别认证（创新功能）

10.1.2 权限控制

- 基于角色的访问控制（RBAC）
- 装饰器权限验证
- 中间件权限拦截

10.2 数据安全

10.2.1 密码安全

- 使用PBKDF2算法加密存储
- 盐值随机生成
- 密码复杂度要求

10.2.2 敏感数据保护

- 手机号脱敏显示
- 支付信息加密传输
- 数据库字段加密

10.3 接口安全

10.3.1 CSRF防护

- CSRF Token验证
- SameSite Cookie属性
- Referer检查

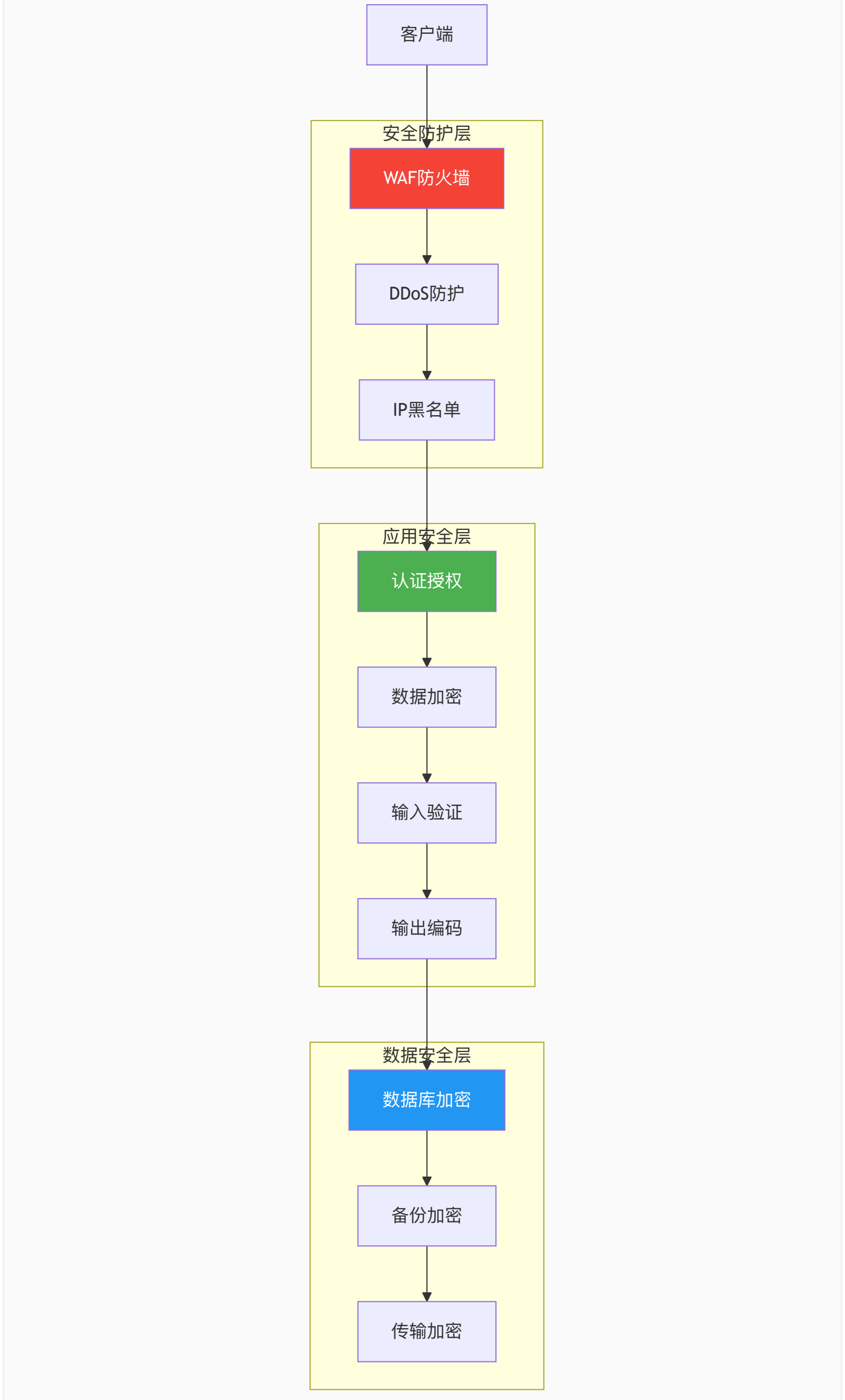
10.3.2 XSS防护

- 模板自动转义
- 输入内容过滤
- CSP策略配置

10.3.3 SQL注入防护

- ORM参数化查询
- 输入验证和过滤
- 最小权限原则

10.4 安全架构图


客户端
安全防护层
WAF防火墙
DDoS防护
IP黑名单
应用安全层
认证授权
数据加密
输入验证
输出编码
数据安全层
数据库加密
备份加密
传输加密

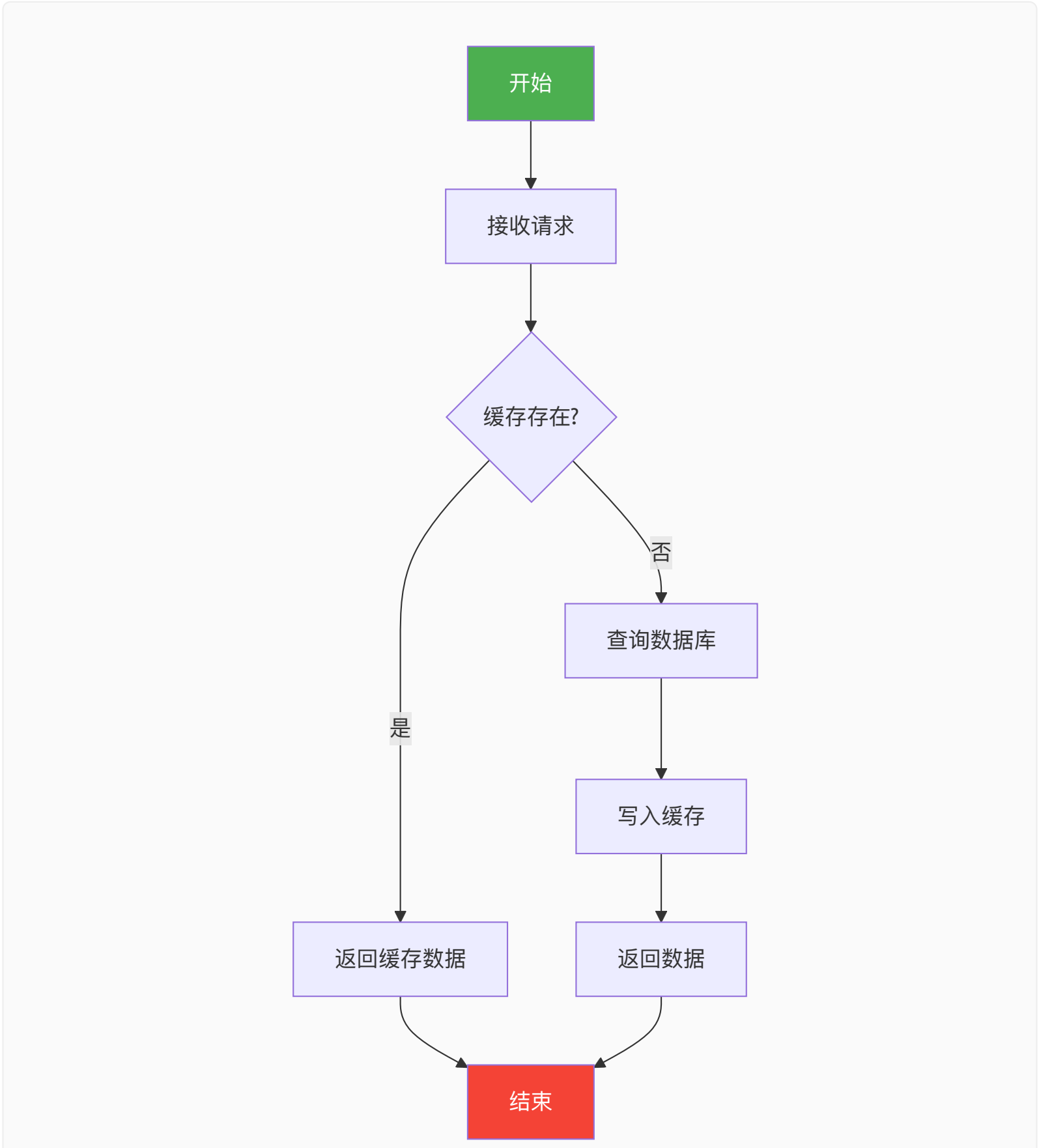
11. 性能优化设计

11.1 缓存设计

11.1.1 缓存策略

- **热点数据缓存**: 商品信息、用户信息
- **查询结果缓存**: 搜索结果、分类列表
- **页面缓存**: 静态页面、首页数据

11.1.2 缓存读写逻辑示意图



11.1.3 缓存更新策略

- **主动更新**: 数据变更时主动更新缓存
- **被动更新**: 设置过期时间，自动失效
- **缓存预热**: 系统启动时加载热点数据

11.2 数据库性能优化

11.2.1 读写分离

- 主库负责写操作
- 从库负责读操作
- 中间件自动路由

11.2.2 连接池配置

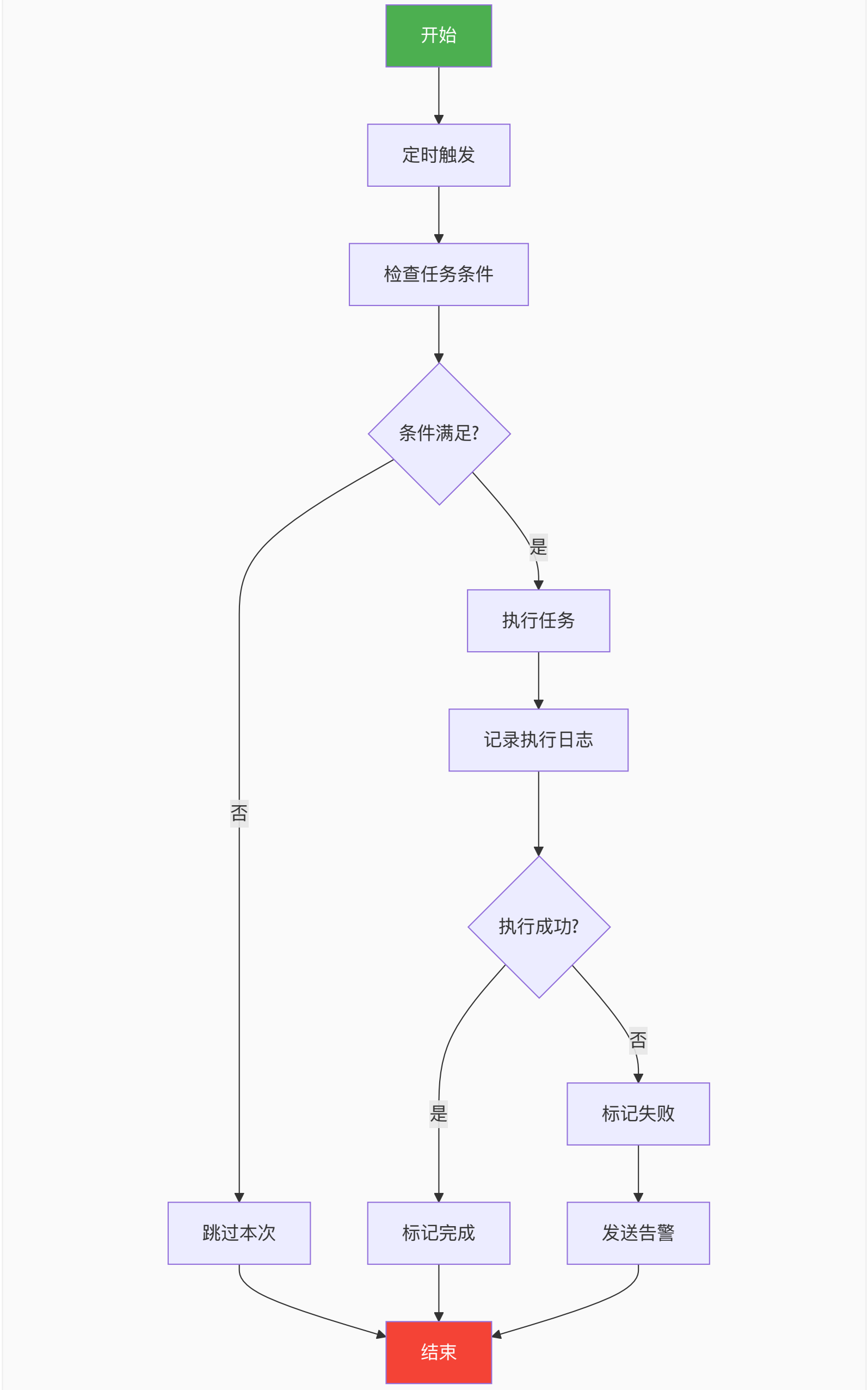
- 使用数据库连接池
- 合理设置连接数
- 连接超时配置

11.3 异步处理

11.3.1 Celery异步任务

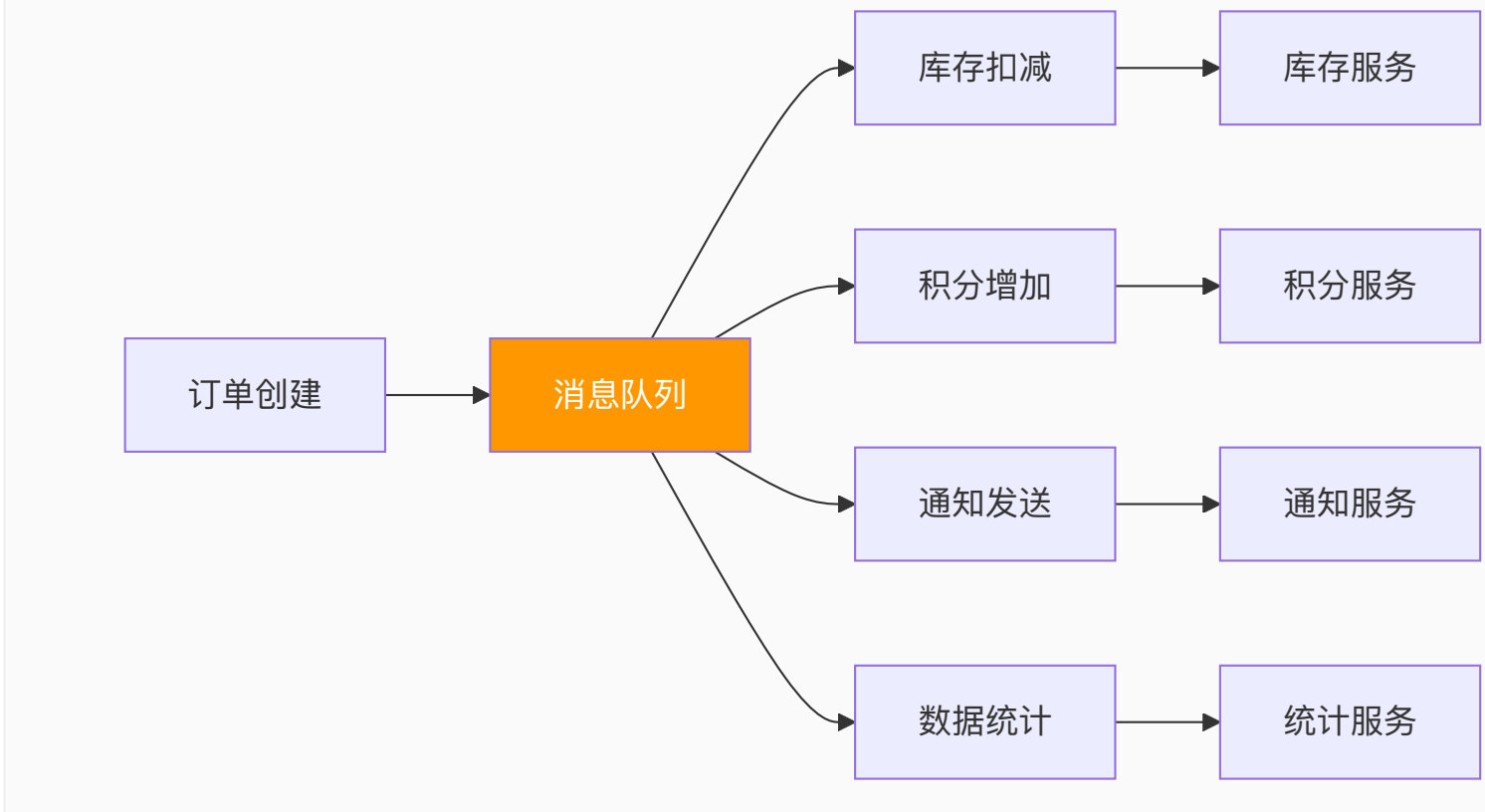
- 订单状态同步
- 邮件发送
- 数据统计
- 日志处理

11.3.2 定时任务执行流程图


开始
定时触发
检查任务条件
条件满足?
是
否
执行任务
记录执行日志
执行成功?
是
否
标记失败
跳过本次
标记完成
发送告警
结束

11.4 消息队列设计

11.4.1 消息时间拓扑图



11.5 前端性能优化

11.5.1 资源优化

- 图片压缩和懒加载
- CSS/JS压缩合并
- CDN加速
- 浏览器缓存

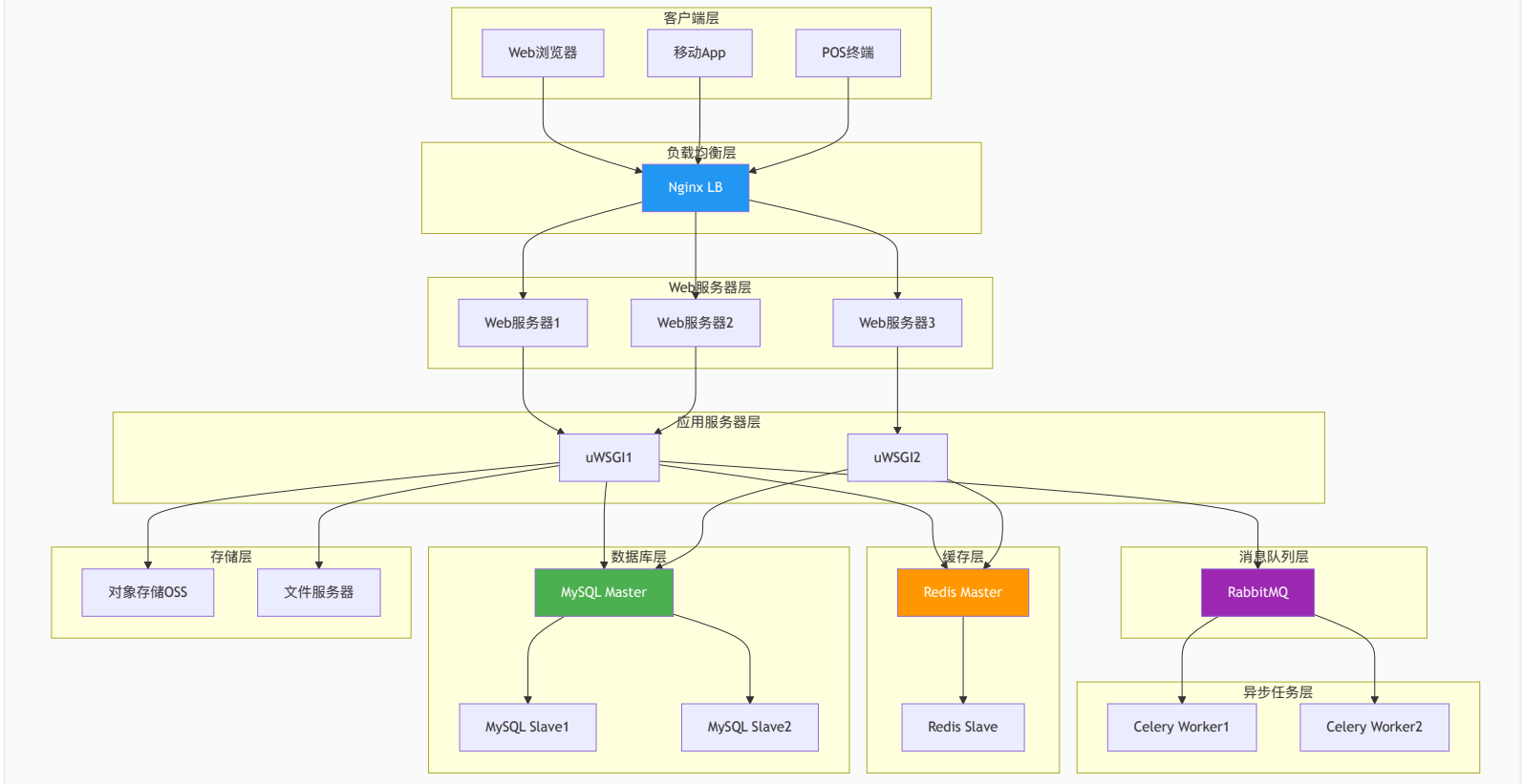
11.5.2 渲染优化

- 虚拟滚动
- 防抖节流
- 代码分割
- 服务端渲染

---

12. 部署运维设计

12.1 UML部署图



12.2 环境配置

12.2.1 开发环境

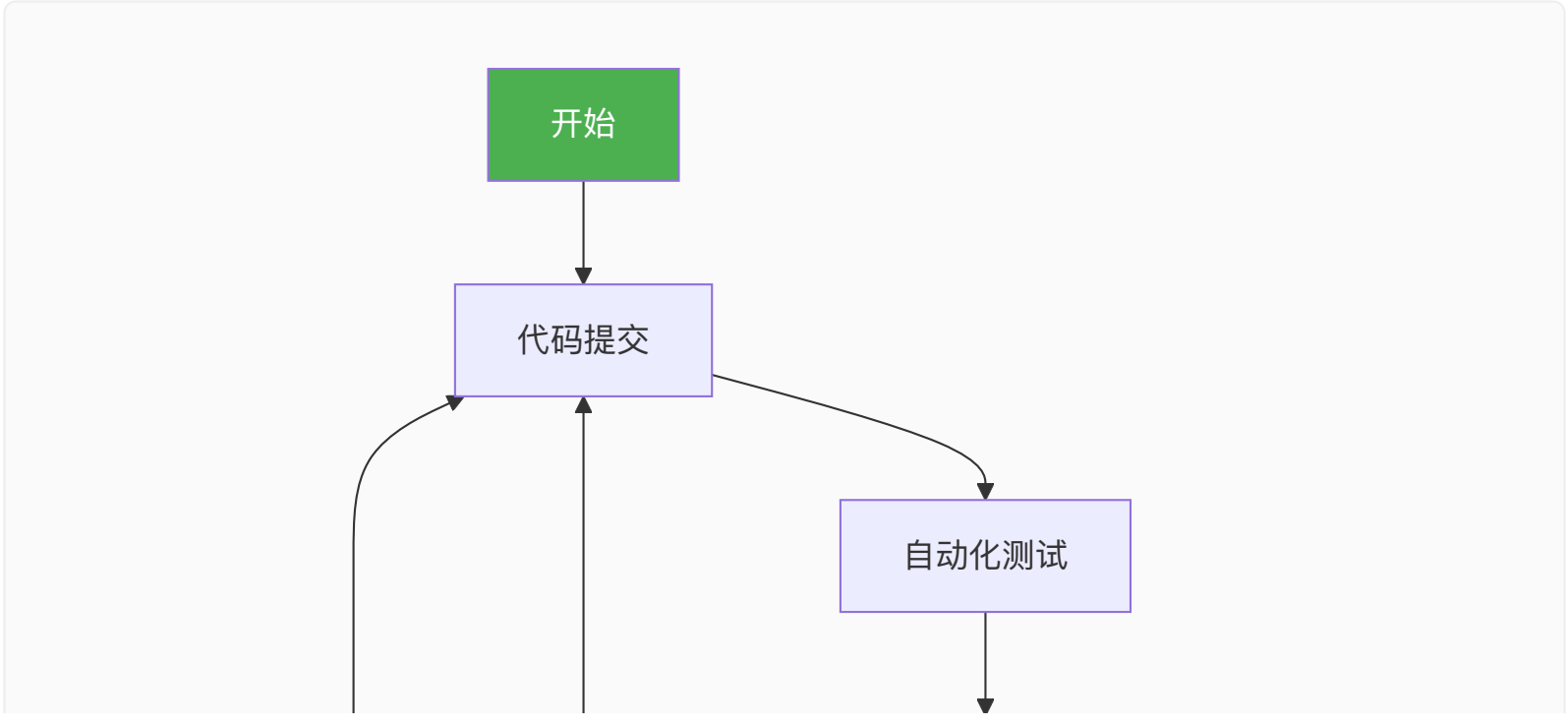
- Python 3.7+
- Django 3.2.25
- MySQL 5.7+
- Redis 5.0+
- Node.js 14+

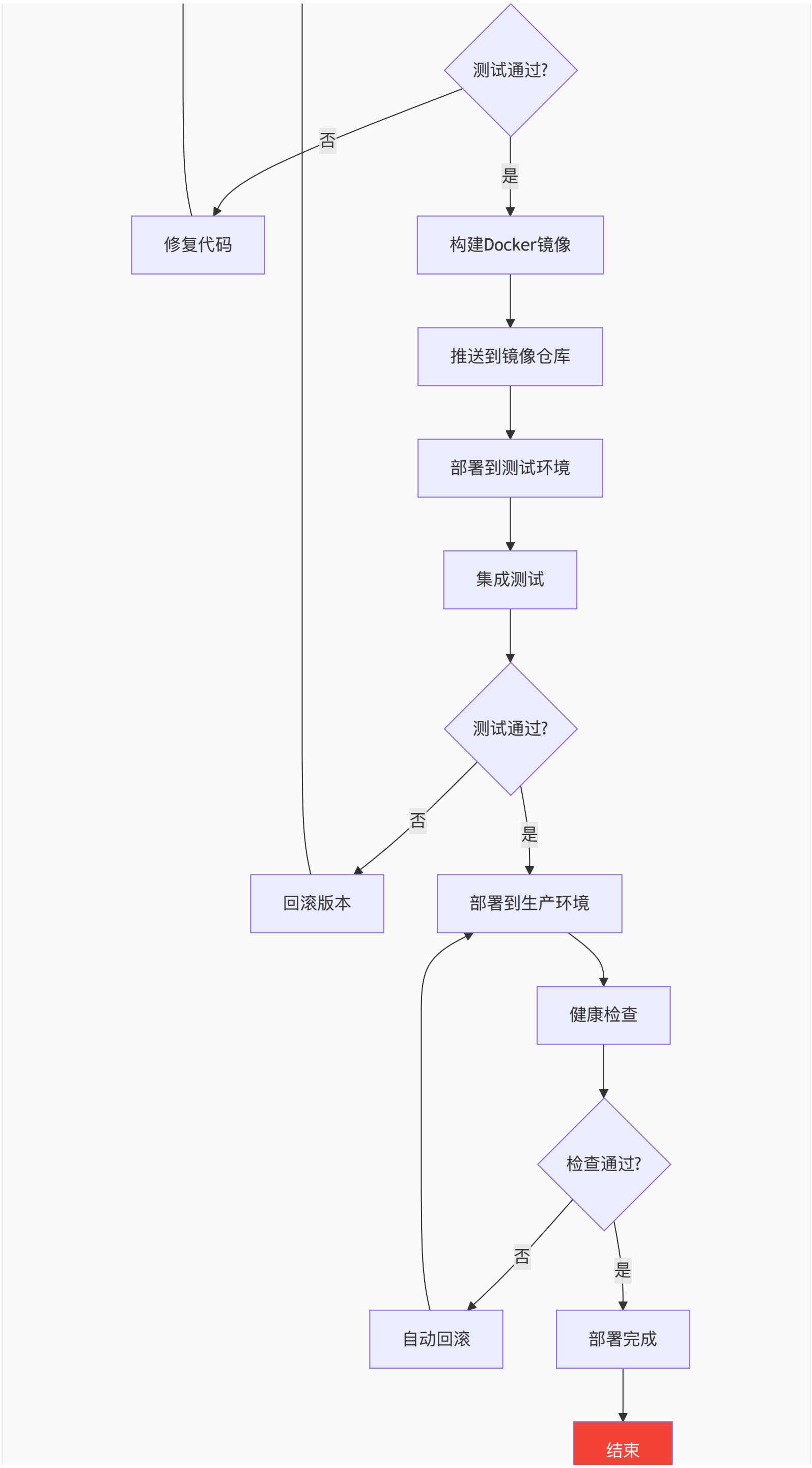
12.2.2 生产环境

- Ubuntu 20.04 LTS
- Nginx 1.18+
- uWSGI 2.0+
- MySQL 8.0+
- Redis 6.0+
- Docker & Docker Compose

12.3 部署流程

12.3.1 部署流程图




测试通过?
否
是
修复代码
构建Docker镜像
推送到镜像仓库
部署到测试环境
集成测试
测试通过?
否
是
回滚版本
部署到生产环境
健康检查
检查通过?
否
是
自动回滚
部署完成
结束

12.4 监控与运维

12.4.1 监控指标

- **系统监控**: CPU、内存、磁盘、网络
- **应用监控**: 请求量、响应时间、错误率
- **业务监控**: 订单量、注册量、交易额

12.4.2 日志管理

- 统一日志格式
- 日志分级存储
- 日志分析平台
- 异常告警机制

---

13. 附录

13.1 术语表

术语	英文	说明
SKU	Stock Keeping Unit	库存量单位
SPU	Standard Product Unit	标准产品单位
POS	Point of Sale	销售终端
RBAC	Role-Based Access Control	基于角色的访问控制
CSRF	Cross-Site Request Forgery	跨站请求伪造
XSS	Cross-Site Scripting	跨站脚本攻击
JWT	JSON Web Token	JSON网络令牌
ORM	Object-Relational Mapping	对象关系映射
API	Application Programming Interface	应用程序接口
WAF	Web Application Firewall	Web应用防火墙

13.2 参考文档

- Django官方文档: https://docs.djangoproject.com/
- Vue.js官方文档: https://vuejs.org/
- MySQL官方文档: https://dev.mysql.com/doc/
- Redis官方文档: https://redis.io/documentation
- Nginx官方文档: https://nginx.org/en/docs/

13.3 版本历史

版本	日期	修改人	修改内容
v2.0	2026-07-30	Kang Xuebing	完整版详细设计文档
v2.1	2026-11-01	GitHub Copilot	补充门店化POS、逐台收银终端、门店权限与库存、许可证运维及订单小票修复
v2.2	2026-11-01	GitHub Copilot	补充门店收银设备mTLS客户端证书身份认证、证书指纹绑定和可信代理部署设计

---

14. 2026年11月系统升级变更

本章记录2026年10月至11月期间在当前系统中落地的升级，作为原始v2.0设计的实施补充。原章节描述与本章不一致时，以本章所列的当前实现和部署迁移为准。本次升级包含门店级POS业务隔离、用户门店角色、逐台收银机配置、门店独立库存、门店小票、订单销量修复、mTLS客户端证书身份认证以及签名许可证运维约定。

14.1 升级范围总览

子系统	升级内容	主要实现位置
门店基础数据	门店增删改查、启停、门店选择	`pos.models.Store`、门店管理视图与模板
用户授权	用户与门店关联、店长/收银员/库管角色维护	`pos.models.UserStore`、用户门店管理视图与模板
收银终端与设备认证	每店配置多台收银机；每台绑定唯一编号、数据库IP及客户端X.509证书SHA-256指纹	`pos.models.StoreTerminal`、终端管理视图、mTLS反向代理
POS订单	POS订单、挂单、促销和扫码设备可关联门店	`POSOrder.store`、`SuspendedOrder.store`、`PromotionProduct.store`、`BarcodeScanner.store`
库存隔离	每店每SKU单独库存；初始化只创建缺失记录	`pos.models.StoreStock`、门店库存初始化视图
收据	显示订单实际关联门店，终端编号单独列出	收据预览模板、打印小票视图
业务授权	每个请求验证签名、服务器指纹和有效期	`hot_mall.license`、`LicenseMiddleware`
订单修复	SKU销量使用真实字段`sales`做原子递增	POS订单创建视图

14.2 门店级收银业务模型

1. **门店（Store）**：保存门店编号、门店名称、地址、电话、店长、营业时间和状态。门店编号唯一。
2. **用户门店关系（UserStore）**：建立用户和门店多对多关系的显式关联实体，并附带角色。`(user, store)`唯一，每个用户在不同门店可以有不同关联记录。
3. **门店收银终端（StoreTerminal）**：每台物理或逻辑收银机一条记录，包含归属门店、唯一终端编号、终端名称、数据库管理的IPv4/IPv6地址、客户端X.509证书SHA-256指纹、启停状态和审计时间。数据库只保存证书指纹，不保存证书私钥。
4. **门店库存（StoreStock）**：以门店和SKU为复合唯一业务键，记录库存、临期数量和过期数量；库存不再由不同门店共享一份可售数量。
5. **交易上下文**：POS订单和挂单均保存门店及终端编号；促销和条码扫描器可指定门店。促销门店为空表示全局促销。

14.3 管理界面与路由

管理员通过POS管理页面访问以下功能：

功能	路由名称	路径模式	权限
门店列表	`pos:store_list`	`/pos/store/`	超级管理员
新增/编辑门店	`pos:store_add` / `pos:store_edit`	`/pos/store/add/`、`/pos/store/edit/<pk>/`	超级管理员
门店终端列表	`pos:store_terminal_list`	`/pos/store/<store_pk>/terminals/`	超级管理员
添加终端	`pos:store_terminal_add`	`/pos/store/<store_pk>/terminals/add/`	超级管理员
编辑/删除终端	`pos:store_terminal_edit` / `pos:store_terminal_delete`	`/pos/store/<store_pk>/terminals/<pk>/edit/`、`.../<pk>/delete/`	超级管理员
用户门店关联	`pos:user_store_list`、`pos:user_store_add`、`pos:user_store_edit`、`pos:user_store_delete`	`/pos/user-store/...`	超级管理员
门店选择	`pos:select_store`	`/pos/select-store/`	已登录用户，按门店授权过滤
库存初始化	`pos:init_store_stock`	`/pos/init-stock/`	超级管理员

门店管理、用户门店权限和库存初始化页面返回用户中心（`users:info`）。门店列表的“终端配置”入口按门店打开终端清单，展示该店终端数量、编号和状态。

角色授权边界：当前 `UserStore.role` 支持分配和编辑店长、收银员、库管三种角色；普通用户的门店访问依据是否存在用户-门店关系判断。门店、用户关系、终端和库存初始化等管理页面仍由超级管理员权限保护。当前尚无按角色细分到每个操作的权限矩阵，不能仅依据角色名称假定具体业务动作已被服务端强制授权。

14.4 门店与终端选择及服务端校验

1. 收银员登录后访问 `/pos/cashier/` ；没有有效门店时重定向到门店选择页。
2. 普通用户的候选门店来自 `UserStore` 关系且必须启用；超级管理员可以访问任意启用门店。
3. 选择门店时服务端验证当前用户有权访问该店，再写入Session。所有后续POS请求重新验证Session门店仍启用且关联仍有效；否则清除门店Session并要求重新选择。
4. 收银台显示当前门店的启用终端。URL参数中的终端编号只有匹配当前门店选项时才能选中；没有登记终端时显示未配置状态，不生成开发默认编号。
5. 收银设备通过mTLS呈交设备客户端X.509证书。TLS终止代理持有受信任客户端CA，在TLS握手阶段验证证书链、有效期及客户端持有对应私钥的证明；仅对握手验证成功的连接转发至Django。
6. TLS代理清除外部客户端提交的证书相关同名头，并覆盖设置 `X-Client-Cert-Verify` 、 `X-Verified-Client-Cert` 和 `X-Verified-Client-IP` 。其中证书PEM需URL编码后转发，验证结果为 `SUCCESS` ，客户端IP由代理根据实际连接获得。
7. Django只信任 `POS_TRUSTED_TLS_PROXY_IPS` 中配置的代理对端地址。请求必须来自可信代理，代理验证状态必须为 `SUCCESS` ；服务端解析客户端证书PEM并计算DER编码SHA-256指纹，再按当前门店匹配启用终端的编号、数据库IP和证书指纹。
8. 终端缺少IP/证书指纹、代理未配置或不可信、证书未通过代理验证、请求证书/IP/编号不匹配时均失败关闭；不使用settings终端白名单或客户端直传证书指纹回退。
9. Django不直接信任 `X-Forwarded-For` 。Django应用端口必须仅允许可信代理访问，否则攻击者可绕过TLS握手直接伪造代理元数据。 `POS_TRUSTED_TLS_PROXY_IPS` 仅标识可提交已验证TLS元数据的代理，不是设备白名单。

14.5 门店库存初始化与销售规则

- 管理员在"库存初始化"页面选择启用门店，系统对所有SKU逐条执行 `get_or_create(store, sku)` 。
- 新记录从全局SKU库存字段复制初始值： `stock` 、 `near_expiry_num` 、 `expired_num` 。
- 已有门店库存不覆盖。因此重复初始化是幂等的，适用于为新门店分配初始数据，不是库存同步工具。
- 扫码遇到缺少门店库存记录的SKU时，创建零库存记录；不把全局库存悄悄复制到该店。
- 库存检查、结算可用量和库存扣减使用当前门店的StoreStock。可用数量为门店总库存扣除过期数量和临期数量，结算扣减采用数据库锁和条件更新保护并发。
- 销售完成后SKU销量统计通过 `SKU.objects.filter(...).update(sales=F('sales') + quantity)` 原子更新SKU的 `sales` 字段，不将 `sku__sales` 关联路径误作更新字段。

14.6 POS订单、挂单和小票

- 新POS订单写入当前门店外键、收银员、终端编号和客户端请求号。幂等请求由唯一 `client_request_id` 约束，校验同一收银员和终端后返回既有订单。
- 挂单和取单根据当前门店、收银员与终端编号隔离，避免跨门店或跨设备读取交易草稿。
- 门店级促销优先于全局促销；全局促销作为没有匹配门店促销时的候选。
- 收据预览和打印文本都从订单上的 `store` 外键读取门店编号、名称； `terminal_id` 只标注为终端编号。历史订单门店外键为空时打印"未关联门店"，不使用当前Session门店冒充订单门店。
- 门店编号和终端编号语义不同。例如订单记录门店 `STORE002` 、终端 `STORE001-POS01` 时，小票两项分别按订单实际字段显示，便于追溯设备和经营归属。

14.7 数据库迁移与升级顺序

门店及POS数据按迁移建立，当前POS迁移链包含门店、用户门店关联、门店库存、订单/挂单/促销/扫描器门店外键； `0021_storeterminal` 建立 `tb_store_terminal` ， `0022_storeterminal_ip_address` 增加终端IP字段， `0023_storeterminal_certificate_fingerprint` 增加客户端证书指纹字段。部署新版本时应先备份数据库，再执行：

```
python manage.py showmigrations pos
python manage.py migrate pos
python manage.py check
```

迁移增加门店终端表、IP和证书指纹字段。既有终端的IP/指纹字段为空；出于失败关闭原则，管理员登记有效IP和客户端证书指纹，并完成mTLS代理部署之前，这些终端不能发起POS请求。已有POS订单和挂单的`store_id`为可空，以保留历史数据；历史记录不能由当前选择门店自动补填，若需补录应基于原始交易凭据单独执行审核过的数据修复。

新建门店后，管理员应依次完成：创建门店、分配用户与角色、为每台收银机登记唯一终端编号、按业务需要初始化门店SKU库存、使用收银员账号登录并选店、在收银台选择终端、执行扫码和结算验证。

14.8 收银设备客户端证书与mTLS运维

设备证书信任模型

- 每台收银设备由企业内部CA或设备证书管理系统签发独立客户端X.509证书；设备持有对应私钥，并在mTLS握手时证明其私钥所有权。
- 私钥仅保存在收银设备或受控硬件密钥存储中。私钥不上传Django、不进入数据库、不随业务请求传输，也不得写入应用日志或代码仓库。
- `StoreTerminal.certificate_fingerprint`保存客户端证书DER编码的SHA-256摘要（64位十六进制）。系统不保存完整证书或私钥。
- 客户端证书由TLS反向代理根据受信任CA和证书有效期验证；Django对代理提供的证书PEM重新解析并计算SHA-256，再与当前门店终端记录指纹精确比较。
- 终端证书指纹重复由管理视图拒绝；当前模型没有数据库唯一索引，批量导入、脚本或并发管理程序仍需避免并发重复写入。

终端登记与证书部署

1. 为每台收银机独立签发客户端证书和私钥，验证证书用途适用于客户端认证。
2. 在终端管理页面登记门店、终端编号、固定IP和客户端证书SHA-256指纹，然后启用终端。
3. 将私钥和客户端证书通过受控设备管理渠道安装到收银机；只录入指纹，不上传私钥。
4. TLS反向代理配置服务器证书、可信客户端CA和强制客户端证书校验。
5. 代理在成功验证TLS证书后覆盖设置`X-Client-Cert-Verify: SUCCESS`、URL编码的`X-Verified-Client-Cert`和`X-Verified-Client-IP`。
6. Django环境中的`POS_TRUSTED_TLS_PROXY_IPS`仅配置固定代理地址或受控CIDR。防火墙/安全组必须禁止非代理主机访问Django upstream。
7. 验收正确证书、终端编号和IP请求可用；替换证书、错误证书、错误IP、伪造证书验证头或绕过代理的请求均被拒绝。

客户端证书指纹可从CA实际签发的PEM客户端证书计算。`pos-terminal-client.crt`是占位文件名，使用前必须替换为真实证书路径；项目不会自动生成收银设备证书。先确认文件可读且证书可解析，再计算指纹：

```
CLIENT_CERT="/path/to/issued/pos-terminal-client.crt"
if [ ! -r "$CLIENT_CERT" ]; then
    printf '客户端证书不存在或不可读: %s\n' "$CLIENT_CERT" >&2
    exit 1
fi
openssl x509 -in "$CLIENT_CERT" -noout >/dev/null || exit 1
openssl x509 -in "$CLIENT_CERT" -outform DER | openssl dgst -sha256
```

只将成功输出的SHA-256值录入终端配置。若证书读取或解析失败，不得使用管道仍可能输出的空输入摘要（例如`e3b0...`）；设备私钥不得进入该管理页面。

证书轮换与应急吊销

- **计划轮换**：签发新客户端证书，在维护窗口更新终端数据库指纹并向设备部署新证书，验证新证书可以收银后撤销旧证书。切换过程应设计成短时维护窗口，避免同一终端同时接受两个证书指纹。
- **遗失或泄露**：先停用对应`StoreTerminal`，再由CA/代理侧吊销证书；完成事件调查后签发新设备证书、更新指纹

并重新启用终端。

- **终端退役**：停用或删除终端记录，并吊销客户端证书。删除数据库记录不等同于CA证书吊销。
- **代理或CA变更**：先更新可信客户端CA、验证链和代理来源IP，再使用独立验收设备验证握手、请求头覆盖和Django实际观察到的 `REMOTE_ADDR` 。

代理信任边界与配置责任

`POS_TRUSTED_TLS_PROXY_IPS` 是当前mTLS实现新增的唯一POS安全环境变量，配置由Django TCP连接实际看到的TLS代理地址。它不存储终端编号、终端IP或证书指纹。代理配置必须覆盖而不是透传外部请求中的 `X-Client-Cert-Verify` 、 `X-Verified-Client-Cert` 、 `X-Verified-Client-IP` ；否则客户端可能伪造TLS已验证状态或证书材料。

应用当前按 `REMOTE_ADDR` 识别可信代理，并从代理转发的PEM证书计算指纹。反向代理、负载均衡、容器网络或多级代理部署需要确保代理链地址准确；不能把可被客户端任意设置的 `X-Forwarded-For` 直接作为信任依据。上线前应在真实代理链路做端到端验收。

14.9 签名许可证与本地开发运维

- 开发和生产settings均通过 `LICENSE_FILE` 、 `LICENSE_PUBLIC_KEY` 或 `LICENSE_PUBLIC_KEY_FILE` 指定证书和验证公钥； `LicenseMiddleware` 在请求进入业务视图前执行验证。
- 证书必须具备合法Ed25519签名、必要payload字段、匹配的 `server_fingerprint` 和有效的 `not_before` / `expires_at` 时间范围。失败请求返回HTTP 403和错误原因；不能通过直接编辑证书JSON或跳过中间件修复签名/指纹问题。
- 服务器指纹变化时，先确认运行机器和配置的许可证路径，再执行 `python manage.py license fingerprint` 取得当前指纹，由许可证签发方使用受控私钥签发匹配该机器的新证书；部署证书和公钥后再验证登录页。
- 私钥仅保存在受控签发环境。不得将生产签发私钥放进应用仓库、部署包、日志或共享目录；本地许可证目录应受版本控制忽略规则保护。更换机器、容器基底或影响机器标识的运行环境时，应按许可证流程重新签发，而不是关闭授权验证。

14.10 验证记录与回归范围

升级验收使用针对性自动化测试覆盖门店权限和Session失效、用户门店角色编辑、收银台终端选择、数据库IP匹配/不匹配、客户端证书指纹匹配/不匹配、未验证证书拒绝、非可信代理拒绝、伪造证书头拒绝、重复指纹拒绝、错误终端拒绝、终端停用拒绝、库存初始化不覆盖、订单小票门店与终端分列。执行Django system check与迁移一致性检查，确保模型和迁移一致。

最近验证命令示例：

```
python manage.py test pos.tests
python manage.py check
python manage.py makemigrations --check --dry-run pos
```

本次mTLS相关回归测试使用临时内存生成的测试证书，不在测试目录生成或保存客户端私钥文件；覆盖正确证书放行、错误/伪造证书拒绝、客户端IP不符拒绝、非可信代理拒绝、终端停用拒绝及重复指纹拒绝。

POS终端设置发生变化后，应重启Web进程并重新选择当前门店的终端；订单归属以服务端保存的 `POSOrder.store_id` 为准，不能仅凭浏览器标题或终端编号判断门店。

文档结束